Sistemi operativi

05/03/2026

Il sistema operativo è tecnicamente di basso livello, in quanto gestisce processi molto vicini all'hardware

La cpu ha sempre due modalità, user mode e kernel mode

Il bios fa "parte" dell'hardware in un certo senso, un software diverso dal sistema operativo

Kernel: Blocco caricato in ram che prende il controllo del sistema e permette di usare il sistema operativo

Il sistema operativo crea astrazione, ha istruzioni e documentazione per capire come utilizzaare hardware

Per esempio il "file" è un'astrazione offerta dal sistema operativo, non esiste in realta, fisicamente c'è solo di disco

Cose come aprire file, leggere e scrivere sono chiamate dirette al sistema operativo, in quanto il file potrebbe anche essere in realtà divisi in mille pezzi sparsi per tutto il disco, cosa che invece il sistema operativo semplifica facendoci vedere solo l'entità "file"

Parallelismo altro esempio di astrazione, vengono create delle "cpu virtuali"

Tutti gli spazi di memoria da gestire operano multiplexing nello spazio

24/03/2026

Intro

Si supera il test (senza voto, solo ammesso o non ammesso)
Non si chiedono strutture dati complesse o algoritmi complessi (la cosa più complessa chiesta è stata l'ordinamento).

Domande a risposta aperta nel test
Testo principale: sufficiente per l'esame (+ lezioni)
Consiglio: usare macchina virtuale con una qualunque distribuzione linux.
Al lab: troviamo shell con gcc.

Lezione 1

Generalità

Sistema operativo: software che gestisce l'hardware. Si frappone come struttura intermedia, tramite, tra i suoi utenti e l'hw stesso. Gli utenti del sistema operativo sono i programmi in esecuzione sul calcolatore (processi).

Sistema operativo: alto o basso livello? Dipende dal punto di vista. Dal punto di vista degli utenti, basso livello. Dal punto di vista dell'hw, alto livello.

Modalità di esecuzione della CPU

Ad eccezione di sistemi molto semplici (es. alcuni tipi di microcontrollori), ogni CPU moderna supporta almeno due modalità di esecuzione:

1. Kernel mode (o supervisor mode): il codice in esecuzione ha pieni poteri: può eseguire qualsiasi istruzione supportata dalla CPU, accedere liberamente all'hardware e modificare le strutture di controllo del sistema;
2. User mode: il codice in esecuzione è limitato: può eseguire la grande maggioranza delle istruzioni normali, ma alcune operazioni - dette operazioni privilegiate - gli sono precluse.

Cosa sono le operazioni privilegiate?

Sono operazioni che, se accessibili a qualsiasi programma, comprometterebbero la stabilità o la sicurezza dell'intero sistema. Un esempio

chiave è la configurazione della MMU (Memory Management Unit), un componente hardware integrato nella CPU.

La MMU è il meccanismo che impedisce a un processo di accedere arbitrariamente a zone di memoria che non gli appartengono. Questo è possibile perché ogni processo opera nel proprio spazio di indirizzamento virtuale, isolato da quello degli altri processi. Questi sono i cosiddetti spazi compartimentali.

Configurare i “paletti” della MMU - cioè definire i limiti e i permessi degli spazi di indirizzamento - è un’operazione privilegiata, eseguibile solo in kernel mode. Se qualsiasi processo utente potesse farlo, potrebbe tranquillamente leggere o scrivere nella memoria di un altro processo, violando l’isolamento che il sistema operativo garantisce.

SO: entità a sé stante che gestisce gli altri processi. Ovviamente deve potersi difendere dagli altri processi, quindi ogni processo deve essere isolato.

Kernel mode, user mode e sistema operativo

Una prima suddivisione intuitiva potrebbe essere:

- kernel mode → codice del sistema operativo
- user mode → codice di processi utente, estranei al SO

In realtà questa distinzione non è così netta: esistono componenti che logicamente fanno parte del sistema operativo ma vengono eseguite in user mode (es: da completare con alcuni esempi). La modalità di esecuzione e l’appartenenza al SO sono quindi due dimensioni separate.

Sequenza di avvio: da zero al kernel

Il BIOS è un piccolo software residente in una memoria non volatile sulla scheda madre, strettamente integrato con l’hardware. È al di fuori del normale schema kernel/user mode - viene eseguito prima ancora che esista un sistema operativo. Il suo compito è essenzialmente duplice:

1. Inizializzare l’hardware (CPU, RAM, dispositivi di I/O)
2. Caricare in RAM il bootloader, un piccolo programma residente in una posizione nota sul disco

Bootloader: è un programma minimale il cui unico scopo è:

1. Individuare il kernel sul disco

2. Caricarlo in RAM

3. Passargli il controllo

Una volta che il bootloader gli cede il controllo, il kernel - che è il nucleo del sistema operativo - prende possesso dell'hardware. Da questo momento in poi è il kernel a gestire tutte le risorse del sistema: CPU, memoria, dispositivi, e l'esecuzione di tutti i processi successivi.

SO: entità che crea astrazioni. L'hw, qualcunque hw, ha una sua documentazione utile per capire in che modo deve essere usato quell'hw.

Le astrazioni del sistema operativo

Il tema centrale di questa sezione è l'astrazione: il sistema operativo nasconde la complessità dell'hardware e offre interfacce più semplici e ad alto livello. Vediamo i due esempi principali.

Astrazione 1: il file system

I dischi fisici non conoscono il concetto di "file": operano per blocchi di dimensione fissa (tipicamente $1\ \mathrm{K}, 2\ \mathrm{K}, 4\ \mathrm{K}, 8\ \mathrm{K}$ byte). Tutto ciò che il disco sa fare è leggere e scrivere questi blocchi grezzi.

Il concetto di file è un'astrazione introdotta dal sistema operativo. Il kernel studia le specifiche hardware, scrive i driver necessari, e costruisce sopra di essi un'interfaccia comoda e ad alto livello. Dietro le quinte, un singolo file può essere frammentato su blocchi sparsi in punti diversi del disco - ma tutto questo è invisibile al programmatore, che interagisce semplicemente con un nome e un contenuto.

System call

Ogni volta che un programma vuole accedere a un file, invoca una chiamata di sistema (syscall). In quel momento:

1. La CPU passa da user mode a kernel mode
2. Il kernel esegue una serie di controlli sui permessi
3. Il kernel svolge l'operazione richiesta e restituisce il risultato al processo

Nota: qualunque system call viene sempre eseguita in kernel mode.

Astrazione 2: la CPU virtuale (e i processi)

Il sistema operativo dà a ogni processo l'illusione di avere una CPU dedicata tutta per sé. Che il parallelismo sia reale (multi-core) o simulato (single-core con time-sharing), il programmatore non deve occuparsene - il kernel gestisce tutto in modo trasparente.

Fisicamente esiste una sola CPU (o un numero limitato di core), ma il sistema operativo crea l'illusione che ogni processo disponga di una CPU virtuale tutta sua, completa di registri propri. In realtà il kernel gestisce N processi su una CPU fisica, distribuendo il tempo di esecuzione tra di essi in modo trasparente.

L'obiettivo è sempre lo stesso: sollevare il programmatore dalla complessità. Chi scrive un programma non deve preoccuparsi di condividere la CPU con altri processi - si comporta come se la macchina fosse interamente a sua disposizione.

Cosa serve per eseguire un processo?

Ogni programma, per essere eseguito, ha bisogno del proprio codice e dei propri dati, che devono risiedere in RAM. Questa affermazione è però solo parzialmente vera - vedremo in seguito perché non è sempre necessario che tutto si trovi in RAM contemporaneamente.

Nota del Corso: approfondiremo ognuna di queste astrazioni in modo più tecnico più avanti.

Gestione delle risorse: multiplexing

Gestire una risorsa significa fornirne un utilizzo ordinato e coordinato tra tutti i processi che ne hanno bisogno. Esistono due approcci fondamentali.

Multiplexing nel tempo

La risorsa è unica e può essere usata da un solo processo per volta: il kernel la assegna in modo esclusivo, suddividendo il tempo tra i vari richiedenti.

CPU: è l'esempio principale. Il kernel assegna la CPU a un processo per un breve intervallo di tempo, poi la toglie e la assegna al successivo. Per farlo in modo trasparente, deve occuparsi del context switch:

- Salvataggio del contesto: quando un processo viene interrotto, il kernel salva in memoria RAM i valori dei registri su cui stava lavorando

- Ripristino del contesto: quando il processo torna in esecuzione, quei valori vengono ricaricati nella CPU

L'unica cosa che fisicamente si sposta dalla CPU alla RAM sono quindi i valori dei registri. Da notare che anche il kernel stesso usa la CPU per svolgere questo lavoro, eseguendo in kernel mode per il tempo necessario.

Stampante: è un esempio di multiplexing nel tempo con una logica diversa. Non è possibile alternare tra processi nel mezzo di un lavoro di stampa - farlo produrrebbe un output mescolato e inutilizzabile. Il kernel quindi funge da coordinatore: chi inizia a stampare un documento deve completarlo prima che la risorsa venga ceduta al processo successivo.

Multiplexing nello spazio

In questo caso la risorsa viene suddivisa fisicamente tra i vari processi, ognuno dei quali ottiene una porzione dedicata.

RAM

Ogni processo ha bisogno di un proprio spazio in RAM per codice e dati. L'approccio più intuitivo è dividere la RAM in blocchi contigui assegnati staticamente (es. SO, P1, P2, P3). Nei sistemi moderni però si usa la memoria virtuale, che funziona diversamente:

- Lo spazio di ogni processo viene frammentato e distribuito in RAM dove è disponibile, senza necessità di blocchi contigui
- Ogni processo ha l'illusione di uno spazio di indirizzamento contiguo e dedicato, ma fisicamente i suoi dati possono essere sparsi ovunque

Questo approccio offre un vantaggio fondamentale: un processo che cresce dinamicamente non ha bisogno di trovare uno spazio contiguo abbastanza grande - il kernel gli assegna semplicemente nuovi frammenti dove ci sono. Il prezzo da pagare è una gestione più complessa, di cui parleremo quando affronteremo la memoria virtuale nel dettaglio.

Chiamate di sistema e thread

Operazioni come open, read, write in C sono system call che il programma indirizza al kernel, il quale le esegue per conto del processo (ad esempio per accedere al file system).

Thread e distinzione tra processo e thread

Un processo è un programma in esecuzione. Ha il suo spazio di memoria, i suoi dati, il suo codice - e il kernel gli dà l'illusione di una CPU dedicata. Per default, un processo ha un solo flusso di esecuzione: le istruzioni vengono eseguite una dopo l'altra, in sequenza. Se devo leggere un file e poi elaborarlo, prima finisco di leggere, poi elaboro.

Un thread è un flusso di esecuzione aggiuntivo all'interno dello stesso processo. Se ne creo uno, il processo ha due flussi che avanzano "contemporaneamente" (con la stessa logica di prima - realmente in parallelo su multi-core, oppure in time sharing su single-core).

La differenza cruciale rispetto a due processi separati è che i thread di uno stesso processo condividono la stessa memoria: stessi dati, stesso heap, stesso codice. Non hanno spazi isolati come i processi. Quindi:

- processo → programma isolato con la sua memoria
- thread → flusso di esecuzione dentro un processo, memoria condivisa con gli altri thread dello stesso processo

È possibile chiedere al sistema operativo di creare più flussi di esecuzione all'interno dello stesso programma. Ogni thread esegue una procedura diversa, ma tutti condividono lo stesso spazio di memoria del processo. Questo permette, ad esempio, di leggere un file e contemporaneamente svolgere un'altra elaborazione.

Nel codice useremo gli standard POSIX e la libreria pthreads, che garantiscono portabilità: il codice è compilabile su tutte le principali piattaforme. Windows è solo parzialmente compatibile con POSIX, ma la
maggior parte delle cose che vedremo funzionano anche lì, eventualmente tramite WSL 2.

Breve storia dei sistemi operativi

I calcolatori nascono come pure macchine hardware, inizialmente meno potenti di una moderna calcolatrice da tavolo. L'evoluzione tecnologica ha attraversato diverse generazioni:

1. Valvole a vuoto - ingombranti, calde, inaffidabili
2. Transistor - più piccoli, veloci e affidabili
3. Circuiti integrati - migliaia di transistor su un singolo chip

4. CPU moderne - miliardi di transistor in spazio ultracompatto

Man mano che i sistemi diventavano più potenti e commercialmente distribuiti, nacque la necessità di offrire un'interfaccia utilizzabile. Con i primi sistemi operativi si esplorarono tecniche fondamentali come la multiprogrammazione e il time sharing (la suddivisione del tempo di CPU tra più processi). Con i primi sistemi Apple e successivamente con Windows nacquero le interfacce GUI.

Scrivere un sistema operativo da zero è estremamente complesso, quindi nella pratica quasi tutti i sistemi operativi moderni nascono come modifiche o derivazioni di sistemi preesistenti (Linux ne è l'esempio più emblematico).

Richiami di architettura degli elaboratori

Architettura della CPU

La CPU è collegata alle periferiche tramite bus (databus, controlbus, addressbus). Il suo compito fondamentale può essere astratto come l'esecuzione di una sequenza di istruzioni secondo il ciclo fetch → decode → execute:

1. Fetch: preleva l'istruzione dalla memoria all'indirizzo indicato dal Program Counter
2. Decode: la interpreta
3. Execute: la esegue

Registri principali

Registri generici

Utilizzati per operazioni aritmetiche, logiche e di trasferimento dati. Possono avere dimensioni diverse: 16, 32 o 64 bit. Sono liberamente leggibili e scrivibili - non sono istruzioni privilegiate.

Program Counter (PC)

Contiene l'indirizzo di memoria della prossima istruzione da eseguire. Viene aggiornato automaticamente ad ogni ciclo, ma può essere modificato esplicitamente (es. nei salti).

Stack Pointer (SP)

Punta alla cima dello stack. Viene gestito automaticamente da istruzioni come call e return, ma può essere modificato manualmente se necessario.

Program Status Word (PSW)

Registro speciale che rappresenta lo stato corrente della CPU tramite una collezione di bit (flag). Alcuni esempi:

- Flag di confronto: memorizzano l'esito dell'ultima operazione di confronto (es. maggiore, uguale, minore). Liberamente leggibili e modificabili.
- Mode bit: indica la modalità di esecuzione corrente - 0 per user mode, 1 per kernel mode (supervisor). La modifica di questo bit è un'operazione privilegiata.

Istruzioni privilegiate e istruzioni illegali

Quando la CPU incontra un'istruzione da eseguire, controlla il mode bit della PSW:

- Se è a 1 (kernel mode) → l'istruzione viene eseguita normalmente
- Se è a 0 (user mode) e l'istruzione è privilegiata → la CPU rifiuta l'esecuzione, interrompe il processo e passa il controllo al sistema operativo

Questo meccanismo si applica non solo alle istruzioni privilegiate, ma a tutte le istruzioni illegali, ovvero operazioni non consentite nel contesto corrente. Esempi:

- Tentativo di eseguire un'istruzione privilegiata in user mode
- Tentativo di accedere alla memoria di un altro processo
- Divisione per zero (non privilegiata, ma comunque illegale)

In tutti questi casi il controllo viene ceduto al SO, che decide come gestire la situazione (tipicamente terminando il processo con un errore).

Transizione tra modalità

- Da kernel a user: operazione libera. Il kernel può in qualsiasi momento abbassare la modalità impostando il mode bit a 0 . È lui che decide quando "declassarsi".

- Da user a kernel: non può avvenire arbitrariamente - vedremo nella prossima lezione il meccanismo preciso con cui un processo utente può richiedere il passaggio in kernel mode in modo controllato.

CPU superscalare

Le CPU moderne non eseguono istruzioni strettamente una alla volta. In un'architettura superscalare, la CPU dispone di multiple unità di esecuzione specializzate (aritmetica intera, virgola mobile, accesso memoria, ecc.). Le fasi di fetch e decode sono gestite in unità multiple che confluiscono in un buffer interno, da cui le unità di esecuzione prelevano e processano più istruzioni in parallelo, aumentando il throughput complessivo (numero di istruzioni eseguite al secondo).

Lezione 2

Funzioni di libreria vs chiamate di sistema

- Funzione di libreria: semplice call a un indirizzo in memoria, eseguita interamente in user mode
- Chiamata di sistema (syscall): richiesta di un servizio offerto dal kernel, eseguita in kernel mode. Il meccanismo che la rende possibile è la TRAP

TRAP

La TRAP è un'istruzione macchina (come mov, add, ecc.) che il compilatore inserisce nel codice binario del processo al posto della chiamata di sistema. Ad esempio, una open in C viene tradotta in un'istruzione TRAP con un codice numerico associato (es. 115).

Quando la CPU incontra la TRAP durante l'esecuzione:

1. La CPU passa da user mode a kernel mode
2. Il controllo viene ceduto al sistema operativo
3. Il kernel esegue le verifiche necessarie (il file esiste? il processo ha i permessi?)
4. Il kernel svolge l'operazione richiesta
5. La CPU torna in kernel mode → user mode e il processo riprende

La TRAP viene eseguita normalmente all'interno della pipeline, come qualsiasi altra istruzione. È il meccanismo controllato con cui un processo utente può richiedere servizi al kernel.

Interrupt hardware

Un interrupt hardware è il meccanismo con cui una periferica notifica un evento alla CPU, interrompendone l'esecuzione corrente. È un evento asincrono: non dipende da ciò che la CPU stava facendo.

Quando arriva un interrupt:

1. La CPU salva il contesto minimo necessario (PC e PSW) nello stack corrente
2. Consulta la tabella degli interrupt (una tabella di indirizzi di memoria [dove si trova il codice che dovrà essere eseguito], uno per tipo di interrupt)
3. Salta al codice del kernel associato a quell'interrupt, eseguendolo in kernel mode
4. Al termine, ripristina PC e PSW e riprende l'esecuzione precedente

Se l'interrupt arriva mentre la CPU era già in kernel mode, il ripristino della PSW mantiene semplicemente la modalità kernel - non cambia nulla di sostanziale.

Interrupt da eventi eccezionali

Alcuni interrupt non provengono dall'hardware esterno ma da errori nell'esecuzione stessa, come:

- Divisione per zero
- Istruzione illegale (es. operazione privilegiata in user mode, accesso a memoria altrui)

In questi casi l'istruzione problematica scatena l'interrupt. Sono errori fatali: il kernel non può fornire un risultato corretto e tipicamente termina il processo.

Multithreading hardware (Hyperthreading)

Il multithreading hardware (o hyperthreading) è un'ottimizzazione della singola CPU, non va confuso con il multiprocessing.

Il problema che risolve: anche con pipeline avanzate, la CPU ha tempi morti mentre aspetta il completamento di operazioni lente (es. accessi in memoria). In

quel momento l’unità di elaborazione è ferma.

La soluzione: dotare la CPU di due set di registri (due contesti) con un’unica unità di elaborazione. Il SO carica due processi, uno per set. Quando il processo P1 genera un tempo morto, la CPU commuta autonomamente e a latenza zero sul contesto di P2, e viceversa.

Alcune precisazioni importanti:

- Non è vera parallelizzazione: in ogni istante la CPU sta elaborando un solo contesto
- Dal punto di vista del software, la CPU si presenta come due CPU logiche
- Il guadagno in prestazioni è reale ma modesto - non bisogna aspettarsi un raddoppio delle performance
- Due set di registri sono sufficienti: aggiungerne un terzo complicherebbe l’hardware senza benefici apprezzabili

Nota: per CPU single-thread (quindi con un solo set di registri) nella pratica si hanno tanti set di registri quanti sono i processi, grazie all’astrazione fornita dal SO, che illude i processi di avere un set di registri tutto per loro.

PCB (Process Control Block): è una struttura dati che il kernel mantiene in RAM per ogni processo attivo. Viene considerata come la “carta di identità del processo”: contiene i valori dei registri del processo (es: PC, PSW), lo stato del processo, il PID etc.

Sistemi multi-core e multi-CPU

Perché più core invece di frequenze più alte?

Aumentare la frequenza (GHz) di una singola CPU oltre certi limiti è controproducente: i problemi fisici (calore, consumo energetico, instabilità) superano i guadagni in prestazioni. La strada seguita dall’industria è stata invece quella di pipeline più efficienti e più core.

Multi-core vs multi-CPU

- Multi-CPU: più chip fisici distinti sulla scheda madre, ognuno con la propria unità di elaborazione
- Multi-core: un singolo chip che contiene più unità di elaborazione al suo interno

I multi-core consumano meno energia e disperdono meno calore. Dal punto di vista del sistema operativo e del programmatore, la distinzione è irrilevante: in entrambi i casi si ha a disposizione più unità di elaborazione indipendenti.

Attenzione al parallelismo reale

Con N core non si ottiene un incremento di prestazioni N volte. Il motivo è che le risorse - RAM e storage in primis - sono condivise tra tutti i core. Questa condivisione introduce contese e tempi di attesa che erodono il guadagno teorico.

Hyperthreading e multi-core insieme

Se ho 2 CPU fisiche con hyperthreading, ho 4 contesti: C1A, C1B, C2A, C2B. Il SO deve assegnare i processi in modo consapevole: mettere due processi su C1A e C1B significa caricarli entrambi sulla stessa CPU fisica, sprecando la seconda. La scelta ottimale è C1A e C2A (o C2B) - ovvero distribuire i processi su CPU fisiche diverse.

GPU: perché non la trattiamo

Le GPU dispongono di centinaia di unità di elaborazione specializzate per il parallelismo massivo. Tuttavia il loro utilizzo non è gestito dal sistema operativo, ma è demandato direttamente alle applicazioni che scelgono di sfruttarlo (es. tramite CUDA o OpenCL). Per questo motivo non rientra nei temi di questo corso.

Gerarchia della memoria

Le memorie si dispongono in una gerarchia, ordinate per velocità decrescente (e costo per bit decrescente):

Registri → Cache → RAM → SSD → Disco magnetico → Disco ottico Più si scende nella gerarchia, più la memoria è lenta, economica e capiente.

Registri

I registri rappresentano la memoria con la velocità maggiore, con la minor capacità e con il più alto costo per bit rispetto agli altri strati della memoria di un elaboratore. Sono veloci quanto la CPU, quindi accedervi non comporta alcun ritardo. La capacità di memorizzazione al loro interno è nell’ordine di $32

\times$ 32 bit in una CPU a 32 bit, e di 64×64 bit in una CPU a 64 bit (in entrambi i casi meno di 1 KB).

Nota: una CPU a 32 bit ha registri da 32 bit, una a 64 bit ha registri da 64 bit. Quindi con una CPU a 64 bit le operazioni sono più efficienti. Un'altra differenza fondamentale tra CPU a 32 bit e CPU a 64 bit è lo spazio di indirizzamento: $2^{\wedge} 32$ indirizzi possibili per la CPU a 32 bit, quindi circa 4 GB di RAM indirizzabile massima, e $2^{\wedge} 64$ per le CPU a 64 bit.

Cache

La cache è una memoria intermedia tra i registri e la RAM. Il suo scopo è ridurre il collo di bottiglia causato dalla lentezza della RAM rispetto alla CPU.

La cache lavora per **linee di cache (**tipicamente da 64 byte): blocchi di dati prelevati dalla RAM e tenuti temporaneamente in cache. Quando la CPU richiede un dato:

- Cache hit: il dato è già in cache → accesso velocissimo
- Cache miss: il dato non è in cache → viene prelevata un'intera linea di cache dalla RAM. Il vantaggio è che i dati vicini a quello richiesto vengono portati in cache insieme, sfruttando il principio di località: è probabile che la CPU richieda presto dati nella stessa zona di memoria.

Quando la cache è satura e bisogna fare spazio, il sistema sceglie quale linea eliminare tramite appositi algoritmi di rimpiazzamento - gli stessi che rivedremo quando affronteremo la memoria virtuale.

La gestione della cache è una scelta completamente hardware, trasparente al sistema operativo.

Livelli di cache

- L1: la più veloce, lavora a strettissimo contatto con i registri, dato che si trova all'interno della CPU. Pochi kilobyte, dedicata a ogni core.
- L2 (e L3): più capiente ma più lenta. Può essere dedicata a ogni core oppure condivisa tra più core.

Problema di coerenza nelle cache condivise

Nei sistemi multi-core, se due core condividono una stessa linea di cache e uno dei due la modifica localmente, sorge un problema di coerenza: l'altra copia non è più aggiornata. È necessario un meccanismo di sincronizzazione che propaghi le modifiche - questo è uno dei problemi fondamentali del parallelismo, apparentemente semplice ma in realtà delicato.

La cache condivisa può diventare un collo di bottiglia, dato che più core tentano di accedervi e ci possono essere problemi di coerenza. Vantaggi: se un processo migra da un core all'altro, trova i suoi dati già in cache, e la capacità totale della cache è usata in modo più flessibili (ci sono meno "limiti" di spazio per i processi rispetto al modello con cache dedicata).

RAM (memoria principale)

La RAM è la memoria più capiente tra quelle che lavorano direttamente con la CPU. Ha però due limiti fondamentali:

- Capienza limitata: non è abbastanza grande da contenere tutti i dati di tutti i programmi contemporaneamente
- Volatilità: il suo contenuto si azzera allo spegnimento

Per questo è necessaria una memoria non volatile e capiente: la memoria secondaria.

ROM: Read-Only Memory

La ROM è una memoria non volatile e non modificabile (o modificabile solo in condizioni speciali). Serve per memorizzare dati che non devono mai cambiare e che devono essere disponibili immediatamente all'accensione. Un esempio di applicazione è il BIOS: questo risiede in una ROM (o in una sua variante moderna) sulla scheda madre.

Memoria secondaria

Dal punto di vista del software, la memoria secondaria è indirizzabile come una sequenza di blocchi di dimensione fissa, che il SO può leggere e scrivere. È il livello su cui vengono costruite le astrazioni come i file.

Dischi magnetici (HDD)

Un HDD è un tipo di memoria magnetica non volatile, composta da una serie di dischi concentrici impilati, con superfici magnetizzabili su cui una testina legge e scrive i dati. Per individuare un blocco servono tre coordinate:

1. Cilindro: su quale traccia posizionare il pettine delle testine → richiede un movimento fisico meccanico
2. Testina: su quale superficie (quale disco della pila) si trova il dato (cioè sopra/sotto)
3. Settore: quale porzione del cerchio - si aspetta che la rotazione del disco porti il settore corretto sotto la testina

Questo introduce due latenze principali:

- Seek time: tempo di posizionamento meccanico della testina sul cilindro giusto
- Latenza rotazionale: tempo di attesa che il settore desiderato passi sotto la testina

Sono queste latenze meccaniche il motivo per cui gli HDD sono lenti rispetto alle memorie elettroniche.

SSD

Le SSD sono memorie elettroniche non volatili (tecnologicamente simili alla RAM ma persistenti). Non hanno parti meccaniche in movimento, quindi eliminano completamente le latenze degli HDD. Rappresentano la direzione verso cui si sta migrando.

Dischi Ottici

Seppur oggi siano quasi obsoleti per uso generale, i dischi ottici sono un tipo di memoria ottica non volatile, che sfrutta la tecnologia dei laser.

Controller dei dischi

Il controller è una piccola unità di elaborazione dedicata - un mini-calcolatore collegato al calcolatore principale - dotato di una propria CPU, buffer interni e porte di I/O. Il suo compito è pilotare i dispositivi hardware, e ne esistono di diversi tipi: controller dei dischi, della scheda grafica, della tastiera, della rete e così via.

Perché non ci pensa direttamente la CPU principale?

Gestire un disco meccanico richiede precisione temporale estrema: bisogna essere pronti a fermare il motore esattamente nella posizione corretta, al momento giusto. In un sistema multiprogrammato, la CPU è costantemente impegnata in molti compiti contemporaneamente e non può garantire questo livello di precisione. Delegare questo lavoro a un controller dedicato risolve il problema.

Interfaccia del controller: comandi primitivi

Il controller espone un'interfaccia verso il sistema operativo formata da una serie di comandi primitivi: azionare o spegnere il motore, verificare la posizione della testina, leggere o scrivere in una determinata posizione, ecc. Sono comandi semplici ma estremamente ostici da usare direttamente rientrano nella categoria delle interfacce brutte e scomode. Per fare qualcosa di concreto,

come leggere un singolo blocco, è necessario orchestrare una sequenza complessa di questi comandi primitivi.

Linearizzazione degli indirizzi

Per astrarre i dettagli fisici del disco, il controller espone i dati come una sequenza lineare di blocchi numerati progressivamente. Questo indirizzo lineare e monodimensionale viene poi tradotto internamente nella tripletta di coordinate fisiche (cilindro, testina, settore). Il controller infatti prende tutti i blocchi del disco e li numera progressivamente, da 0 a N. Il controller poi si occupa internamente di tradurre quel numero nella tripletta fisica corrispondente. Da notare che sui cilindri più esterni ci sono più dati rispetto a quelli più interni, per via della maggiore superficie disponibile.

Correzione degli errori

Ad ogni blocco scritto, il controller aggiunge una manciata di byte di codice di controllo (error correction code). In fase di lettura, il controller verifica questo codice:

- Se tutto è andato bene, legge l'informazione e il codice di controllo senza problemi
- Se c'è stato un problema, il codice non corrisponde: la testina si riallinea e ritenta la lettura
- Dopo circa 5-10 tentativi falliti, quella zona del disco viene considerata degradata

Vale la pena notare che gli error correction code hanno uno svantaggio: occupano spazio aggiuntivo su disco, riducendo leggermente la capienza disponibile.

Alcuni sistemi gestiscono i blocchi danneggiati tramite rimappatura: il blocco problematico (es. blocco 1000) viene reindirizzato a un'area di riserva, appositamente riservata per questo scopo. Non si apre fisicamente il disco - è tutto gestito a livello logico dal controller. Non esistono zone del disco intrinsecamente più importanti di altre, ma ovviamente ci sono dati più importanti di altri.

Interfacce e driver

Il problema dell'interfaccia proprietaria

Ogni controller parla un proprio dialetto proprietario: una sequenza specifica di operazioni sulle porte di I/O, documentata nel manuale del produttore. Ad

esempio: “se vuoi leggere un blocco, scrivi X sulla porta 1, Y sulla porta 2, aspetta mezzo secondo, poi leggi dalla porta $3^{\prime \prime}$. Passaggi tediosi e specifici per ogni produttore.

Questo pone un problema: il sistema operativo dovrebbe conoscere tutti i dialetti di tutti i controller esistenti? No - il problema si risolve con il driver.

Driver

Il driver è un componente software, tipicamente scritto dal produttore stesso dell’hardware, che svolge il ruolo di traduttore tra due interfacce:

- Interfaccia verso il basso (proprietaria): parla il dialetto specifico del controller, eseguendo la sequenza corretta di letture e scritture sulle porte di I/O
- Interfaccia verso l’alto (standard): espone al sistema operativo comandi ad alto livello e standardizzati, come “leggi il blocco X dal dispositivo Y”

L’interfaccia esposta verso il sistema operativo è pubblica e standardizzata: chiunque può produrre hardware compatibile con essa, il che abbassa i costi di produzione. L’interfaccia proprietaria verso il basso, invece, non è standard ognuno si inventa il proprio dialetto. Nota: le interfacce verso l’alto sono poche, e sono di riferimento per qualsiasi produttore. Ovviamente però un driver può essere compatibile solo con uno standard.

Vale la pena notare che il driver non deve necessariamente essere scritto dal produttore dell’hardware: l’importante è che sia compatibile con il dialetto del controller.

Perché il driver deve girare in kernel mode

Le porte di I/O si leggono e scrivono tramite istruzioni privilegiate, eseguibili solo in kernel mode. Un processo utente non può accedere direttamente alle porte del controller - se potesse, potrebbe fare danni gravissimi al disco e ai dati. Per questo il driver deve girare in kernel mode, diventando a tutti gli effetti parte del kernel. Questo spiega perché un driver mal scritto può causare crash di sistema: un bug in kernel mode può compromettere l’intero sistema.

Mappatura delle porte I/O in memoria

Esiste un meccanismo alternativo che permette, in certi casi, di eseguire un driver in user mode, limitando i danni potenziali in caso di codice difettoso.

Il meccanismo funziona così:

1. Il SO, tramite un'operazione privilegiata fatta una sola volta (una tantum), istruisce la MMU a mappare le porte di un determinato controller su specifici indirizzi dello spazio di memoria
2. Da quel momento, leggere e scrivere su quegli indirizzi di memoria equivale a comunicare direttamente con quel controller - senza bisogno di istruzioni privilegiate
3. Il driver può quindi girare in user mode, accedendo al proprio controller attraverso quegli indirizzi mappati

Il vantaggio fondamentale è l'isolamento: poiché nella mappatura sono incluse solo le porte di quel controller specifico, il driver può al massimo danneggiare il proprio controller. Non può accedere alla memoria di altri processi, né interagire con altri controller. È un escamotage utile per contenere i danni di driver scritti male.

Alcune precisazioni importanti:

- L'atto della mappatura è privilegiato - è il SO a decidere quali porte mappare e a quale driver
- Le successive letture e scritture sugli indirizzi mappati non sono privilegiate - non c'è motivo di limitarle
- L'entità che legge e scrive sulle porte del controller deve essere sempre e solo una: se più entità accedessero contemporaneamente alle stesse porte si creerebbero problemi di concorrenza

Comunicazione asincrona: il problema dell'attesa

Quando un processo (es. un editor) richiede dei blocchi dal disco, si blocca in attesa. Il SO invia la richiesta al controller, ma non può restare in attesa attiva - sprecherebbe la CPU nel frattempo. Esistono due approcci per gestire questa attesa:

Busy waiting (approccio inefficiente)

Il SO interroga continuamente e ripetutamente la porta del controller per sapere se l'operazione è completata. È un approccio semplice ma estremamente inefficiente: la CPU è occupata a fare polling (cioè interrogare ripetutamente) invece di eseguire altri processi utili.

Interrupt (approccio corretto)

Quando l'operazione è completata, il controller invia un interrupt alla CPU. Il SO viene "svegliato", controlla l'esito sulla porta (es. $5=$ successo, $3=$ errore)

e procede. Nel frattempo la CPU era completamente libera di fare altro. Esistono tuttavia una serie di meccanismi che possono ritardare l'arrivo dell'interrupt.

DMA (Direct Memory Access)

Anche usando gli interrupt rimane un problema: i dati letti dal disco si trovano nei buffer interni del controller e devono essere trasferiti in RAM. Senza DMA, questo trasferimento viene eseguito dalla CPU principale, parola per parola - un processo seriale che tiene occupata la CPU inutilmente, aumentando l'overhead del sistema operativo (ovvero il tempo che la CPU spende per far funzionare il sistema stesso anziché eseguire lavoro utile).

Il controller DMA risolve il problema con un approccio elegante:

1. Dispone di un canale diretto tra i buffer del controller e la RAM, che bypassa completamente la CPU principale
2. Prima ancora di notificare la CPU, il controller DMA si occupa autonomamente di trasferire i dati dal buffer interno del controller alla RAM
3. Solo al termine del trasferimento invia un interrupt alla CPU, che ne prende atto sapendo che i dati sono già pronti in RAM

In questo modo la CPU viene coinvolta solo alla fine, quando il grosso del lavoro è già stato fatto. L'overhead del SO si riduce significativamente, e la CPU è libera di eseguire altri processi durante l'intero trasferimento.

Lezione 3 (12/03/26) Categorie di Sistemi Operativi

Mainframe

Calcolatori con capacità di gestione delle risorse di gran lunga superiori a quelle a cui siamo abituati: storage ingente, elevata capacità di calcolo. Sono la base su cui si appoggiano grandi infrastrutture di elaborazione dati.

Sistemi Batch

Il problema di partenza era il seguente: i calcolatori negli anni ' 50 -' 60 erano enormemente costosi, e quindi non esisteva un calcolatore per ogni persona, ma ce n'era solo uno per tanti utenti, quindi doveva essere condiviso tra più operatori.

Come gestire l'accesso a questa risorsa condivisa nel modo più efficiente possibile? La soluzione fu quella di raccogliere e accodare i job le unità di lavoro da eseguire - e inoltrarli al sistema in blocco (batch, appunto). Il sistema,

non appena si liberava una risorsa, prendeva in carico il job successivo dalla coda, notificando la disponibilità all'utente.

La caratteristica fondamentale che distingue i sistemi batch dai sistemi multiutente interattivi è che i job non sono interattivi (caratteristica chiave): l'utente li sottomette, poi aspetta il risultato senza alcun dialogo con la macchina durante l'esecuzione. Nel batch, quindi, a differenza dei moderni PC, si ha un tipo di comunicazione a senso unico e differita.

Il vantaggio principale è la massimizzazione dell'utilizzo del calcolatore, permettendo a più utenti di sfruttarlo nel tempo. Questo modello non è affatto obsoleto: dietro i grandi servizi cloud esistono meccanismi molto simili. Viene ancora oggi usato per sistemi che elaborano milioni di transazioni bancarie o per il training di una rete neurale. Anche localmente, questo modello di gestione dei job è stato adottato al DMI.

Server

Un server è una macchina con capacità superiori a quelle di un comune PC, progettata per erogare servizi specifici fruibili da più utenti contemporaneamente. A differenza del sistema batch, il server opera in modalità interattiva: deve essere reattivo nei confronti degli utenti in tempo reale.

È un sistema intrinsecamente multi-utente, e a differenza del PC - che multiutente lo è diventato quasi per accidente - il server è progettato fin dall'inizio per supportare una multi-utenza pesante e continuativa. Deve funzionare in modo stabile e indipendente dalle operazioni eseguite dai singoli processi o utenti.

Personal Computer

Il PC gestisce più processi contemporaneamente attraverso un'interfaccia grafica con cui l'utente interagisce direttamente: è quindi un sistema interattivo e multiprogrammato. Tecnicamente è anche multi-utente, ma non è stato progettato con questo obiettivo primario - a differenza dei server, che nascono esplicitamente per servire molti utenti in parallelo in modo robusto.

Definizione di multiprogrammato: sistema che tiene più programmi in memoria contemporaneamente, pronti per essere eseguiti. Nota che mentre la multiprogrammazione si riferisce ai processi (quindi ci sono più processi in memoria, e la CPU passa dall'uno all'altro quando uno si blocca su I/O), mentre il multithreading si riferisce ai thread (che sono le sotto-unità di un processo).

Quindi un processo è un programma in esecuzione, mentre un thread vive dentro un processo (è un suo flusso di esecuzione).

Sistemi Operativi per Smartphone

I SO per mobile sono sistemi interattivi, multiprogrammati e quasi sempre mono-utente. Le modalità operative sono simili a quelle dei sistemi desktop. Un aspetto rilevante è che la grande maggioranza dei SO per mobile utilizza un kernel Linux come base.

Rispetto ai sistemi desktop, i SO mobile pongono un'enfasi particolare sulla gestione energetica: il risparmio della batteria è un vincolo progettuale centrale, che influenza profondamente le politiche di scheduling e di gestione dei processi.

Sistemi Embedded

I sistemi embedded sono dispositivi elettronici con una struttura interna analoga a quella di un calcolatore: microonde, stampanti, forni evoluti, elettrodomestici "intelligenti". Al loro interno girano più processi gestiti da un SO - in certi casi addirittura una versione di Linux.

La caratteristica distintiva è che si tratta di sistemi chiusi non è possibile installare nuove applicazioni. Il numero di processi è fisso e limitato (tipicamente una dozzina). Questa non è una limitazione accidentale, ma una scelta progettuale precisa: si assume che l'utente non abbia necessità di modificare il comportamento del dispositivo oltre le funzioni previste.

Questa chiusura porta un vantaggio sostanziale per il SO: il comportamento dei processi è altamente prevedibile. Sapendo esattamente quali processi girano e come si comportano, il SO può operare in modo più semplice ed efficiente. Può persino permettersi di rinunciare ai meccanismi di isolamento dei processi (normalmente usati per contenere processi potenzialmente pericolosi o instabili), poiché il rischio è minimo in un ambiente controllato.

Sistemi Real-Time

I sistemi real-time hanno un ambito applicativo specifico e ben delimitato: macchinari industriali, robot, bracci meccanici, sistemi di controllo. Sono sistemi multiprogrammati e vicini all'interattività, ma con una peculiarità fondamentale: non basta che il sistema risponda - deve rispondere entro tempi garantiti.

Il vincolo temporale è imprescindibile. Se il sistema non riesce a rispettare i tempi di reazione promessi, le conseguenze possono essere fisiche e gravi: un

pezzo che cade dal nastro trasportatore, un macchinario che si danneggia, potenzialmente un'esplosione. Per questo motivo sistemi operativi generalpurpose come Linux o Windows non sono adatti a questi contesti: non offrono garanzie sufficienti sui tempi di risposta.

Si distingue tra:

- Hard real-time: il mancato rispetto delle scadenze temporali ha conseguenze fisiche dirette e inaccettabili. È il caso dei sistemi industriali descritti sopra.
- Soft real-time: il mancato rispetto delle scadenze degrada le prestazioni o la qualità del servizio, ma non causa conseguenze catastrofiche. Esempio: servizi di streaming. Lo scheduler del SO deve dare priorità al player video.

I sistemi real-time condividono con i sistemi embedded una caratteristica importante: sono anch'essi sistemi chiusi, con un insieme prefissato di processi noti a priori. Questo semplifica il lavoro del SO e contribuisce a rendere il comportamento del sistema più prevedibile e controllabile.

Nota trasversale: tematica ecologica

Una questione sempre più rilevante in tutti questi sistemi è il risparmio energetico. Nei grandi data center il consumo energetico è enorme - basti pensare al raffreddamento delle sale server - e anche a livello di singolo processo esistono sprechi significativi. Il SO moderno deve tenere conto di questi aspetti nella gestione delle risorse.

Struttura Software del Sistema Operativo

Creare un sistema operativo è uno dei progetti software più complessi e corposi che esistano - per questo ne esistono pochissimi. Solitamente, invece di costruirne uno da zero, si preferisce prendere un SO esistente e adattarlo alle proprie esigenze.

Esistono diverse possibili strutture per un SO. Nei casi reali, difficilmente un SO rispecchia esattamente una sola di queste categorie: la maggior parte adotta un approccio ibrido, che mescola caratteristiche di più modelli.

Sistemi Monolitici

In un sistema monolitico è difficile separare una componente dalle altre: tutto è interconnesso e interdipendente. Il termine aveva inizialmente un'accezione negativa. Esempi storici: MS-DOS, UNIX.

Nei primi sistemi monolitici si riscontrava una profonda disorganizzazione, spesso descritta come approccio “spaghetti-code”: il codice era aggrovigliato, senza struttura chiara. Questo portava a scarsa manutenibilità e a difficoltà di sviluppo nel tempo. Era quasi impossibile far girare due applicazioni contemporaneamente, e anche farne girare una sola poteva creare problemi se l’applicazione era scritta male.

Questo sistema è scritto come una raccolta di procedure, collegate all’interno di un solo grande programma. Un crash in una di queste procedure può mandare in crash tutto il sistema operativo. L’occultamento delle informazioni è praticamente inesistente: tutte le procedure sono visibili a tutte le altre.

Dentro questo tipo di sistema vi sono tre tipi di procedure:

1. Procedure di servizio richiesta: è la procedura che viene invocata quando un processo utente esegue una syscall, ovvero quando un’applicazione chiede al kernel di fare qualcosa (aprire file, allocare memoria etc). Il controllo passa a quesa procedura, che interpreta la richiesta e la smista;
2. Procedure di servizio: sono le procedure che implementano effettivamente le singole syscall. Ogni syscall ha una sua procedura di servizio corrispondente.
3. Procedure di utilità: sono funzioni di supporto usare dalle procedure di servizio. Svolgono operazioni comuni di cui più procedure di servizio potrebbe aver bisogno (es. copiare dati dalla memoria utente alla memoria kernel).

Nota bene: il Tanenbaum cita questa struttura nel contesto delle architetture monolitiche, ma in realtà è la struttura concettuale tipica di tutti i SO, solo che nei monolitici è più evidente dato che sono sistemi più scarni/grezzi e con meno architettura.

Una prima traccia di isolamento compare nei sistemi UNIX, che introducono la distinzione tra modalità utente e modalità kernel, permettendo di separare i dati tra le varie applicazioni e di tenere distinti i loro ambienti di esecuzione.

Sistemi a Struttura a Livelli (o a Strati)

Questo modello riprende un’idea già nota nelle reti di calcolatori: la stratificazione dei protocolli. Proprio come TCP dà per scontato che IP funzioni correttamente senza preoccuparsi di come, qui ogni strato si appoggia a quello sottostante senza curarsi dei dettagli implementativi interni.

La struttura è la seguente: lo strato più basso corrisponde all'hardware fisico. Al di sopra si trovano una serie di strati, ognuno dei quali implementa un'astrazione. Ogni strato:

- offre un servizio allo strato immediatamente superiore tramite un'interfaccia,
- utilizza i servizi forniti dallo strato immediatamente inferiore per implementare la propria funzionalità.

È in sostanza una forma di incapsulamento: ogni strato risolve un problema ben circoscritto, nascondendo la propria implementazione agli altri.

Esempio: uno strato potrebbe occuparsi della gestione delle CPU virtuali ovvero prendere l'unica CPU fisica disponibile e dare a ogni processo l'illusione di avere una CPU tutta per sé, fornita on-demand. L'algoritmo di scheduling quello che decide a quale processo assegna temporaneamente la CPU nel prossimo istante - sarebbe di competenza di questo strato.

Esempio di dipendenza tra strati: si consideri la gestione della memoria virtuale, che permette ai processi di occupare più spazio di quello fisicamente disponibile in RAM, spostando parte dei dati su disco. È evidente che la gestione del disco debba trovarsi in uno strato sotto la gestione della memoria virtuale, poiché questa ne dipende. Tuttavia non è sempre così semplice stabilire l'ordine: una chiamata su disco è tipicamente bloccante, il che farebbe pensare che la gestione delle CPU virtuali debba stare sotto quella del disco così da poter utilizzare un'altra CPU virtuale mentre si aspetta. Questo mostra che l'ordine degli strati non è banale da decidere. Infatti, saremmo stati tentati di mettere le CPU virtuali in uno strato superiore a quello della gestione del disco, dato che intuitivamente sembrano riguardare qualcosa di più lontano dall'hardware rispetto al disco.

Vantaggi:

- Il compito di ogni strato è semplice e circoscritto, più facile da implementare rispetto a dover scrivere un intero SO.
- Favorisce la qualità del software, la possibilità di fare testing e uno sviluppo per gradi.
- Lo sviluppo può procedere in parallelo su più strati, da team diversi, assumendo che gli strati sottostanti funzionino correttamente.

Svantaggi:

- Decidere l’ordine corretto degli strati può essere non banale, come mostrato nell’esempio sopra.
- Uno strato può consultare solo l’interfaccia dello strato immediatamente sottostante - non può saltare livelli. Questo genera richieste in cascata, aggiungendo overhead, problemi di prestazioni e complessità al sistema. Rispetto ad altri approcci, certe operazioni risultano più costose proprio per questo motivo.

Sistema ad Anelli Concentrici

Strutturalmente molto simile al modello a strati: si parte da un nucleo centrale corrispondente all’hardware, e si aggiungono anelli concentrici verso l’esterno, fino ad arrivare all’ultimo anello che rappresenta l’interfaccia utente. È stato usato nel sistema MULTICS.

Dal punto di vista della progettazione software le due idee sono praticamente identiche. La differenza rilevante è che nella struttura ad anelli, a runtime, esiste un meccanismo di isolamento tra uno strato e l’altro. Questo meccanismo funziona in modo simile a una TRAP (o sys-call): ogni volta che uno strato ha bisogno di un servizio offerto da uno strato sottostante, effettua una chiamata speciale che attraversa il confine tra gli anelli. Man mano che si scende verso l’interno, si acquisiscono più privilegi - più ci si avvicina all’hardware, più poteri si hanno.

Vantaggio rispetto ai semplici strati: il sistema è più robusto. Ogni anello non può interferire direttamente con gli altri (grazie alla trap), rendendo il sistema più resistente ai bug e ai malfunzionamenti.

Svantaggio: questa robustezza ha un costo. La chiamata passa attraverso un meccanismo di TRAP, che è più lenta di una semplice chiamata a funzione. L’overhead è quindi maggiore rispetto al modello a strati puro.

In pratica, sia la struttura a strati che quella ad anelli concentrici nella loro forma radicale non vengono mai usate nei SO reali - troppo rigide e penalizzanti in termini di prestazioni.

Sistemi a Microkernel

L’idea di fondo è opposta rispetto al monolitico: si estrae dal kernel tutto ciò che è possibile estrarre, riducendo il kernel a un insieme minimo di funzionalità indispensabili. Tutto il resto viene implementato come processi utente moduli che girano al di fuori del microkernel, in spazio isolato, in modalità utente.

Cosa rimane nel microkernel? Solo ciò che non può stare fuori - ad esempio lo scheduling, che manipola la CPU a stretto contatto con l'hardware ed è alla base stessa dell'esistenza dei processi.

Come comunicano i componenti? Attraverso un sistema di messaggi: i vari processi (e servizi) si scambiano messaggi tra loro (quindi comunicazione tra pari), oppure li inviano al kernel. La primitiva di comunicazione è il messaggio, e l'infrastruttura che la gestisce è fornita dal microkernel.

Vantaggi:

- Tutto ciò che gira nel microkernel è piccolo, essenziale e più facile da scrivere e verificare rispetto a un kernel completo.
- I servizi esterni, girando in spazio utente, sono incapsulati: la loro implementazione interna è nascosta agli altri. Sono forzati a esporre interfacce standard basate sui messaggi.
- Se un servizio ha un bug, il problema resta isolato in quello spazio. Ad esempio, un bug nello stack di rete non compromette il file system, e viceversa.
- È possibile auto-riparare un servizio difettoso: se viene diagnosticato un problema, il servizio può essere terminato e riavviato da capo, senza dover riavviare l'intero sistema. Ciò è possibile grazie al Reincarnation server, un server il cui compito è proprio quello di monitorare continuamente i processi driver e i servizi che girano in spazio utente. Se uno di questi processi si comporta in modo anomalo (crash, produce errori, etc), allora il reincarnation server termina il processo difettoso e lo ricrea da zero, riportandolo allo stato iniziale.

Nota: il reincarnation server lo chiamiamo server in quanto è un componente che gira in spazio utente e offre servizi agli altri processi (sua definizione in questo contesto).

Problema pratico - i driver: come fa, ad esempio, il driver della stampante a interagire con il controller della stampante se gira in modalità utente? La soluzione è la mappatura delle porte di I/O in memoria (tramite la MMU, Memory Management Unit): il microkernel, quando crea l'ambiente di lavoro per un driver, sa già con quale hardware dovrà interagire. Può quindi mappare specifiche porte nella memoria del processo driver, permettendogli di comunicare con quella porzione di hardware senza avere accesso a tutto il resto.

Esempio celebre: MINIX, scritto da Tanenbaum. Una copia di MINIX è presente all'interno delle CPU Intel, dove gestisce una piccola parte del software interno del processore.

Mac OS implementa al suo interno un microkernel (in forma ibrida). Svantaggio: se si esagera portando fuori dal microkernel ogni possibile servizio, si accumula un overhead significativo - ogni comunicazione tra servizi passa attraverso messaggi, e i messaggi hanno un costo.

Struttura a Moduli

L'idea è simile a quella del microkernel: si ha un nucleo di funzionalità inscindibili, e poi una serie di funzionalità progettate come moduli - molto simili agli oggetti nella programmazione OO. Questi moduli possono invocare i servizi offerti dagli altri moduli. Esempio: lo stesso algoritmo di scheduling può essere concepito come un modulo sostituibile.

Differenza fondamentale rispetto al microkernel: quando un modulo viene caricato in memoria, diventa parte integrante del kernel ed è eseguito direttamente in modalità kernel. Qualunque modulo può chiamare direttamente la procedura di un altro modulo con una semplice call kernelkernel.

Vantaggio: overhead praticamente zero nella comunicazione tra moduli una semplice chiamata a funzione, non un messaggio. Qui le comunicazioni sono kernel-kernel, mentre nel sistema a microkernel il flusso era spazio utente → kernel → spazio utente.

Svantaggio: se un modulo ha un bug, questo può mandare in crash l'intero sistema. Se un driver è difettoso, compromette tutto. In pratica però, nei sistemi ben realizzati come il kernel Linux, questi eventi sono estremamente rari.

Il sistema carica i moduli su richiesta, quando ne riconosce la necessità non tutti i moduli esistenti vengono caricati all'avvio, solo quelli effettivamente utili.

Linux adotta questa struttura a moduli. Mac OS adotta un approccio ibrido: ha estensioni del kernel simili ai moduli, ha una struttura parzialmente a strati, e incorpora un microkernel - violando però il purismo del microkernel con queste estensioni.

Macchine Virtuali

Una macchina virtuale (VM) è un'astrazione completa che simula un hardware reale, permettendo di far girare su di essa un sistema operativo come se girasse su una macchina fisica. Sono nate insieme ai primi calcolatori, poi sono tornate di grande attualità nell'ultimo decennio. Quindi è un programma

software che simula l'esistenza di un computer (con CPU virtuale, RAM virtuale, disco virtuale, scheda di rete virtuale etc). La VM è la macchina simulata "vuota", dove possiamo installare un SO scelto da noi. Il programma che si occupa della gestione della VM a livello software è noto come hypervisor.

Compiti dell'hypervisor:

1. Creazione e gestione delle risorse virtuali come RAM, CPU, disco, scheda di rete etc;
2. Isolare le VM tra loro;
3. Schedulare l'accesso alla CPU fisica;
4. Tradurre le operazioni hardware: quando il SO dentro la VM vuole accedere al disco, sta accedendo a un disco virtuale. L'hypervisor intercetta questa operazione e la traduce in un'operazione reale sul disco fisico. Lo stesso vale per la rete, la CPU, etc.

Utilizzi principali:

- Eseguire più sistemi operativi contemporaneamente sulla stessa macchina fisica.
- Testare un proprio SO in sviluppo senza rischiare la macchina reale.
- Sicurezza e isolamento dei servizi: invece di far girare N servizi come N processi sullo stesso SO (con tutti i rischi che ne derivano), si può assegnare a ogni servizio la propria macchina virtuale isolata, con la propria rete virtuale. Così, se un attaccante compromette un servizio, è intrappolato dentro quella VM.

Come funziona tecnicamente la virtualizzazione?

Il codice destinato alla VM (ovvero il codice del SO e delle applicazioni installate dentro la VM) viene eseguito direttamente sulla CPU fisica - non viene interpretato dall'hypervisor. Questo vale per le normali istruzioni applicative: girano in modo nativo, senza rallentamenti. Il problema sorge quando la VM deve accedere a periferiche, dispositivi di I/O, rete: lì il software simulato ha bisogno di un intermediario, che introduce overhead. Nel caso in cui il codice non riguardi periferiche di alcun tipo, allora l'hypervisor non interviene e il codice viene fatto girare direttamente sulla CPU fisica. Invece, nel caso in cui la VM faccia una richiesta di accesso a una periferica, l'hypervisor la intercetta e la gestisce sulla periferica reale per conto della VM, restituendo il risultato.

Il vincolo fondamentale è che la macchina virtuale deve essere completamente indistinguibile da una macchina reale per il software che ci gira sopra - deve funzionare in modo trasparente, senza che il SO virtualizzato sappia di essere dentro una VM. Questo vincolo crea alcune complessità.

Paravirtualizzazione: una soluzione che “bara” sul paradigma classico. Il SO virtualizzato sa di trovarsi in un ambiente virtuale, e questa consapevolezza permette di usare driver ad hoc che lavorano a stretto contatto con il sistema di virtualizzazione, bypassando alcuni meccanismi e riducendo l’overhead. Esempio pratico: installare Windows su VirtualBox lo fa girare, ma installare i Guest Additions (driver dedicati per scheda di rete virtuale, scheda video, ecc.) lo fa girare significativamente più veloce.

1. Hypervisor di tipo 1: il software che gestisce le VM gira direttamente sull’hardware, senza un SO sottostante. È la tipologia storica originale. Un esempio moderno è KVM (Kernel-based Virtual Machine) di Linux: una funzionalità integrata nel kernel che permette di far girare VM su Linux, rendendo Linux stesso un hypervisor (o meglio: il kernel di Linux acquisisce capacità da hypervisor, ma rimane anche un SO normale a tutti gli effetti). Si parla di KVM come un ibrido tra tipo $\mathbf{1}$ e tipo 2, in quanto non è tipo 1 puro perché sotto c’è comunque un SO completo, e non è tipo 2 puro perché le funzionalità di virtualizzazione sono integrate nel kernel, non sopra a un SO come applicazione separata (es. software separato come VirtualBox che si installa sopra il SO come un’applicazione).
2. Hypervisor di tipo 2: il caso più diffuso nella pratica quotidiana. Si ha un hardware con un SO ordinario, e sopra di esso si installa un software (come VirtualBox) che permette di creare e gestire macchine virtuali multiple. Si parla di software ospitante (il SO reale) e software ospitato (il SO nella VM). L’hypervisor di tipo 2 non può svolgere tutte le operazioni come semplice processo utente: alcune sue componenti devono lavorare a livello kernel. Per questo VirtualBox richiede l’installazione di componenti kernel - una sorta di driver che si innesta nel SO ospitante. Appare come un processo utente, ma ha in realtà una parte che opera a livello kernel.

Tradeoff: il tipo 1 è più efficiente perché non ha il SO ospitante in mezzo, il tipo 2 è più comodo perché si installa come una normale applicazione su una macchina già esistente.

Utilizzi hypervisor di tipo 1: non viene utilizzato per far girare una sola VM, in quel caso non avrebbe senso e converrebbe usare un ordinario SO. Lo si usa invece per far girare decine o centinaia di VM contemporaneamente sulla

stessa macchina fisica. Il vantaggio è che le VM sono tutte isolate fra di loro. Ognuna ha il suo SO, il suo spazio di memoria, etc. Se una crasha non tocca le altre. Il contesto di utilizzo tipico sono i datacenter.

Supporto hardware alla virtualizzazione: se la CPU dispone di componenti specificamente progettati per favorire la virtualizzazione (come le estensioni Intel VT-x o AMD-V), l'intera operazione risulta più semplice da implementare e genera meno overhead.

Sicurezza e Isolamento dei Servizi

Cosa significa attaccare un servizio? L'attaccante deve prima individuare un bug nel software del servizio - una vulnerabilità che gli permette di interagire col sistema in modo non previsto. L'obiettivo più semplice è il denial of service: sfruttare il bug per far crashare il servizio, rendendolo inutilizzabile.

L'obiettivo più pericoloso è invece prendere possesso del servizio. Un metodo comune è la code injection: l'attaccante invia richieste HTTP appositamente costruite che contengono codice malevolo. Se il web server non le gestisce correttamente, si ritrova ad eseguire quel codice esterno - e a quel punto l'attaccante ha il controllo del processo.

Quanto danno può fare l'attaccante una volta preso il controllo?

Dipende dai privilegi con cui gira quel processo. Normalmente i processi vengono eseguiti come utenti ordinari, con privilegi limitati - e questo contiene il danno. Ma se l'amministratore ha configurato il servizio in modo che giri con privilegi di root (permessi massimi a livello di SO, scelta pessima), allora l'attaccante che prende controllo di quel processo ha pieno controllo sull'intera macchina: può leggere qualsiasi file, accedere alla posta elettronica, modificare dati, ecc.

Anche partendo da un processo con privilegi ordinari, un attaccante può tentare la cosiddetta privilege escalation: sfruttare bug nel kernel per passare dalla modalità utente alla modalità root, acquisendo così pieni poteri sul sistema.

La soluzione: isolamento con le VM

Un modo efficace per limitare i danni è assegnare a ogni servizio la propria macchina virtuale, con la propria rete virtuale. Se un attaccante compromette un servizio, è intrappolato dentro quella VM - non può raggiungere gli altri servizi né la macchina fisica sottostante. Il danno resta confinato.

Lezione 4 (17/03/26)

Processi

Definizione di processo

Un processo sequenziale (o semplicemente processo) è un'istanza in esecuzione di un programma: una sequenza di passi che la CPU esegue nel tempo. I sistemi moderni sono multi-programmati, ovvero gestiscono una moltitudine di processi contemporaneamente. Partendo dallo stesso codice sorgente è possibile istanziare due processi ben distinti: essi condividono solo il codice (solo a livello RAM, grazie a un'ottimizzazione del sistema operativo, dato che è read-only; virtualmente anche il codice viene considerato separato), ma sono entità separate e indipendenti.

Processo vs Programma: mentre un programma è qualcosa che può essere semplicemente memorizzato sul disco senza fare nulla, un processo è un'attività di un certo tipo, che rappresenta l'esecuzione effettiva del programma stesso.

Spazio di indirizzamento

A ogni processo è associato uno spazio di indirizzamento: un'area di memoria dedicata esclusivamente a quel processo. Valgono due proprietà fondamentali:

- il processo non può uscire dal proprio spazio;
- gli altri processi non possono entrare in quello spazio (isolamento e privacy: un processo non deve poter leggere i dati di un altro).

Lo spazio di indirizzamento è suddiviso in più zone:

Text

Contiene il codice eseguibile del programma. Quando si istanzia un processo a partire da un eseguibile, il SO copia il programma in memoria e lo racchiude in questa "scatola".

Data

Contiene variabili e dati statici: entità che esistono per tutta la durata del processo. Vengono inizializzate al momento della copia del file in memoria sono già precompilate, e il loro posizionamento immediatamente dopo la zona text è legato al meccanismo che ne permette l'istanziazione. Ovvero: sono precompilate nel senso che i valori delle variabili statiche inizializzate (es. int x $=5$) sono già scritti dentro il file eseguibile prima ancora che il processo parta, e quindi non vengono calcolati a run-time. La parte sul posizionamento, invece, riguarda il formato ELF degli eseguibili (es. gli eseguibili Linux), che

organizzano il file in sezioni ordinate: prima il codice (text), poi i dati statici (data). Contiene quindi anche variabili globali e statiche, che ovviamente sono leggibili e modificabili durante tutta la durata del processo.

Stack

Struttura di supporto all'esecuzione, allocata all'estremità alta dello spazio di indirizzamento. Ha dimensione variabile e cresce verso il basso. Gestisce le chiamate a funzione, le variabili locali e i valori di ritorno. Lo spazio vitale di una procedura esiste sullo stack solo per la durata della procedura stessa: quando la procedura ritorna, il suo frame viene rimosso.

Heap

Zona dedicata all'allocazione dinamica della memoria, che cresce verso l'alto (direzione opposta allo stack). È necessaria per un motivo preciso: se tutta la memoria fosse gestita dallo stack, non sarebbe possibile allocare oggetti che sopravvivano alla funzione che li ha creati. Con l'heap, invece, l'oggetto allocato dinamicamente persiste anche dopo che il frame della funzione è stato rimosso dallo stack.

Stack e heap si contendono la memoria libera che si trova tra di loro: se le due strutture si toccano, la memoria è esaurita.

Stato del processo e registri della CPU

Un processo non è solo il suo codice e i suoi dati: lo stato di esecuzione della CPU fa parte integrante dello stato del processo. Riguarda, in particolare, tutto ciò che descrive la situazione della CPU in un dato istante, come Program Counter, contenuto interno dei registri, Stack Pointer etc. Questo è fondamentale per capire come il SO possa sospendere un processo e riprenderne l'esecuzione in seguito (context switch). Infatti, se a un certo punto del processo questo viene interrotto perché la CPU viene passata a un altro processo, allora il SO deve salvare lo stato interno del processo all'interno del PCB del processo stesso, per permettere il context switch verso l'altro processo e fare sì che più avanti la CPU possa tornare al processo di partenza e riprendere esattamente da dove aveva lasciato.

PCB - Process Control Block

Il SO mantiene per ogni processo esistente una struttura dati chiamata PCB (Process Control Block), che ne rappresenta i metadati. L'insieme di tutti i PCB attivi forma la tabella dei processi, dove ogni entry corrisponde a un processo

esistente in quel dato istante. L'indice nella tabella può essere considerato l'identificativo del processo (PID, Process ID).

Il PCB contiene tipicamente:

- stato del processo (running, ready, waiting...);
- valori dei registri della CPU al momento della sospensione;
- informazioni sulla memoria (puntatori allo spazio di indirizzamento);
- informazioni sulle risorse aperte (file, socket [ovvero un canale di comunicazione tra due processi, etc);
- dati per lo scheduling (priorità, tempo di CPU utilizzato...).

In un dato istante, la CPU è usata da un solo processo. Gli altri n-1 processi non stanno usando la CPU, ma il loro stato deve essere preservato. Per ogni processo sospeso, il PCB contiene una copia speculare dei registri della CPU al momento della sospensione: tanti campi quanti sono i registri dell'architettura, ognuno con il valore che il registro aveva quando il processo è stato interrotto. Quando il processo tornerà in esecuzione, questi valori verranno ripristinati nella CPU.

Ogni processo, nel corso della propria vita, può aprire un certo numero di file. Quando un file viene aperto, è necessario tenere traccia della posizione corrente di lettura/scrittura all'interno del file - questo meccanismo è implementato tramite un file pointer. Non si tratta di qualcosa implementato in hardware: il file pointer è un'informazione software, mantenuta in una struttura dati apposita. Il PCB contiene un riferimento (puntatore) a questa struttura - tecnicamente non è il PCB stesso a contenere direttamente i dati, ma un puntatore ad essi. Quindi nel PCB c'è un puntatore a una struttura dati nota come file descriptor table, un array di puntatori dove ogni puntatore punta a una struttura dati chiamata open file description, che contiene le informazioni effettive sul file aperto:

1. *File Pointer:**quest'ultimo ci dice a che punto del file eravamo arrivati, infatti punta a una posizione all'interno del file. In particolare, il file pointer è un numero intero che indica quanti byte sono stati letti dall'inizio del file fino a quel momento.
2. **Modalità di apertura**
3. Altre informazioni sul file

In Linux, i PCB sono organizzati internamente tramite un Red-Black Tree (RBT), struttura utilizzata dagli algoritmi di scheduling per accedere e gestire i processi in modo efficiente.

Gestione dei Processi

Pseudoparallelismo e Time Slicing

Nei sistemi multi-programmati, il parallelismo tra processi è in realtà uno pseudoparallelismo: in un dato istante, la CPU esegue sempre e solo un processo alla volta. Tuttavia, dal punto di vista dell'utente, la resa è quasi indistinguibile da un vero parallelismo.

La tecnica alla base di questo meccanismo è il time slicing (o multiplexing del tempo): il tempo di CPU viene suddiviso in fettine (time slice), e ogni processo riceve il proprio quanto di tempo a rotazione.

Questo crea l'illusione della CPU virtuale: ogni processo vive nell'astrazione di avere una CPU tutta per sé. Il programmatore non scrive il codice pensando di doversi contendere la CPU con altri processi - suppone semplicemente di avere una CPU dedicata e lavora in quel mondo semplificato.

Creazione dei processi

Ogni processo può essere creato solo da un altro processo. Ma chi crea il primo processo? È il sistema operativo stesso, durante la fase di avvio: subito dopo aver caricato il kernel e organizzato le proprie strutture dati, il SO crea il primo processo. Nei sistemi UNIX questo processo è chiamato init, e il suo compito è creare tutti i processi necessari al funzionamento del sistema. Quello che percepiamo come interfaccia grafica, ad esempio, non è il SO stesso: è un processo separato con cui interagiamo, creato a partire da init.

Albero dei processi (UNIX)

Nei sistemi UNIX, la struttura di creazione dei processi forma un albero: ogni processo ha un padre, e può avere uno o più figli. Quando un processo muore, i suoi figli diventano orfani e vengono adottati dal processo init, che mantiene così sempre integra la struttura ad albero. In alcuni casi, come la caduta di una connessione di rete, il processo che gestiva quella connessione muore e i suoi figli vengono adottati secondo questa regola di base.

Approcci alla creazione di un nuovo processo

Qualunque SO deve mettere a disposizione almeno una syscall per creare un nuovo processo. Esistono due approcci principali: quello Windows e quello UNIX.

Approccio Windows: CreateProcess

In Windows esiste una singola chiamata - CreateProcess - che accetta una decina di argomenti e permette al chiamante di specificare dettagliatamente le caratteristiche del processo figlio:

- Il processo figlio riceve un PID (Process ID, un numero intero) univoco, assegnato dal sistema (non dal padre). Quando un processo muore, il suo PID viene liberato e può essere riciclato.
- Il padre specifica quale programma eseguibile dovrà eseguire il figlio (percorso, nome del file, ecc.).
- È possibile passare una riga di comando con argomenti (es. il nome del file da aprire, nel caso di un editor).

Esempio concreto: quando si clicca sull'icona della calcolatrice, è la GUI (processo padre) che invoca CreateProcess, indicando come eseguibile la calcolatrice. Il SO crea la nuova entità, aggiorna la tabella dei processi, e il figlio inizia la propria esecuzione.

Approccio UNIX: fork + exec

In UNIX il meccanismo di base è diverso e si articola in due syscall distinte.

fork

La fork non prende argomenti. Quando un processo padre la invoca, sta chiedendo al SO una cosa precisa: "crea una copia di me stesso". Il risultato è un nuovo processo figlio con un proprio PID, il cui spazio di indirizzamento è una copia quasi perfetta di quello del padre (stessa memoria, stesse variabili, stesso stato di esecuzione). L'unica differenza necessaria è il PID, che deve essere univoco.

Sembrerebbe uno spreco clonare l'intero spazio di indirizzamento, ma il SO è abbastanza furbo da non farlo davvero: non perde tempo a copiare informazioni che già possiede, operando in modo efficiente e parsimonioso in termini di RAM. Il meccanismo preciso verrà approfondito studiando la gestione della memoria.

exec

La exec permette a un processo esistente di eseguire un nuovo programma all'interno del proprio spazio di indirizzamento. Quando un processo invoca exec indicando un eseguibile, il SO non crea un nuovo processo: svuota lo spazio di indirizzamento del chiamante (stack, heap, text, data) e lo reinizializza

con il codice del nuovo programma. Il processo mantiene lo stesso PID, ma da quel momento in poi esegue un codice completamente diverso.

fork $\boldsymbol{+}$ exec $\boldsymbol{=}$ CreateProcess

La combinazione delle due syscall permette di ottenere lo stesso risultato di CreateProcess:

1. Il padre invoca fork → nasce un figlio clone.
2. Il figlio invoca exec → si dimentica di essere un clone del padre e inizia a eseguire il nuovo programma (es. la calcolatrice).

Il padre aveva già stabilito cosa il figlio avrebbe dovuto fare: appena il figlio inizia, lancia exec e si trasforma nel nuovo programma. La CreateProcess può essere simulata tramite fork + exec, ma non viceversa - il modello UNIX è più generale.

Il motivo per cui UNIX usa questo meccanismo è in parte storico (è nato prima di Windows), ma anche funzionale: il modello della clonazione e dello sdoppiamento si presta molto bene alla programmazione parallela. Su un sistema multicore con 8 CPU , ad esempio, un programma può clonare sè stesso tramite fork e lasciare che i cloni comunichino e si coordinino tra loro, sfruttando più CPU contemporaneamente, e quindi "dividersi" il lavoro all'occorrenza, più semplice con la fork perché padre e figlio all'inizio condividono il contesto: strutture dati, variabili etc. Questo è un modello di programmazione parallela basato su processi distinti, diverso dalla programmazione multi-threading. La fork segue il principio copy on write, e quindi all'inizio il figlio e il padre condividono la stessa area di memoria, i dati del padre non vengono veramente copiati in un nuovo spazio di memoria, sarebbe uno spreco. La copia viene fatta solamente quando bisogna modificare qualcosa, ma finché si è in read-only non c'è motivo di copiare.

Il meccanismo di creazione dei processi tiene traccia delle relazioni di parentela: ogni processo sa chi lo ha creato (il padre) e il SO mantiene questa informazione internamente. Questo permette di tracciare la gerarchia dei processi - l'albero che abbiamo già visto - e ha conseguenze pratiche: in alcuni casi, la relazione di parentela conferisce diritti specifici o permette di implementare funzioni particolari, come la wait con cui il padre attende la terminazione del processo figlio.

Terminazione dei Processi e Stati del Processo

Motivi di terminazione

Un processo può terminare per quattro ragioni principali:

1. **Uscita normale (volontaria)**

Il processo ha completato il proprio lavoro e comunica al SO il desiderio di terminare tramite una chiamata di sistema (exit). Per convenzione, restituisce un exit status: un valore numerico che indica l'esito dell'esecuzione. Un valore pari a 0 indica tipicamente successo.

1. **Uscita con errore (volontaria)**

Sempre spontanea, ma in una situazione di errore. Il processo si accorge di un problema che non può risolvere - ad esempio, un file indicato sulla riga di comando non esiste - e decide di terminare restituendo un codice di errore. È comunque il processo stesso a chiedere di terminare: si tratta di un'uscita volontaria.

1. **Errore critico (involontaria)**

Il processo si trova in una situazione così problematica da non poter continuare in alcun modo. Esempi tipici:

- esecuzione di un'istruzione illegale;
- divisione per zero;
- tentativo di accesso a un'area di memoria non permessa.

In questi casi il processo viene terminato in modo brusco e involontario: non è lui a chiedere di terminare, ma il sistema che non ha altra scelta.

1. **Richiesta di terminazione esterna (involontaria)**

Un processo distinto chiede al SO di terminare il processo in questione. È involontaria e imprevedibile per il processo che viene terminato - come se ricevesse un interrupt dall'esterno, indipendentemente da cosa stia facendo.

Alcune note importanti su questo meccanismo:

- Nei sistemi multi-utente, un processo può essere terminato solo dal proprio proprietario: non è possibile terminare i processi di un altro utente. L'amministratore di sistema non ha questo vincolo.
- Nei sistemi UNIX, questa syscall si chiama kill.
- Un processo può rifiutarsi di assecondare la richiesta di terminazione (entro certi limiti).

- È esattamente quello che avviene quando si clicca il pulsante di chiusura di una finestra: si sta interagendo con la GUI, che a sua volta chiede al SO di terminare quel processo.

Nei sistemi UNIX esiste un meccanismo chiamato segnali, che ricorda gli interrupt hardware ma opera a livello software. Un segnale è una notifica asincrona inviata a un processo per comunicargli un evento - può provenire dal kernel (es. in caso di errore critico), da un altro processo (es. richiesta di terminazione tramite kill), o dall'utente (es. Ctrl+C nel terminale). I segnali sono asincroni rispetto all'esecuzione del processo: arrivano indipendentemente da cosa stia facendo il processo in quel momento, esattamente come un interrupt hardware arriva indipendentemente da cosa stia facendo la CPU. In particolare: qualcuno genera il segnale, il kernel lo annota nel PCB del processo destinatario, la CPU continua a fare quello che stava facendo. Al prossimo passaggio in modalità kernel (es. syscall, interrupt del clock, context switch etc), il kernel controlla se ci sono segnali pendenti per il processo. Se sì, prima di restituire il controllo al processo, gestisce il segnale.

Stati di un processo

Il SO modella il ciclo di vita di un processo come un automa a stati finiti. Esistono tre stati principali:

Ready (pronto)

Il processo è pronto per essere eseguito ed è in attesa di ottenere la CPU. Tutti i processi in questo stato sono candidati validi per l'esecuzione.

Il SO mantiene una coda dei processi pronti: una struttura dati che raccoglie i PCB di tutti i processi in stato ready. Il criterio base di questa coda è FIFO (First In, First Out): si preleva sempre dal lato opposto rispetto all'inserimento, dando priorità ai processi arrivati prima.

Nota: la coda FIFO è solo uno dei modelli possibili (il più semplice). In generale, si utilizza una struttura dati che contiene i PCB di tutti i processi in stato di ready e poi lo scheduler, in base all'algoritmo di scheduling utilizzato, sceglierà il prossimo processo da lanciare. Il criterio della FIFO è solo uno dei tanti.

Running (in esecuzione)

Il processo sta attualmente usando la CPU. Su un sistema con una sola CPU, in questo stato può trovarsi un solo processo alla volta.

Blocked (bloccato)

Il processo è in attesa di un evento esterno - tipicamente il completamento di un'operazione di I/O - e non può fare nulla finché quell'evento non si verifica. Non ha senso tenerlo nella coda dei pronti, perché non è effettivamente pronto: se lo scheduler lo scegliesse, troverebbe un processo che non può ancora andare avanti.

Transizioni tra stati

Da Running a Blocked

Quando un processo in esecuzione effettua una chiamata di sistema bloccante - tipicamente una richiesta di I/O come la lettura di un file dal disco - entra in stato blocked. Le chiamate di I/O sono definite "lente" o "bloccanti" perché, commisurate ai ritmi della CPU, richiedono tempi enormi: anche decine o centinaia di millisecondi. Tenere la CPU ferma in attesa di quell'esito sarebbe uno spreco enorme: ci sono altri processi pronti che potrebbero usarla. Il processo viene quindi spostato nello stato blocked, e la CPU viene assegnata a qualcun altro.

Da Blocked a Ready

Quando l'evento atteso si verifica - ad esempio, il controller del disco invia un interrupt al SO per segnalare che la lettura è completata - il SO individua quale processo era in attesa di quell'esito e lo sposta dallo stato blocked allo stato ready, non direttamente in running. Il processo deve aspettare il proprio turno nella coda: sarà lo scheduler a sceglierlo quando sarà il momento.

Il ruolo dello Scheduler e del Dispatcher

Quando la CPU è libera, entrano in gioco due meccanismi distinti:

- Lo scheduler applica un algoritmo di scelta e seleziona il prossimo processo da eseguire tra quelli in stato ready.
- Il dispatcher si occupa della parte operativa: preleva i valori dei registri dal PCB del processo scelto e li carica nella CPU, preparandola per l'esecuzione di quel processo. Questa operazione fa parte del context switch.

Obiettivo del meccanismo

L'intero sistema di stati e transizioni ha un unico scopo: massimizzare l'utilizzo della CPU. Ogni volta che un processo è in attesa di qualcosa, la CPU non deve restare ferma - deve essere impiegata per qualcosa di più proficuo. Lo stato blocked esiste esattamente per questo: isola i processi in attesa, lasciando la

coda dei processi pronti composta solo da processi che possono effettivamente fare lavoro utile.

Transizioni di Stato e Prelazione

La transizione Running → Ready: il problema della monopolizzazione

Finora abbiamo visto le transizioni running → blocked → ready, legate alle operazioni di I/O. Ma esiste un'altra transizione importante: running → ready, che avviene tramite prelazione.

Il problema nasce da una domanda semplice: cosa succede se un processo non effettua mai operazioni di I/O? In teoria, quel processo terrebbe la CPU per sé indefinitamente, monopolizzandola.

- In un sistema batch questo è accettabile: i processi girano uno alla volta, senza interazione con l'utente.
- In un sistema interattivo non lo è: se un processo monopolizza la CPU, la GUI si blocca e il sistema smette di rispondere.

Per evitare questo, nei sistemi interattivi il SO deve poter togliere la CPU a un processo anche in assenza di operazioni di I/O. Questa operazione si chiama prelazione.

Il meccanismo della prelazione

Il SO stabilisce un tempo massimo di utilizzo della CPU per ogni processo. Se il processo non cede spontaneamente la CPU entro quel tempo, il SO gliela sottrae forzatamente e rimette il processo nella coda dei processi pronti.

Ma c'è un problema tecnico: come fa il SO a riprendere il controllo della CPU, se in quel momento la CPU è sotto il controllo del processo in esecuzione?

La risposta è: tramite gli interrupt.

Il ruolo degli interrupt

Ogni volta che si verifica un interrupt, la CPU interrompe il processo corrente e trasferisce il controllo al SO. Il SO prende transitoriamente possesso della CPU, gestisce l'interrupt, e poi decide cosa fare.

Ma cosa succede se non si verifica nessun interrupt - ad esempio perché tutti i dispositivi sono fermi? Il SO sfrutta uno specifico interrupt legato al clock di sistema: il compito di questo componente hardware è quello di generare interrupt periodici con una frequenza prestabilita. Questo garantisce che il SO possa riprendere il controllo della CPU periodicamente, indipendentemente da quello che sta facendo il processo in esecuzione.

La maggior parte delle volte il SO restituisce immediatamente la CPU al processo interrotto. Ma se si accorge che quel processo la sta usando da troppo tempo, lo rimette nella coda dei processi pronti e assegna la CPU a un altro processo. Questo è esattamente il meccanismo della prelazione. Caso limite: se il processo prelazionato è l'unico nella coda dei pronti, lo scheduler lo sceglierà di nuovo immediatamente e la CPU gli verrà restituita.

Nota: ci sono situazioni in cui il SO ritarda deliberatamente gli interrupt, quando sta eseguendo operazioni delicate che non devono essere interrotte (es. la modifica della tabella dei processi). Ovvero, istruisce la CPU a mascherare temporaneamente gli interrupt e di ignorarli per un breve periodo.

Salvataggio dello stato durante un interrupt

Quando scatta un interrupt, è necessario salvare lo stato del processo interrotto per poterlo ripristinare in seguito. Non tutto viene salvato automaticamente:

- Program Counter (PC) e PSW (Program Status Word) vengono salvati automaticamente dall'hardware al momento dell'interrupt.
- Tutti gli altri registri devono essere salvati manualmente dal SO nel PCB del processo - come se si trattasse di una scena del crimine da preservare intatta.

Dopo aver gestito l'interrupt, il SO deve decidere se riprendere il processo interrotto o schedularne un altro (come nel caso della prelazione). Se riprende un processo diverso, deve:

1. Caricare i valori dei registri dal PCB di quel processo.
2. Configurare la mappatura della memoria, informando la MMU (Memory Management Unit) su come gestire lo spazio di indirizzamento del nuovo processo.

Le code dei processi

La coda dei processi pronti non è l'unica struttura di questo tipo. Il SO mantiene diverse code:

- Coda dei processi pronti (ready queue): contiene tutti i processi in stato ready, in attesa della CPU.
- Code dei processi bloccati (I/O queue): contengono i processi in stato blocked, in attesa di un evento. Ne esiste una per ogni tipo di evento: una per le richieste al disco, una per l'input da tastiera, ecc. Quando un evento

si verifica, il SO va a guardare la coda specifica associata a quell'evento e sblocca i processi in attesa di esso.

Il processo padre in attesa del figlio

È comune che un processo padre si blocchi in attesa della terminazione del proprio figlio. L'esempio più chiaro è la shell:

1. L'utente digita un comando e preme invio.
2. La shell crea un processo figlio per eseguire quel comando.
3. La shell (processo padre) entra in stato blocked, in attesa che il figlio termini.
4. Quando il processo figlio termina, la shell si risveglia e torna in stato ready.

Questo comportamento è sincrono: la shell aspetta il figlio prima di tornare attiva. Nei sistemi UNIX è possibile modificare questo comportamento aggiungendo & alla fine del comando: il processo figlio viene lanciato in modo asincrono, e la shell non aspetta la sua terminazione ma torna subito disponibile per nuovi comandi. Questo verrà approfondito durante il laboratorio.§ Riguarda la wait di cui abbiamo parlato poco prima.

Comunicazione e Sincronizzazione tra Processi

Il problema della collaborazione tra processi

Finora abbiamo trattato i processi come entità isolate. Ma nella realtà esistono scenari in cui più processi devono collaborare e comunicare. Questo introduce una serie di problemi che analizzeremo in questa sezione.

InterProcess Communication (IPC)

Il modello della pipe

Un primo esempio di comunicazione tra processi è il meccanismo della pipe, familiare dalla riga di comando: cmd1 | cmd2 | cmd3

Ogni processo ha tre canali standard:

- Standard input - da cui riceve dati.
- Standard output - su cui produce dati.
- Standard error - canale separato per i messaggi di errore, per non mischiarli con l'output ordinario.

Con la pipe, tutto ciò che il comando 1 produce in output viene dato in input al comando 2 , e così via. Il trasferimento avviene attraverso un buffer di I/O, che

gestisce in modo naturale le esigenze di sincronizzazione.

Sintesi: la pipe è un meccanismo che il SO mette a disposizione per far comunicare due processi. Meccanismo che sta dietro al carattere | nella riga di comando. Esempio: ls | grep .txt → ls produce la lista dei file, e la scrive sul suo standard output. grep, invece, la riceve sul suo standard input e filtra solo i file .txt. I due processi non si conoscono: si limitano a scrivere e leggere dal proprio canale standard, e il sistema operativo fa da tramite grazie a un buffer. La pipe è la soluzione più semplice per far comunicare due processi. Tuttavia, ha dei limiti evidenti:

1. Unidirezionale: i dati vanno solo in una direzione, da cmd1 a cmd2, e cmd2 non può rispondere a cmd1.
2. Rigida: un processo scrive, l'altro legge.
3. Non permette di condividere strutture dati: i due processi non lavorano sugli stessi dati, si passano solo messaggi.

Per questi motive, la pipe è considerata come un esempio primordiale di comunicazione tra processi, in quanto è il meccanismo IPC più basilare esistente.

Il problema della sincronizzazione

Senza buffer, la comunicazione diretta tra processi creerebbe problemi in entrambe le direzioni:

- Il consumatore è troppo veloce: il comando 2 vuole ricevere dati, ma il comando 1 non ne ha ancora prodotti. Il comando 2 deve aspettare.
- Il produttore è troppo veloce: il comando 1 vuole produrre output, ma il comando 2 non è pronto a riceverlo perché sta ancora processando i dati precedenti. Il comando 1 deve aspettare.

In entrambi i casi l'attesa deve essere passiva - il processo entra in stato blocked senza consumare CPU. Il contrario sarebbe il busy waiting: un loop continuo in cui il processo controlla ripetutamente se i dati sono arrivati, sprecando cicli di CPU inutilmente. Funzioni come scanf o cin implementano correttamente un'attesa passiva.

Il buffer risolve entrambi i problemi: permette al produttore di depositare dati anche se il consumatore non è pronto, e al consumatore di attendere passivamente se non ci sono dati disponibili.

Questo schema modella il classico problema produttori-consumatori: un problema standard che ricorre frequentemente nella programmazione concorrente e che vedremo spesso anche nelle esercitazioni di laboratorio.

Memoria condivisa

Un secondo meccanismo di IPC è la memoria condivisa. Il SO può creare un segmento di memoria condivisa - una zona di memoria che viola intenzionalmente l'isolamento tra processi, su richiesta esplicita dei processi stessi.

Il funzionamento è il seguente:

- Il processo P1 chiede al SO di creare un segmento condiviso (es. 10 KB) e di mapparlo nel proprio spazio di indirizzamento.
- Il processo P2 fa la stessa richiesta per lo stesso segmento.
- I due processi ora hanno una finestra comune: una zona di memoria accessibile da entrambi, attraverso cui possono comunicare.

In questa zona condivisa si possono costruire strutture dati comuni - ad esempio una coda in cui un processo inserisce dati e l'altro li estrae, creando un canale di comunicazione bidirezionale.

Una caratteristica importante: il segmento condiviso è un'entità esterna ai processi. Se nessuno lo distrugge esplicitamente, rimane in vita anche dopo che i processi terminano.

Nota: per i thread il discorso è più semplice - condividono già lo stesso spazio di indirizzamento, quindi qualsiasi struttura dati allocata nell'heap è automaticamente accessibile a tutti i thread del processo.

Notiamo che, a differenza del modello della pipe, nel modello della memoria condivisa i processi comunicano in modo bidirezionale grazie alla struttura dati condivisa. Tuttavia, questo introduce il problema delle race condition: se due processi accedono contemporaneamente alla stessa struttura dati, il risultato dipende dall'ordine di esecuzione, e questo chiaramente è un errore.

Thread

Definizione di thread

Finora abbiamo visto il processo come un ambiente di esecuzione con un singolo flusso di esecuzione: il codice viene eseguito istruzione per istruzione, partendo dal main e terminando con la exit. Questo è il modello classico a cui siamo abituati.

L’idea dei thread è quella di avere più flussi di esecuzione all’interno dello stesso processo - come se il processo avesse più CPU virtuali a disposizione contemporaneamente. Ogni thread è un flusso indipendente che esegue una parte del codice del programma.

Un thread è un flusso di esecuzione all’interno di un processo. Un processo può avere più thread che condividono lo stesso spazio di indirizzamento (stessa memoria, stessi file aperti, stesse risorse), ma ciascuno ha il proprio program counter, il proprio stack e i propri registri.

Il vantaggio rispetto ad avere più processi separati è che i thread condividono la memoria, quindi comunicare tra loro è immediato (basta leggere/scrivere sulla stessa variabile). Tra processi separati invece la memoria è isolata, e per comunicare servono meccanismi come lo scambio di messaggi con il percorso utente → kernel → utente.

Perché usare i thread: vantaggi

Disaccoppiamento delle operazioni

Avere più flussi di esecuzione permette di disaccoppiare operazioni diverse, eseguendole in parallelo invece che in sequenza. Il vantaggio principale è legato al meccanismo di bloccaggio visto in precedenza: se un thread si blocca in attesa di un’operazione di I/O, gli altri thread possono continuare a lavorare.

Esempi concreti

- Word processor: un thread gestisce il refresh dell’interfaccia grafica, un altro si occupa del correttore ortografico. L’utente continua a scrivere senza che l’interfaccia si blocchi.
- Browser web: ogni scheda o richiesta può essere gestita da un thread separato.
- Web server: si mette in ascolto su un socket di rete. Quando arriva una richiesta, un thread dedicato la gestisce - leggendo file locali, eseguendo operazioni di I/O bloccanti - mentre gli altri thread restano liberi di gestire nuove richieste in arrivo.

Sfruttamento del multicore

Su un sistema con più core, thread diversi dello stesso processo possono girare su CPU diverse contemporaneamente, sfruttando appieno l’hardware disponibile con una singola applicazione.

Vantaggio anche su singolo processore

Il beneficio del non-bloccaggio esiste anche su sistemi con un singolo processore: mentre un thread è bloccato in attesa di I/O, un altro thread può usare la CPU per fare lavoro utile. Abbiamo già incontrato questo concetto quando abbiamo parlato di multi-threading.

Struttura interna: cosa condividono e cosa no

I thread vivono all'interno dello stesso processo e condividono lo stesso spazio di indirizzamento. Nel dettaglio:

Condiviso tra tutti i thread:

- Spazio di indirizzamento del processo.
- Segmento text (codice del programma) - i thread possono eseguire funzioni diverse dello stesso programma.
- Segmento data (variabili globali e statiche).
- Heap (memoria allocata dinamicamente).

Privato per ogni thread:

- Stack - ogni thread ha il proprio, perché ogni thread ha la propria storia di chiamate a funzione.
- Program Counter - ogni thread tiene traccia autonomamente di quale istruzione sta eseguendo.
- Registri della CPU - al momento della sospensione, ogni thread ha il proprio stato di esecuzione.

Rischi: race condition

Il fatto che tutti i thread condividano lo stesso spazio di indirizzamento significa che un thread può mettere mano ovunque - inclusi i dati degli altri thread. Questo è potente ma pericoloso.

Quando più thread lavorano sugli stessi dati senza coordinazione, si possono verificare race condition: situazioni in cui il risultato dipende dall'ordine temporale in cui i thread accedono ai dati. Il sintomo tipico è un codice che funziona correttamente la maggior parte delle volte, ma produce risultati errati in condizioni di stress o con certi pattern di scheduling.

I thread tra loro sono fondamentalmente collaborativi - non ostili come potrebbero essere processi distinti - ma questa collaborazione deve essere gestita con attenzione tramite meccanismi di sincronizzazione appropriati.

Context switch tra thread

Il context switch è il lavoro svolto dal SO, innescato da un interrupt, in cui viene salvato lo stato di esecuzione del processo corrente e la CPU viene assegnata a un altro. Questa operazione si chiama anche dispatching. Più è veloce, minore è l'overhead e più efficiente è il SO.

I thread offrono un vantaggio importante in questo senso. Consideriamo due processi P1 (con thread A, B, C) e P2 (con thread D, E):

- Context switch tra thread fratelli (es. da A a B, all'interno di P1): è molto più veloce del context switch classico. I thread condividono già lo stesso spazio di indirizzamento - non è necessario cambiare la mappatura della memoria. Si salvano solo i registri e il program counter del thread corrente.
- Context switch tra processi diversi (es. da B a D, da P1 a P2): è il context switch classico, più costoso. Bisogna salvare l'intero stato del processo, cambiare la mappatura della memoria e informare la MMU del nuovo spazio di indirizzamento. DUBBIO : capire meglio il ruolo della MMU.

Operazioni sui thread

thread_create

Crea un nuovo thread all'interno del processo. A differenza della fork, l'ambiente non viene duplicato ma condiviso: il nuovo thread nasce già nello stesso spazio di indirizzamento del processo. Il thread capostipite ovvero il thread che nasce insieme al processo - può creare altri thread aiutanti all'occorrenza.

thread_exit

Un thread la chiama quando ha terminato il proprio lavoro. Segnala che quel flusso di esecuzione non ha più nulla da fare e può essere terminato.

thread_join

È l'analogo della wait per i processi. Permette a un thread di sincronizzarsi con la terminazione di un altro thread: il thread che chiama thread_join entra in stato blocked e aspetta che il thread specificato termini, dopodiché si risveglia. Nell'esempio fatto sul mergesort (vedi la parte sui task nella programmazione multicore), il thread che si occupa di fare il merge chiama una thread join sui figli, entrando in stato di blocked e aspettando che loro finiscano il loro compito.

thread_yield

Permette a un thread di cedere spontaneamente il controllo della CPU a favore di un thread fratello. È particolarmente utile in un ambiente collaborativo - come il modello a thread utente visto in precedenza - dove la prelazione non è disponibile e la collaborazione tra thread diventa indispensabile. In un sistema con prelazione è meno critica, ma può comunque essere utile.

Lezione 19/03/26

Programmazione Multicore

Considerazioni generali

La programmazione multicore introduce un onere aggiuntivo per chi scrive i programmi: richiede un modo di ragionare diverso rispetto alla programmazione sequenziale tradizionale.

Analisi dei task

Il primo passo è individuare i task, ovvero i compiti che l'applicazione deve svolgere. Esempi tipici: il salvataggio e caricamento di documenti, il correttore ortografico, la gestione della GUI, ecc.

Il principio generale è: un task, un thread - ogni thread si occupa di un compito specifico. Tuttavia, questa corrispondenza 1:1 non è sempre la scelta migliore, per diverse ragioni:

- I thread non sono gratuiti: hanno un costo in termini di risorse di sistema.
- I task non sono sempre paragonabili tra loro: alcuni sono pesanti e continuativi (es. il correttore ortografico, attivo per tutta la durata dell'applicazione), altri sono leggeri o temporali (es. il salvataggio di un documento, che si attiva solo in certi momenti).

Bisogna quindi creare task con finestre temporali che ne giustifichino l'esistenza come thread separati. Se un task si svolge in pochissimo tempo, non vale la pena dedicargli un thread apposito: è preferibile inserirlo all'interno di un task più grande, associando così più task a un unico thread. Questa coesistenza non crea problemi.

Analisi dei dati

Parallelamente all'analisi dei task, è necessaria un'analisi dei dati: ogni task avrà bisogno di strutture dati per svolgere il proprio compito. Si presentano due situazioni:

- Dati privati di un singolo task (caso più semplice): la struttura dati è usata solo da quel thread e non da altri. Esempio: il dizionario del correttore

ortografico, usato esclusivamente dal thread del correttore.

- Dati condivisi tra più task: più thread lavorano sulle stesse strutture dati. In questo caso servono meccanismi opportuni di sincronizzazione e coordinazione per gestire l'accesso concorrente senza incorrere in problemi.

Dipendenze tra task

Dall'analisi dei dati possono emergere delle dipendenze tra task. Alcuni esempi:

- Durante il caricamento di un documento, la componente GUI potrebbe dover essere disattivata parzialmente (es. la funzione di modifica) finché il caricamento non è completato.
- Un algoritmo di ordinamento come quicksort o mergesort può essere parallelizzato in modo naturale. Dividendo il problema in due sottoproblemi, si ottengono tre task:

I primi due task sono indipendenti - ognuno lavora sulla propria parte del vettore e può girare su thread (e CPU) diversi, rendendo l'ordinamento più veloce e scalabile. Il terzo task, il merge, è invece dipendente dagli altri due: può iniziare solo quando entrambi gli ordinamenti sono completati.

1. Ordinare la prima metà del vettore.
2. Ordinare la seconda metà del vettore.
3. Eseguire il merge dei due risultati.

Difficoltà: debugging e race condition

La programmazione concorrente introduce problemi che non esistono nella programmazione sequenziale.

Il debugging è significativamente più difficile, perché il comportamento del programma può dipendere da come i thread vengono schedulati - e la schedulazione non è deterministica.

Il problema più insidioso è la race condition (corsa critica): una situazione in cui il risultato dell'esecuzione dipende dall'ordine temporale in cui i thread accedono a dati condivisi. Il sintomo tipico è un codice che funziona correttamente in condizioni normali, ma produce risultati errati o si comporta in modo imprevedibile sotto stress. A parità di input, si possono ottenere output diversi - il che rende questi bug estremamente difficili da riprodurre e correggere.

Thread a Livello Utente e Modelli di Threading

Contesto storico

All'inizio, i primi sistemi operativi gestivano un solo processo. Questo non significava però che fosse impossibile strutturare i programmi in termini di thread: era possibile farlo tramite i cosiddetti thread a livello utente, implementati attraverso un run-time environment.

Il Run-Time System

Il run-time system è una libreria esterna - scritta da qualcun altro - che il programmatore include nel proprio codice e le cui funzionalità (come thread_create) vengono richiamate come normali chiamate a funzione. Non è coinvolto il kernel: si tratta di codice utente che chiama altro codice utente, senza mai entrare in modalità kernel.

Il run-time system opera come un sub-affittuario della CPU: il SO gli assegna un quanto di tempo (ad esempio un secondo di CPU), e da lì il run-time gestisce autonomamente i propri thread. Per farlo, si occupa di:

- salvare e ripristinare i registri della CPU;
- mantenere una tabella dei thread, in cui tiene traccia delle informazioni di ciascun thread (analoga al PCB per i processi, ma gestita interamente in spazio utente).

Sintesi: il run time system è una libreria che gira in modalità utente. il SO gli assegna una quantità di tempo in cui usare la cpu, e lui si occupa di schedulare il tempo tra i vari thread. Il SO non sa nulla di quello che il run-time system fa con quel tempo, dal punto di vista del SO esiste solo il processo, a cui viene assegnata la CPU per una determinata quantità di tempo.

Il meccanismo di cambio di contesto: Thread Yield

Poiché il run-time system opera interamente in spazio utente, non ha accesso ai meccanismi di prelazione (interrupt periodici che forzano il passaggio da un thread all'altro). Non può quindi interrompere un thread arbitrariamente come farebbe il SO con i processi.

La soluzione è la collaborazione volontaria tra thread, realizzata tramite la chiamata thread_yield. L'idea è che ogni thread, quando ritiene di aver fatto abbastanza per quel turno o quando non ha più nulla da fare, cede spontaneamente la CPU virtuale ai suoi thread fratelli. thread_yield non è una chiamata di sistema vera e propria, ma una funzione della libreria. Il suo funzionamento è il seguente:

1. Salva tutti i registri della CPU nella voce della tabella dei thread relativa al thread corrente.
2. Sceglie quale thread eseguire successivamente, applicando un algoritmo di scheduling - con il vincolo che non possono essere usati algoritmi che prevedono la prelazione.

Questo vincolo non è particolarmente stringente in questo contesto: i thread sono scritti dallo stesso programmatore, e l'idea di fondo è quella della collaborazione. Con pochi thread la scelta del successore è poco critica, ma con molti thread la qualità dello scheduling può diventare importante.

Svantaggi del modello a thread utente

1. **Chiamate di sistema bloccanti**

Il problema più grave riguarda le chiamate di sistema bloccanti. Se un thread effettua una chiamata bloccante (ad esempio la syscall select, usata per verificare lo stato di un dispositivo), e il dispositivo non è pronto, la chiamata blocca l'intero processo - e con esso tutti i thread al suo interno. Il run-time system non ha modo di intervenire, perché il controllo è passato al kernel (dato che è stata fatta la syscall). Ciò comporta un evidente spreco della CPU.

1. **Page fault**

Un problema analogo si verifica con i page fault, che approfondiremo studiando la gestione della memoria. In breve: quando la RAM non è sufficiente per contenere tutti i processi, il SO sposta temporaneamente sul disco alcune porzioni di memoria. Quando un processo cerca di accedere a una di queste porzioni, si verifica un page fault - il SO deve recuperare i dati dal disco, caricarli in RAM, e poi far riprendere l'esecuzione come se nulla fosse accaduto.

Il problema, nel contesto dei thread utente, è il seguente: se il thread C causa un page fault, dal punto di vista del SO è l'intero processo ad averlo causato. Il SO blocca quindi l'intero processo, fermando anche i thread A e B che non c'entrano nulla. I page fault sono fisiologici e continueranno ad esistere finché esiste la memoria virtuale, ma nei thread utente non è possibile isolare il blocco al solo thread responsabile.

Nota: anche il page-fault è bloccante, ma in modo diverso. Mentre nel caso della select il blocco era prevedibile (in quanto si sa che la select è una syscall, e quindi bloccante), il page-fault è bloccate in modo più sottile, dato che si verifica solo quando la porzione di memoria a cui si tenta di accedere non si

trova in RAM ma sul disco. Il SO, anche qui, vede solo che il processo ha richiesto dei dati dal disco, e lo tiene bloccato, quando in realtà all'interno di quel processo ci sono molti thread pronti ad eseguire del lavoro grazie alla CPU.

Vantaggi del modello a thread utente

Il principale vantaggio è la velocità del context switch tra thread. Il passaggio da un thread all'altro avviene interamente in modalità utente, senza mai invocare il kernel. Questo lo rende uno dei meccanismi di context switch più rapidi disponibili, con un overhead minimo.

Modelli di threading

Modello 1-a-molti (Many-to-One)

È il modello descritto finora. Un singolo thread kernel viene associato a molti thread utente all'interno del processo. Il run-time system gestisce tutti i thread su un unico flusso di esecuzione kernel. È il modello storicamente più antico, ed era l'unico disponibile nelle prime fasi.

Modello 1-a-1 (One-to-One)

A ogni thread utente corrisponde un thread kernel dedicato. Ogni flusso di esecuzione interno al processo è gestito direttamente dal SO. Questo risolve i problemi delle chiamate bloccanti e dei page fault (il kernel può bloccare il singolo thread senza fermare gli altri), ma ha un costo maggiore in termini di risorse, poiché ogni thread kernel è più pesante di un thread utente. Qualunque sistema operativo moderno che utilizziamo implementa questo modello tramite chiamate di sistema - ovvero comunicando direttamente col SO.

Vantaggi del modello 1-a-1:

- Il modello 1-a-1 risolve il problema principale del modello 1-a-molti: una chiamata bloccante effettuata dal thread B mette in stato blocked solo il thread B, lasciando i thread A e C dello stesso processo liberi di continuare a girare.
- Un altro vantaggio pratico riguarda la gestione delle risorse: tramite un thread pool si stabilisce un numero massimo di thread (es. dieci), evitando di pagare continuamente il costo di thread_create e thread_exit per ogni nuova operazione. Thread pool: un insieme di thread creati in anticipo, all'avvio dell'applicazione, che restano in vita per tutta la durata del programma pronti a ricevere lavoro. L'idea è che creare e distruggere thread ha un costo (thread create e thread exit non sono gratis): se

un’applicazione gestisce molte operazioni brevi e frequenti (es. web server che gestisce migliaia di richieste), allora la cosa migliore da fase è di assegna una nuova richiesta a un thread libero del pool, e quando il thread termina, invece di essere distrutto, viene reso di nuovo disponibile nel pool per la prossima richiesta. Così si paga il costo di creazione una sola volta, all’avvio (evitando la latenza dovuta alla creazione), e si ha un limite massimo di thread attivi contemporaneamente.

Svantaggio del modello 1-a-1:

- Il costo fisiologico di questo modello è il context switch, che avviene quando il kernel decide di passare da un thread all’altro - sia per scheduling ordinario che per prelazione. Non è necessaria la thread_yield: i thread possono essere scritti in modo “egoistico”, ognuno concentrato sul proprio compito senza doversi preoccupare di cedere volontariamente la CPU. Sarà il kernel a gestire il tutto. Ovviamente, dove esistono strutture dati condivise, la sincronizzazione rimane necessaria - ma il codice non deve essere scritto in modo eccessivamente collaborativo come nel modello 1-a-molti.

Definizione di Thread Kernel: è un thread che il SO conosce e gestisce direttamente. È l’unico flusso di esecuzione di cui il kernel è consapevole, che schedula e a cui assegna tempo CPU. I thread utente, invece, sono i flussi di esecuzione che esistono solo nella visione del run-time system: il kernel non li conosce e non li schedula. Per il kernel, tutta l’attività del processo passa attraverso un unico thread kernel. Quindi, i thread kernel sono gestiti dal SO, e i thread utente dal run-time system.

Modello ibrido (Many-to-Many)

Il modello ibrido, detto molti-a-molti, cerca di prendere il meglio dal modello 1-a-molti e dal modello 1-a-1.

L’idea è la seguente: anziché associare un thread kernel a ogni thread utente (come nel modello 1-a-1), si fa un’analisi preventiva dei task e si decide caso per caso come raggrupparli:

- Task collaborativi, che lavorano a stretto contatto e si scambiano spesso il controllo, vengono raggruppati su un unico thread kernel, sfruttando il meccanismo leggero della thread_yield per il cambio di contesto tra di loro.
- Task che effettuano operazioni di I/O bloccanti, o che richiedono vera indipendenza, vengono assegnati ciascuno a un thread kernel dedicato,

così da non bloccarsi a vicenda.

Esempio: il thread del correttore ortografico e quello del salvataggio possono condividere un unico thread kernel - se uno si blocca in attesa del disco, l'altro probabilmente non è attivo in quel momento, quindi il blocco non crea problemi. Due task invece che lavorano in modo continuo e indipendente avranno ognuno il proprio thread kernel.

Il vantaggio principale è il risparmio di risorse: si usano meno thread kernel, che sono più costosi, e si sfrutta dove possibile il context switch leggero del modello utente. Se l'analisi dei task è fatta bene, questo modello non ha svantaggi significativi.

Compatibilità dei vari modelli

Per garantire la portabilità tra sistemi operativi diversi, esistono standard che uniformano l'interfaccia di gestione dei thread, indipendentemente dal fatto che siano implementati come thread kernel o utente. Su sistemi UNIX, lo standard di riferimento è POSIX (da cui il nome pthreads - POSIX threads): un insieme di interfacce standard che permettono di scrivere codice multithread portabile, senza dover ricorrere a librerie specifiche per ogni sistema operativo.

POSIX ci permette di lavorare più ad alto livello fornendoci un livello maggiore di astrazione: scriviamo il codice usando le interfacce standard (pthread_create, pthread_join etc), senza preoccuparci di come il SO le implementa internamente. Il programmatore così pensa solo a cosa vuole fare, e non a come il SO lo realizza internamente.

Sintesi dei vari Context Switch

1. Context switch tra thread fratelli - modello utente (1-a-molti): avviene interamente in spazio utente tramite thread_yield, senza mai invocare il kernel. È il più veloce, con overhead minimo.
2. Context switch tra thread fratelli - modello 1-a-1: i thread sono fratelli (stesso spazio di indirizzamento), ma il passaggio avviene attraverso il kernel (modalità utente → kernel → utente). Non è necessario riprogrammare la MMU , ma c'è comunque il costo del cambio di modalità.
3. Context switch tra processi distinti: oltre al cambio di modalità kernel, è necessario riprogrammare la MMU per aggiornare la mappatura della memoria al nuovo processo. È il più costoso.

Il problema delle race condition

L’esempio del conto corrente

Supponiamo che due processi P1 e P2 lavorino sulla stessa variabile condivisa x - ad esempio il saldo di un conto corrente. Entrambi eseguono l’operazione $\mathrm{x}+$ + cento volte. Ci aspettiamo che alla fine x sia incrementata di 200. Ma questo non è garantito.

Il motivo è che x++ non è un’operazione atomica. A livello di istruzioni macchina si scompone in tre operazioni:ss $\mathrm{R} \leftarrow \mathrm{X} / /$ leggi il valore di x nel registro $\mathrm{R}++/ /$ incrementa il registro $\mathrm{X} \leftarrow \mathrm{R} / /$ scrivi il nuovo valore in x

Dove R è un registro della CPU - diverso per P1 e P2, ma entrambi condividono la stessa x in memoria.

Uno scenario sfortunato

Supponiamo che $\mathrm{x}=10$ al tempo t :

1. Pl esegue $\mathrm{R} \leftarrow \mathrm{X} \rightarrow$ il suo registro vale 10 .
2. P1 esegue $\mathrm{R}++\rightarrow$ il suo registro vale 11 .
3. Il SO prelaziona P1 prima che possa eseguire $X \leftarrow R$.
4. P2 esegue $\mathrm{R} \leftarrow \mathrm{X} \rightarrow$ legge 10 (P1 non ha ancora scritto).
5. P2 esegue $\mathrm{R}++\rightarrow$ il suo registro vale 11 .
6. P2 esegue $X \leftarrow R \rightarrow$ scrive 11 in X.
7. Il controllo torna a P1, che esegue $X \leftarrow R \rightarrow$ scrive 11 in X.

Risultato: $x=11$ invece di 12 . Un incremento è andato perso. Questo problema non si risolve aggiungendo controlli aggiuntivi: un’ulteriore lettura di controllo è essa stessa un’operazione che può essere interlacciata in modo sfortunato. Più è complessa l’operazione, più sono le combinazioni di interlacciamento problematiche - ad esempio, su un RedBlack Tree, due rotazioni concorrenti porterebbero a un comportamento completamente imprevedibile.

Nota importante: questo problema non riguarda solo i processi, ma anche il kernel stesso. Su sistemi multicore, più flussi di esecuzione possono invocare syscall contemporaneamente, accedendo alle stesse strutture dati del kernel.

Anche il kernel deve quindi garantire la mutua esclusione sulle proprie strutture dati interne.

Sezioni critiche e mutua esclusione

Definizione

Una sezione critica è una porzione di codice in cui il processo accede a strutture dati condivise - in lettura o in scrittura. Anche la sola lettura può essere pericolosa: ad esempio, leggere un nodo da un albero mentre un altro processo lo sta modificando può produrre risultati inconsistenti.

L'obiettivo è garantire che le sezioni critiche siano atomiche: o vengono eseguite completamente, o non vengono eseguite affatto. Non deve essere possibile osservare uno stato intermedio.

Per gestire l'accesso alle sezioni critiche, si inseriscono delle guardie:

- Enter region: codice eseguito prima di entrare nella sezione critica verifica se è possibile entrare, altrimenti attende.
- Leave region: codice eseguito all'uscita dalla sezione critica - segnala che la sezione critica è libera.

Le quattro proprietà di una buona soluzione

1. Mutua esclusione (fondamentale) Se il processo P1 è nella propria sezione critica, nessun altro processo può essere nella propria sezione critica contemporaneamente. I due processi possono coesistere, ma non possono essere entrambi in sezione critica nello stesso istante.
2. Indipendenza dall'hardware (desiderabile) La soluzione non deve fare assunzioni sulla velocità di esecuzione della CPU o sul numero di core disponibili. Deve funzionare correttamente su qualsiasi architettura. Esempio: il programmatore sa che P2 è più veloce di P1, e quindi scrive la soluzione contando sul fatto che quando P1 arriva alla enter region, P2 ha già finito la sua sezione critica - quindi non c'è conflitto. Ma non si possono fare assunzioni di questo tipo: magari ci sono macchine in cui P2 risulta essere più lento. Altro esempio: scrivo una soluzione che funziona solo perché assumo che la CPU sia abbastanza veloce da uscire dalla sezione critica prima che un certo evento accada. Su una CPU più lenta, questa assunzione si rompe e la soluzione fallisce. Altro esempio: oppure magari assumo di avere una sola CPU, e quindi penso che due istruzioni non potranno mai essere eseguite contemporaneamente (falso nel caso di sistemi multi-core).

3. Nessun blocco inutile (desiderabile) Un processo che si trova fuori dalla propria sezione critica non deve poter bloccare un altro processo che vuole entrare nella propria. Non c'è alcun motivo per cui un processo che sta eseguendo codice non critico debba impedire ad altri di accedere alle loro sezioni critiche.
4. Nessuna attesa infinita (desiderabile) Nessun processo deve aspettare indefinitamente di entrare nella propria sezione critica. Prima o poi deve riuscire ad accedere.

Nota aggiuntiva: una buona soluzione deve implementare l'attesa in modo passivo - il processo che non può entrare nella sezione critica deve entrare in stato blocked, non eseguire busy waiting.

Prima soluzione: disabilitare gli interrupt

Idea di base

Su un sistema con una sola CPU, il problema dell'interlacciamento è causato dalla prelazione, che a sua volta è causata dagli interrupt del clock. Se si disabilitano gli interrupt, la prelazione non può avvenire, e le sezioni critiche diventano di fatto atomiche.

Problemi

- Rischio di blocco del sistema: se gli interrupt rimangono disabilitati troppo a lungo, eventi importanti vengono ritardati o persi. Un programmatore potrebbe dimenticarsi di riabilitarli, oppure uscire dalla sezione critica per un errore senza riabilitarli - con conseguenze disastrose.
- Pericoloso in mano ai processi utente: lasciare che un processo utente disabiliti gli interrupt è una pessima idea. È un'operazione privilegiata che solo il kernel dovrebbe poter fare, e solo per periodi brevissimi (es. operazioni delicate).
- Non funziona su sistemi multicore: su un sistema con più core, il problema non è la prelazione - è che due core diversi possono eseguire simultaneamente le istruzioni di fetch e store delle rispettive sezioni critiche. Disabilitare gli interrupt su un core non impedisce all'altro core di accedere alla stessa struttura dati contemporaneamente.

In sintesi: disabilitare gli interrupt è una soluzione accettabile solo per il kernel, su sistemi a singolo core, per periodi molto brevi. Non è una soluzione generale. Notiamo infatti che viola la proprietà 2.

Seconda soluzione: variabili di lock, TSL e XCHG

Idea di base

L'idea è molto semplice: si usa una variabile condivisa, chiamata lock, che può valere 0 o 1 .

- $\mathbf{O}$ per indicare che la sezione critica è libera, e quindi si può entrare
- 1 per indicare che la sezione critica è occupata, e quindi bisogna aspettare prima di entrarci

Prima di entrare nella sezione critica, il processo controlla il lock. Se vale 0, lo imposta a 1 ed entra. Quando esce, lo riporta a 0.

Problemi

Questa soluzione, tuttavia, non funziona (per lo stesso motivo per cui non funziona $\mathrm{x}++$). Controllare il lock e impostarlo a 1 non sono un'operazione atomica. Sono due istruzioni separate. Tra il controllo e l'impostazione, un altro processo potrebbe inserirsi, leggere anch'esso 0 , ed entrare nella sezione critica. Il risultato è che così ben due processi si trovano nella sezione critica contemporaneamente.

Per funzionare correttamente, le variabili di lock devono essere gestite a livello kernel, in quanto quest'ultimo riesce a rendere atomica questa operazione. Istruzione hardware atomica che faccia le due cose (leggere il valore ed impostarlo di conseguenza) esiste. Questo concetto ****prende il nome di TSL (Test and Set Lock), e l'equivalente concreto su architettura x86 è XCHG (Exchange). Poiché queste istruzioni sono privilegiate, solo il kernel le può eseguire. Il processo utente non può usare TSL direttamente, quindi chiede al kernel tramite una syscall di eseguire l'operazione atomica per conto suo. Il kernel usa TSL, che è atomica a livello hardware, e nessun altro processo può inserirsi nel mezzo. Che si fa se la syscall per la TSL viene eseguita contemporaneamente da due processi? Questa situazione non si può verificare, proprio perché la TSL viene eseguita a livello hardware: quando un core sta eseguendo TSL su una variabile, il bus di memoria viene bloccato per tutti gli altri core per la durata di quell'istruzione. Il secondo processo deve quindi aspettare che il primo finisca.

Ma in un sistema multi-core è possibile eseguire due syscall contemporaneamente? Sì, il kernel può essere eseguito contemporaneamente su due core diversi (il kernel è unico solo come codice e struttura dati, ma poi

se ne possono istanziare diversi, a seconda delle syscall che vengono fatte). Tuttavia, anche per il kernel, bisogna garantire la mutua esclusione.

Problema: il busy waiting è ancora presente, in quanto quando i processi trovano il lock occupato, continuano a restare all'interno del while e a chiedere: ora la sezione critica è libera? Senza mai bloccarsi.

Terza soluzione: alternanza stretta

Idea di base

Si basa su una variabile globale condivisa - turno - che indica quale processo ha il diritto di entrare nella sezione critica. Il funzionamento è questo: se turno è uguale a 0, allora è il turno di P0 e P1 e gli altri processi devono aspettare. Quando un processo termina la propria sezione critica, imposta turno all'altro processo.

Problemi

Notiamo che questa soluzione garantisce la mutua esclusione, ma ha un difetto evidente: i processi devono alternarsi strettamente. P0 non può entrare due volte di fila nella sezione critica, deve aspettare che P1 ci entri almeno una volta, anche se P1 non ha nulla da fare. Questa viola la proprietà numero 3: un processo fuori dalla sezione critica (P1 che non ha nulla da fare) sta bloccando un altro processo che vuole entrare nella propria sezione critica.

Nota: P1 potrebbe entrare nella sezione critica, ma non è obbligato a farlo. Tuttavia, se P1 non lo fa, P0 rimane bloccato indefinitamente ad aspettare (violazione della proprietà 4). PO, prima di poter rientrare nuovamente nella sezione critica, deve aspettare che Pl entri nella sezione critica (non sappiamo quando lo farà e se lo farà), che P1 esegua le operazioni critiche, e che esca e imposti turn $=0$.

Quarta soluzione: algoritmo di Peterson

Idea di base

Migliora il meccanismo dell'alternanza stretta appena descritto. Usa due variabili:

1. Un array interested[2] - un processo segnala se vuole entrare nella propria sezione critica, quindi non è altro che un array di booleani;
2. Una variabile turno - indica a chi cedere la precedenza in caso di conflitto.

Quando un processo vuole entrare, segnala il proprio interesse (interested[i] = True) e cede cortesemente il turno all'altro (turn = j). Se entrambi vogliono

entrare contemporaneamente, uno dei due cede - ma se solo uno vuole entrare, lo fa liberamente senza aspettare. Questo risolve il problema dell'alternanza stretta: un processo può entrare più volte di fila nella sezione critica, purché l'altro processo non sia interessato.

Gestisce correttamente N prcoessi ed è una soluzione puramente software non richiede supporto hardware speciale. Nota: questa versione è "semplificata" e funziona solo con due processi. Esiste anche la versione per N processi ma è più complessa.

Quindi, nell'algoritmo di Peterson turn serve solo a rompere i conflitti. Il flusso è il seguente: P0 vuole entrare e imposta interested a True. P0 cede il turno a P1. P0 controlla: P1 è interessato e il turno è di P1? se la risposta ad almeno una di queste domande è no, allora P0 entra, in caso contrario aspetta.

Nel caso in cui aspetta, tocca a P1. P1 aveva segnalato di voler entrare (in quanto P0 sta aspettando, sennò non avremmo fatto il controllo su P1). P1 cede il turno a PO (a meno che non gliel'abbia già ceduto). Controlla se è il turno di P0 e se P0 è interessato. Poiché la risposta a entrambe queste domande è sì, P0, che si trova all'interno del while dove controlla costantemente se P1 è interessato e se è il suo turno, nota che è finalmente il turno di P0: quindi esce dal while ed entra nella sezione critica. P1 invece resta all'interno del suo while perché P0 è interessato ed è anche il suo turno.

Il meccanismo del cedere il turno è utile per gestire il caso in cui entrambi vogliono entrare contemporaneamente. Turn decide chi sarà ad entrare. Chi ha scritto su turn per ultimo, aspetta.

Nota importante: nella versione di MDR, come è possibile notare in foto, il processo che modifica la variabile turn non "cede" il suo turno all'altro processo, ma mette sè stesso dentro il turno. Invece di cedere il turno, si limita a scrivere sè stesso all'interno del turno. Anche se qui turn ha un significato diverso: invece di controllare nel while se è il turno dell'altro processo, si guarda se è il proprio turno, ma qua con turno si intende "tocca a me aspettare".

Problemi: è ancora presente busy waiting, in quanto il processo resta in un ciclo a non fare nulla, consumando inutilmente la CPU. Si può generalizzare al caso di N processi (e non 2), ma sarebbe più complesso. Questo approccio può avere problemi sui moderni multi-processori a causa del riordino delle istruzioni (flag e turn potrebbero essere invertiti), rompendo così le garanzie dell'algoritmo.

Lezione 7 (26/03/26) - Lezione 6 integrata col resto

Quinta soluzione: Sleep & Wakeup

Il problema dello spin lock

Tutte le soluzioni viste finora (Peterson, TSL, XCHG) fanno spin lock: il processo che trova la sezione critica occupata resta in un loop attivo a consumare CPU. Spin lock è il nome che si dà al meccanismo di lock che usa busy waiting come stratregia di attesa, il processo "gira su sè stesso" (spin) in un loop controllando ripetutamente se il lock si è liberato, senza mai cedere la CPU.

Questo causa il problema dell'inversione di priorità: immagina un processo a bassa priorità L che è dentro la sezione critica, e un processo ad alta priorità H che vuole entrare. H ha priorità maggiore, quindi lo scheduler gli dà la CPU. Ma H non può entrare, quindi fa spin lock. L non riceve mai la CPU perché H ha priorità più alta e sta "occupando" il processore girando nel while. Risultato: H aspetta L, ma L non può avanzare perché H monopolizza la CPU. Si dice che la priorità è invertita in quanto il processo ad alta priorità è bloccato da quello a bassa priorità (opposto di come dovrebbe funzionare lo scheduling).

Starvation: situazione che si verifica (come nel caso precedente) quando un processo è pronto ad eseguire ma non riceve mai la CPU, perché altri processi gli passano sempre davanti (ovvero hanno più priorità per lo scheduler). Il processo non è bloccato su una risorsa, non è in deadlock, non sta dormendo. Si trova nella coda dei processi pronti, ma lo scheduler, sistematicamente, preferisce altri processi (con più priorità) a lui.

La soluzione al busy waiting: bloccarsi passivamente

Invece di girare a vuoto, un processo che non può entrare dovrebbe addormentarsi, cioè essere rimosso dalla coda dei processi pronti (ready) e messo nello stato blocked. Così la CPU viene liberata per altri processi.

Le due primitive sono:

- sleep() - il processo chiamante si sospende. Passa dallo stato running a blocked. Non consuma più CPU. Potrà tornare in esecuzione solo quando qualcun altro lo sveglia.
- wakeup(processo) - sveglia il processo specificato. Lo sposta dallo stato blocked a ready, rendendolo nuovamente disponibile per lo scheduler.

Definizione di primitiva: una primitiva è un'operazione fondamentale messa a disposizione dal SO (o dal linguaggio) che non può essere scomposta in

operazioni più semplici. Esempi di primitiva: sleep e wakeup, down e up dei semafor, send e receive per lo scambio di messaggi, wait e signal dei monitor.

Il problema del produttore-consumatore

È un caso classico: un produttore genera dati e li inserisce in un buffer limitato di dimensione N. Un consumatore preleva dati dal buffer. Due vincoli fondamentali:

- Il produttore non può inserire se il buffer è pieno (count $==\mathrm{N}$)
- Il consumatore non può prelevare se il buffer è vuoto (count $==0$)

La variabile condivisa count tiene traccia di quanti elementi ci sono nel buffer.

La logica del codice

Produttore: in un ciclo infinito, produce un item. Se il buffer è pieno (count $==\mathrm{N}$), si addormenta. Altrimenti inserisce l'item e incrementa count. Se dopo l'incremento count $==1$, significa che il buffer era vuoto e il consumatore potrebbe star dormendo → lo sveglia con wakeup(consumer).

Consumatore: in un ciclo infinito, se il buffer è vuoto (count $==0$), si addormenta. Altrimenti preleva un item e decrementa count. Se dopo il decremento count $==\mathrm{N}-1$, significa che il buffer era pieno e il produttore potrebbe star dormendo → lo sveglia con wakeup(producer).

Perché non funziona: il lost wakeup

Il problema è che count non è protetto. Ecco lo scenario critico:

1. Il buffer è vuoto, count $==0$
2. Il consumatore legge count $==0$ e sta per chiamare sleep()... ma prima che la sleep venga eseguita, avviene un context switch
3. Il produttore prende la CPU, inserisce un item, count diventa 1
4. Il produttore vede count $==1 \rightarrow$ chiama wakeup(consumer)
5. Ma il consumatore non sta ancora dormendo (non ha ancora eseguito sleep)! Il wakeup va perso, non ha effetto
6. Il consumatore riprende la CPU ed esegue sleep() → si addormenta
7. Il produttore continua a produrre, riempie il buffer fino a count $==N \rightarrow$ chiama sleep() → si addormenta

Risultato: entrambi dormono, nessuno può svegliare l'altro. Il sistema è bloccato per sempre.

Il nocciolo del problema è che tra il controllo di count e la chiamata a sleep() c'è una finestra in cui l'altro processo può inserirsi - è lo stesso tipo di race condition del lock semplice senza TSL.

Il tentativo di soluzione: bit di attesa wakeup

L'idea è aggiungere un flag che "salva" il wakeup se arriva quando il processo non sta dormendo. Quando il processo prova a fare sleep, prima controlla il flag: se è settato, lo azzera e non si addormenta. Così il wakeup non va perso.

Funziona per due processi, ma non scala: con molti produttori e consumatori servirebbero più bit, e la situazione diventa ingestibile. Per questo si passa ai semafori, che sono la soluzione definitiva al problema.

Sesta soluzione: Semafori

Definizione

I semafori sono una generalizzazione delle primitive sleep e wakeup. Sono uno strumento di sincronizzazione che possiede uno stato interno rappresentato da un valore intero, con un'invariante fondamentale: il valore non può mai essere negativo.

Su questo valore agiscono due operazioni: Down (wait): tenta di decrementare il valore del semaforo. Se il valore è maggiore di 0 , lo decrementa e il processo continua l'esecuzione normalmente. Se il valore è 0, il processo si blocca (viene messo nello stato blocked) e resta in attesa su quel semaforo fino a quando non verrà risvegliato. Rappresenta l'ingresso in una sezione critica.

Up (signal): incrementa il valore del semaforo. Se ci sono processi bloccati in attesa su quel semaforo, uno di essi viene risvegliato e può procedere. Se non ci sono processi in attesa, il valore viene semplicemente aumentato di 1. Rappresenta l'uscita dalla sezione critica.

Entrambe le operazioni devono essere atomiche. Senza atomicità, down e up sarebbero a loro volta soggette a race condition sul valore interno del semaforo, vanificando l'intero meccanismo.

Uso come mutex (mutua esclusione)

Per garantire la mutua esclusione su una sezione critica, si definisce un semaforo M (detto "mutex") con valore iniziale 1, ovvero il numero di processi che ammettiamo a lavorare dentro la sezione critica contemporaneamente (al massimo 1).

La struttura è: down(M) // enter region // sezione critica up(M) // leave region Quando il primo processo esegue down(M), il semaforo passa da 1 a 0 e il processo entra nella sezione critica. Tutto funziona senza blocchi, dato che solo una down può provocare un bloccaggio.

Se un secondo processo tenta di entrare, esegue down(M) ma il valore è già 0 : un ulteriore decremento lo porterebbe a -1 , violando l'invariante di non negatività. Quindi il processo viene bloccato, che è esattamente ciò che vogliamo.

Quando il primo processo esce, esegue up(M): il semaforo torna a 1 (oppure, se c'è un processo bloccato, questo viene risvegliato direttamente).

Mutex: semaforo inizializzato a 1 che garantisce mutua esclusione (da cui il nome, mutual exclusion).

Problema del produttore-consumatore (con semafori)

Descrizione del problema

Si ha un buffer condiviso di capacità N slot, e due tipi di processi:

- Produttore: genera dati e li inserisce nel buffer
- Consumatore: preleva dati dal buffer per elaborarli

Due problemi possono verificarsi:

1. Buffer overflow: il produttore continua a inserire dati anche se il buffer è pieno
2. Buffer underflow: il consumatore cerca di prelevare dati da un buffer vuoto

Soluzione con tre semafori

Il problema viene risolto con tre semafori: Semaforo mutex (usato nello stile mutex): inizializzato a 1, cioè il numero massimo di processi che ammettiamo a lavorare dentro la sezione critica. Serve a garantire che solo un processo alla volta modifichi il buffer, evitando che produttore e consumatore tocchino lo stesso slot contemporaneamente.

Semaforo empty (contatore di slot vuoti): inizializzato a N, cioè il numero di posti liberi. Serve al produttore per sapere se può inserire dati. Quando empty vale 0 , il buffer è pieno e il produttore si blocca.

Semaforo full (contatore di slot pieni): inizializzato a $\mathbf{0}$, cioè il numero di posti occupati (in un buffer appena creato è ovviamente vuoto).

Serve al consumatore per sapere se c'è qualcosa da prelevare. Quando full vale 0 , il buffer è vuoto e il consumatore si blocca.

Empty e full sono semafori contatori (a conteggio): il loro valore non oscilla solo tra 0 e 1 come un mutex, ma traccia il numero effettivo di slot vuoti e pieni.

Tuttavia, empty e full da soli non bastano: rimane il problema della struttura dati condivisa (il buffer), risolto dal mutex.

Codice del produttore

`while(true) { prodotto = genera_dato() down(empty) // aspetta che ci sia un

posto libero (decrementa empty) down(mutex) // entra in sezione critica

inserisci_nel_buffer(prodotto)

up(mutex) // esci dalla sezione critica

up(full) // avvisa che c'è un nuovo dato pronto (incrementa full)

}`

Il produttore è un consumatore di slot vuoti e un produttore di slot pieni. Fa down(empty) prima di inserire (decrementa il numero di slot vuoti, come se ne prenotasse uno da riempire) e up(full) dopo aver inserito (incrementa il numero di slot pieni).

Codice del consumatore

`while(true) { down(full) // aspetta che ci sia almeno un dato (decrementa full)

down(mutex) // entra in sezione critica

dato = preleva_dal_buffer()

up(mutex) // esci dalla sezione critica

up(empty) // avvisa che si è liberato un posto (incrementa empty)

elabora(dato)

}`

Il consumatore è un consumatore di slot pieni e un produttore di slot vuoti. Fa down(full) prima di prelevare (decrementa il numero di slot pieni) e up(empty) dopo aver prelevato (incrementa il numero di slot vuoti).

La sezione critica del consumatore è solo preleva_dal_buffer(): è l'unica operazione che tocca la struttura dati condivisa, e viene avvolta da down(mutex) e up(mutex).

Perché elabora(dato) è fuori dalla sezione critica

elabora(dato) (o consume_item) non viene messo dentro la sezione critica perché non tocca il buffer condiviso. Non ha senso tenerlo lì: le sezioni critiche devono essere minimali, contenendo solo lo stretto necessario che accede a dati condivisi. Tenere operazioni non necessarie dentro la sezione critica ridurrebbe inutilmente il parallelismo.

Errore critico: invertire le down nel consumatore

Se nel consumatore si invertono le prime due down, facendo prima down(mutex) e poi down(full): // SBAGLIATO down(mutex) // entra in sezione critica down(full) // aspetta che ci sia un dato... ma siamo dentro la sezione critica!

Se il buffer è vuoto, il consumatore si addormenta sulla down(full) mentre detiene il mutex. A quel punto il produttore, che è l'unico che potrebbe svegliarlo facendo up(full), non può procedere perché per inserire un dato dovrebbe fare down(mutex) - ma il mutex è bloccato dal consumatore addormentato. Nessuno dei due può avanzare: è un deadlock.

La regola generale: non si devono mai fare operazioni potenzialmente bloccanti all'interno di una sezione critica.

E se si invertono le up?

Invertire le up (fare prima up(full) e poi up(mutex) nel produttore, o prima up(empty) e poi up(mutex) nel consumatore) non causa problemi di correttezza. Tuttavia, per simmetria e leggibilità, si preferisce mantenere l'ordine speculare rispetto alle down.

Mutua esclusione nei thread utente

Il problema con i semafori

Nel modello a thread utente (modello 1-a-molti), i thread vengono gestiti interamente nello user space da un runtime environment (una libreria, il runtime system), senza che il kernel ne sia a conoscenza. Il kernel vede un solo processo; al suo interno, il runtime gestisce più thread.

Questo crea un problema con i semafori: la down è una chiamata bloccante che invoca il kernel. Se un thread utente esegue una down e si blocca, il kernel non sa che esistono altri thread nello stesso processo - blocca l'intero processo. Gli altri thread, che potrebbero eseguire la up per sbloccare la situazione, sono bloccati anche loro perché il processo stesso è bloccato. Risultato: stallo. Non

è possibile usare i semafori classici per gestire la mutua esclusione tra thread utente.

Il problema con lo spin lock puro

Si potrebbe pensare di usare TSL/XCHG con un semplice spin lock, dato che non richiedono syscall (non sono istruzioni privilegiate, sono istruzioni hardware, possono essere eseguite in user space). Tuttavia neanche questa soluzione funziona nel modello a thread utente.

Il motivo: se un thread entra nel ciclo di spin lock, usa tutta la CPU che lo scheduler ha concesso al processo, girando nel loop senza mai cederla. Poiché nel modello a thread utente il cambio di thread avviene solo con cooperazione volontaria (il kernel non sa che ci sono più thread), nessun altro thread dello stesso processo potrà mai eseguire e rilasciare il lock. Il thread che detiene il lock non riceverà mai la CPU, perché il thread in spin la sta monopolizzando.

La soluzione: spin lock con thread_yield

L'idea è combinare lo spin lock con una cessione volontaria della CPU ad ogni tentativo fallito.

Il funzionamento del codice mutex_lock per thread utente è il seguente:

1. Si esegue TSL (o XCHG) per tentare di acquisire il lock: si imposta la variabile mutex a 1 e si recupera nel registro il vecchio valore
2. Si controlla il vecchio valore con CMP:

- Se era $\mathbf{0} \rightarrow$ il lock era libero, siamo riusciti a passarlo da 0 a $1 \rightarrow$ abbiamo acquisito il lock → usciamo dalla funzione
- Se era $\mathbf{1} \rightarrow$ il lock era già occupato → invece di ciclare immediatamente (come farebbe uno spin lock puro), si chiama thread_yield

1. thread_yield cede volontariamente la CPU agli altri thread del processo per un po'. Questo permette al thread che detiene il lock di ricevere la CPU, completare il suo lavoro e rilasciare il lock
2. Quando il thread riprende l'esecuzione dopo la yield, ritenta la TSL dall'inizio

È uno spin lock collaborativo: ad ogni tentativo fallito, prima di riprovare, il thread cede il controllo della CPU. Non è perfetto (resta una forma di polling), ma funziona nel modello a thread utente perché garantisce che gli altri thread abbiano la possibilità di eseguire.

Flusso di esecuzione del codice di mutex_lock: TSL legge il vecchio valore di Mutex, e lo setta a 1 in un colpo solo. Se il vecchio valore era 0 (libero), il lock era libero e così lo abbiamo acquisito. Si salta a ok e si fa ritorno. Se il vecchio valore era 1 (occupato), allora chiamo thread_yield per cedere la CPU. Quando il run-time system cede di nuovo la CPU a questo thread, salta di nuovo a mutex_lock per riprovare. Invece di ciclare a vuoto consumando la CPU, a ogni tentativo fallito cedo il controllo agli altri thread.

Nota: il “set to 1” è intrinseco nella semantica dell’istruzione TSL. Infatti TSL fa due cose contemporaneamente: copia il valore corrente di Mutex in Register, e poi setta Mutex a 1.

Futex (Fast User Space Mutex)

Il problema delle prestazioni

In un sistema operativo classico, ogni operazione down e up su un semaforo richiede una syscall. Questo significa che il processore deve passare da user mode a kernel mode, operazione costosa in termini di tempo e risorse a causa del context switch. Se si va a misurare l’overhead dovuto a queste chiamate di sistema, ci si rende conto che è un grande spreco, specialmente nei casi in cui il lock è quasi sempre libero e la contesa è rara.

L’idea del futex

Il futex (Fast User Space Mutex) è un caso particolare di mutex implementato dal kernel Linux. Ha due componenti:

Componente user space: è codice eseguito nello spazio utente (tramite librerie ordinarie, niente di particolarmente complesso). Funziona in modo molto simile al mutex per thread utente: usa una variabile condivisa e la TSL (o XCHG) per verificare se il lock è disponibile. Se il lock è libero, lo acquisisce e ritorna immediatamente. In questo caso non viene fatta nessuna syscall: è tutto una chiamata di libreria, da procedura a procedura, da utente a utente. Il costo è minimo.

Componente kernel: entra in gioco solo in caso di contesa. Se la componente user space scopre che la risorsa è occupata (il lock è già detenuto da qualcun altro), allora invoca una syscall che chiede al kernel di bloccare il thread in modo passivo, esattamente come farebbe la down di un semaforo. Il thread viene messo nello stato blocked e verrà risvegliato quando il lock si libera.

Nota: il thread che viene bloccato dal kernel deve necessariamente essere un thread kernel, sennò il kernel non può nemmeno essere a conoscenza della sua

esistenza.

Il vantaggio chiave

Con un semaforo classico (o mutex standard), il controllo avviene sempre nel kernel, quindi si paga sempre il costo della syscall, anche quando il lock è libero e non c'è nessuna contesa.

Con il futex, il controllo avviene nello spazio utente. Il costo della syscall si paga solo quando è necessario bloccarsi. Nei casi di non contesa o scarsa contesa (che nella pratica sono la maggioranza), il lock viene acquisito e rilasciato interamente in user space, con un costo drasticamente inferiore.

Contesa: più thread vogliono acquisire lo stesso lock nello stesso momento. Si parla di situazioni di scarsa contesa se capita raramente che due thread vogliano il lock contemporaneamente, e la maggior parte delle volte il lock è libero. Ogni tanto però qualcuno deve aspettare. Si parla, invece, di alta contesa se molti thread competono frequentemente per lo stesso lock. Il lock è quasi sempre occupato e i thread passano molto tempo in attesa.

L'approccio con futex è ottimizzato per i casi in cui ci sia scarsa contesa (o nessuna contesa, che è il caso più comune in molti programmi reali).

Con questo approccio il costo della syscall si paga solo nei rari casi in cui serve davvero bloccarsi. Con un semaforo classico, invece, si paga la syscall sempre, anche quando il lock è libero e nessuno sta competendo.

Nota: questo approccio è utilizzato nel modello 1-a-1, e non in quello 1-amolti (tipico dei thread utente, dove invece si usa spin lock con yield), per i motivi spiegati prima.

In sintesi: il futex ottimizza il caso comune (nessuna contesa → nessuna syscall) e ricade sul kernel solo nel caso raro (contesa → syscall per blocco passivo).

Monitor

Perché i monitor se esistono i semafori?

I semafori funzionano, ma sono proni ad errori: il programmatore deve ricordarsi di inserire sempre le down e le up nell'ordine corretto, e dimenticarne una (o invertirne l'ordine) può causare deadlock o violazioni della mutua esclusione. Sono strumenti di basso livello, "pesanti" da usare correttamente.

I monitor sono uno strumento di più alto livello, offerto direttamente dal linguaggio di programmazione. I semafori invece sono universali: essendo

chiamate di sistema offerte dal SO, possono essere usati da qualsiasi linguaggio su qualsiasi sistema operativo. I monitor devono essere supportati dal linguaggio, e i linguaggi che li offrono forniscono sempre anche una gestione dei thread integrata. I monitor si usano esclusivamente nella programmazione multi-thread.

Cos'è un monitor

Un monitor è una specie di oggetto con strutture dati e variabili interne private, e un insieme di funzioni pubbliche che rappresentano l'unico modo per accedere a quelle strutture dati. Non è possibile accedere direttamente ai dati interni del monitor dall'esterno: bisogna passare per le sue funzioni.

La garanzia fondamentale è che tutte le funzioni definite all'interno del monitor vengono eseguite in mutua esclusione. Fintanto che un thread è dentro il monitor (cioè sta eseguendo una delle sue funzioni), nessun altro thread può chiamare qualsiasi funzione di quel monitor. Se un altro thread ci prova, viene bloccato (non è un errore: è il meccanismo di mutua esclusione che funziona come previsto).

Esempio: se ho un buffer definito dentro un monitor con le funzioni insert e extract, quando T1 sta eseguendo insert, nessun altro thread può chiamare né insert né extract finché T1 non ha finito.

La cosa speciale è che il programmatore non deve fare nulla per ottenere la mutua esclusione, a parte dichiarare la classe come monitor. Le funzioni interne sono scritte come normali funzioni, indistinguibili da quelle fuori dal monitor. È l'implementazione del monitor stesso a inserire gli strumenti necessari (tipicamente semafori nascosti) per garantire la mutua esclusione in modo automatico.

Variabili condizione

I semafori nel problema produttore-consumatore non servivano solo per la mutua esclusione, ma anche per la sincronizzazione (aspettare che il buffer non sia pieno/vuoto). I monitor risolvono questo con le variabili condizione: strumenti interni al monitor su cui un thread può eseguire due operazioni:

- wait(condizione): il thread si blocca in attesa che quella condizione venga segnalata. Il thread rilascia automaticamente il monitor, permettendo ad altri di entrarvi.
- signal(condizione): il thread segnala che la condizione si è verificata, risvegliando un thread bloccato su quella condizione (se ce n'è uno).

Le variabili condizione hanno delle etichette (nomi) perché i motivi per cui un thread si blocca possono essere diversi. Ad esempio, nel produttoreconsumatore si definiscono due variabili condizione: full (su cui si blocca il produttore quando il buffer è pieno) e empty (su cui si blocca il consumatore quando il buffer è vuoto). Non si associa nessun valore a queste variabili, perché le variabili condizione non hanno stato interno.

Differenza fondamentale rispetto ai semafori: i semafori hanno un valore intero che può crescere e "accumula" le signal. Le variabili condizione non hanno memoria: se viene fatta una signal e nessuno è in attesa, la signal va persa. Sono un meccanismo di wait e signal senza stato. (Nota: neanche le primitive sleep e wakeup avevano stato interno - era lo stesso problema del lost wakeup. Sono stati i semafori a introdurre il valore intero come soluzione.)

Perché nel monitor non si verifica il problema del lost wakeup: nel momento in cui un thread controlla una condizione (es. count $==$ N) e decide di addormentarsi, nessun altro thread può interromperlo, dato che la mutua esclusione del monitor garantisce che al più un thread è attivo alla volta dentro il monitor. La finestra tra "controllo" e "addormentamento" che causava problemi con sleep/wakeup è eliminata.

Il problema della signal: due thread dentro il monitor

Supponiamo che T1 sia dentro il monitor e si blocchi con wait(empty) perché il buffer è vuoto. T1 rilascia il monitor. T2 entra, inserisce un elemento, e chiama signal(empty) per svegliare T1. A questo punto T1 è stato risvegliato e T2 è ancora dentro il monitor: ci sono due thread pronti a eseguire codice dentro il monitor. Se entrambi procedono, la mutua esclusione crolla. Come si risolve?

Soluzione di Hoare (signal & wait): quando T2 lancia la signal, si ferma immediatamente e cede il posto a T1. T2 viene sospeso dal SO e messo in una coda speciale (coda dei segnalatori). T1 riprende l'esecuzione da dove si era interrotto. Quando T1 finisce la sua operazione e esce dal monitor, T2 viene risvegliato e può riprendere. In questa soluzione, chi lancia la signal si autoaddormenta.

Soluzione Mesa (signal & continue): quando T2 chiama signal, continua a lavorare fino alla fine della sua funzione. T1 viene segnato come pronto, ma deve aspettare che T2 esca dal monitor prima di poter rientrare. È la soluzione utilizzata in molti sistemi reali. Ha un caso limite: se T2, dopo la signal, compie un'operazione che invalida la condizione per cui T1 era stato svegliato (es. estrae un elemento dal buffer dopo aver segnalato che ce n'era uno), T1 si

risveglia in una situazione che non è più valida. Per questo motivo, nella soluzione Mesa, il thread risvegliato deve ricontrollare la condizione dopo il risveglio (usando un while invece di un if).

Soluzione Pascal (signal & return): la signal è sempre l'ultima istruzione del metodo. Quando un thread ha terminato il suo compito, alla fine esegue la signal ed esce immediatamente dal monitor. Il thread risvegliato riprende il controllo subito dopo. Il vantaggio è che non ci sono collisioni: segnalatore e risvegliato non possono mai essere dentro il monitor contemporaneamente, perché il segnalatore è appena uscito. Lo svantaggio è che è poco flessibile: il programmatore è costretto a strutturare le funzioni in modo che la signal sia sempre l'ultima riga.

Nota: in questo problema del produttore-consumatore risolto con i monitor, usiamo l'approccio Pascal (signal and return). Come viene garantita la mutua esclusione? Quando il produttore sta eseguendo pc_monitor.insert(), è "dentro" il monitor. Se il consumatore prova a chiamare pc_monitor.remove(), il semaforo nascosto del monitor lo blocca automaticamente. Il consumatore resta in attesa finché il produttore non esce da insert(). A quel punto il semaforo viene rilasciato e il consumatore può entrare in remove().

Il vantaggio dei monitor è proprio questo: rispetto all'approccio con semafori espliciti, la mutua esclusione è automatica.

Monitor e pthread

Le librerie pthread implementano variabili condizione che, come quelle descritte, non hanno stato interno. Devono sempre funzionare insieme a un mutex (che svolge il ruolo della mutua esclusione del monitor). I dettagli di utilizzo verranno approfonditi nella parte di laboratorio.

Scambio di messaggi tra processi (Message Passing)

Il paradigma

I processi sono isolati tra loro e per comunicare si scambiano messaggi tramite due primitive:

- send(destinatario, messaggio): invia un messaggio
- receive(mittente, messaggio): riceve un messaggio

Questo paradigma risolve anche problemi di sincronizzazione, perché la ricezione è intrinsecamente bloccante: un consumatore che esegue una receive su una coda vuota viene sospeso dal sistema operativo finché non arriva un messaggio. Non spreca cicli di CPU. Allo stesso modo, se la coda dei

messaggi è piena, la send può diventare bloccante: il produttore si ferma finché il consumatore non libera spazio leggendo un messaggio.

Nota importante: parlare di messaggi non significa parlare di comunicazione di rete. Siamo nell'ambito della comunicazione tra processi (IPC) e parliamo di chiamate di sistema. La send/receive prevede di copiare l'informazione nella memoria del kernel (mailbox) per poi copiarla dalla mailbox alla memoria del processo destinatario.

Modello a Mailbox (indirizzamento indiretto)

Il modello a mailbox è l'evoluzione dello scambio di messaggi diretto "da uno a uno". Invece di mandare un messaggio direttamente a un processo specifico (es. "invio a T1"), i messaggi vengono depositati in un contenitore intermedio chiamato mailbox. Qualsiasi processo può leggere dalla mailbox, rendendo la comunicazione più flessibile.

Vantaggi e svantaggi

Svantaggio principale: l'overhead. È il sistema con il maggiore overhead tra quelli visti, perché l'informazione deve essere incapsulata in messaggi e copiata più volte tra spazi di lavoro diversi (utente → kernel → utente). Anche i semafori richiedono syscall, ma non prevedono copie multiple dei dati. Con la memoria condivisa, ad esempio, i processi scrivono direttamente sulla stessa area di memoria fisica, senza copie intermedie.

Vantaggio principale: la trasparenza di localizzazione. Due processi che comunicano tramite messaggi possono trovarsi sulla stessa macchina (comunicazione locale, efficiente) oppure su macchine diverse (comunicazione via rete, con overhead maggiore). Il codice del programmatore non cambia: il modello a messaggi si adatta sia a scenari paralleli locali che a scenari paralleli distribuiti su più macchine.

MPI (Message Passing Interface)

La libreria MPI è il principale standard per il calcolo distribuito e parallelo, ed è tutt'altro che obsoleta (è ampiamente usata in ambito scientifico e HPC). L'idea è creare flussi multipli (processi) che comunicano tra loro tramite messaggi, e poi lanciare il programma sfruttando tutta la capacità di calcolo di un cluster (più computer collegati in rete che lavorano insieme come se fossero un unico sistema, si usa quando un singolo computer non ha abbastanza potenza di calcolo, tipicamente in ambito scientifico, come per simulazioni fisiche, previsioni meteo e bioinformatica).

I processi che girano sulla stessa macchina comunicano tramite messaggi locali in modo altamente efficiente. Quando i processi si trovano su macchine diverse, comunicano tramite lo stack di rete, con overhead maggiore. Il programmatore non deve preoccuparsi di questa distinzione: scrive il codice una sola volta e MPI gestisce la comunicazione in modo trasparente, indipendentemente da dove i processi si trovano.

Quello che è meno usato oggi per la sincronizzazione locale tra thread è il modello send/receive diretto. Per la comunicazione tra processi sulla stessa macchina si preferiscono generalmente meccanismi più efficienti come la memoria condivisa con semafori o mutex.

Differenza principale: nel caso del modello send/receive c'è un indirizzamento diretto, in quanto il mittente specifica il destinatario. Nel caso del modello a Mailbox, c'è un indirizzamento indiretto, in quanto qualsiasi processo che abbia accesso a quella mailbox può fare receive e prelevare il messaggio. La differenza quindi riguarda l'indirizzamento.

Lezione 09/04/26

Problema dei 5 filosofi

Descrizione del problema

Problema classico di sincronizzazione che modella l'accesso esclusivo a un numero limitato di risorse da parte di processi in concorrenza.

5 filosofi sono seduti attorno a un tavolo rotondo e possono fare solo due cose: pensare e mangiare. Tra ogni coppia di filosofi adiacenti c'è una forchetta (5 forchette in totale). Per mangiare, un filosofo ha bisogno di entrambe le forchette adiacenti (quella alla sua sinistra e quella alla sua destra). Se una forchetta è disponibile e l'altra no, il filosofo si trova in una situazione potenzialmente bloccante.

Filosofi e forchette sono numerati da 0 a 4 . Il filosofo i ha alla sua sinistra la forchetta i e alla sua destra la forchetta ($\mathrm{i}+1$) mod N .

Perché proprio 5 filosofi?

Si vuole trovare una soluzione generale valida per N filosofi. 5 è il numero minimo che permette di trovare una soluzione generale che massimizza il numero di filosofi che possono mangiare contemporaneamente (al massimo 2 su 5). Con 4 filosofi esiste una soluzione ad hoc che non è generalizzabile. In generale, con un numero di filosofi pari esistono soluzioni ad hoc, quindi bisogna cercare di trovare soluzioni per un numero dispari di filosofi, e 5 è il più

piccolo numero dispari che rende il problema non banale e più semplice da visualizzare.

take_fork: operazione di sincronizzazione

take_fork(k) è una chiamata potenzialmente bloccante: se la forchetta k è già in uso da un altro filosofo, il chiamante si blocca in attesa che venga rilasciata.

Soluzione 1: approccio naïf (senza controllo)

int $\mathrm{N}=5$ function philosopher(int i$)$ think() take_fork(i) take_fork((i +1$) \operatorname{mod~} \mathrm{N}$) eat() put_fork(i) put_fork((i+1) mod N)

Ogni filosofo prende prima la forchetta sinistra, poi la destra. Problema: se tutti i filosofi prendono la forchetta sinistra nello stesso istante, ognuno resta bloccato in attesa della forchetta destra, che è già stata presa dal vicino. Nessuno può procedere → deadlock.

Soluzione 2: controlla con rilascio, riprova

Idea: ogni filosofo prende una forchetta, poi controlla se la seconda è disponibile. Se non lo è, rilascia la prima forchetta e riprova.

Problema: se tutti i filosofi eseguono questa strategia in modo sincrono (prendono, controllano, rilasciano, aspettano lo stesso tempo, riprovano), si genera un livelock (deadlock attivo): nessuno è formalmente bloccato (tutti continuano a eseguire operazioni), ma nessuno riesce mai a mangiare.

Soluzione 3: rilascio con pausa random

Idea: uguale alla soluzione 2 , ma dopo aver rilasciato la forchetta il filosofo aspetta un tempo casuale prima di riprovare.

Questo rompe la simmetria e rende improbabile che tutti i filosofi riprovino nello stesso istante. È simile alla soluzione usata nel protocollo ALOHA per le reti non coordinate (senza access point): quando due stazioni trasmettono contemporaneamente e si verifica una collisione, ciascuna aspetta un tempo random prima di ritrasmettere.

Limite: la soluzione è probabilistica - non garantisce formalmente l'assenza di starvation. Infatti, un filosofo particolarmente sfortunato, potrebbe non riuscire mai ad accedere alla risorsa.

Soluzione 4: semaforo mutex globale

Idea: usare un singolo semaforo mutex per proteggere l'intera operazione di acquisizione delle forchette. Un filosofo acquisisce il lock, prende entrambe le

forchette, e poi rilascia il lock.

Problema di throughput: in questo modo al massimo un filosofo alla volta può mangiare, anche se in teoria due filosofi non adiacenti potrebbero mangiare contemporaneamente. Si perde parallelismo.

Soluzione 5: semafori (soluzione di Tanenbaum)

Questa è la soluzione corretta basata sui semafori. Introduce le seguenti strutture dati:

- state[N]: vettore di N elementi, dove ogni elemento può valere THINKING (0), HUNGRY (1) o EATING (2).
- mutex: un semaforo binario (inizializzato a 1), usato per proteggere l'accesso alla sezione critica (la modifica di state[]).
- $\mathbf{s}$ [N]: un vettore di N semafori, ciascuno inizializzato a $\mathbf{O}$. Questi non sono mutex: servono per permettere a un filosofo di addormentarsi quando non può mangiare, e di essere risvegliato quando le condizioni cambiano. Ognuno di questi semafori rappresenta la risposta alla domanda "il filosofo i ha il permesso di mangiare?". Se il valore è 0, allora non ha il permesso di mangiare. Con una down il filosofo si addormenta. Se il valore è 1, allora ha il permesso di mangiare, fa una down e il semaforo torna a 0 e il filosofo mangia.

Nota: questi ultimi semafori non sono mutex perché non inizializzati a 1 , ma a 0 , e non proteggono nessuna sezione critica. Sono usati come meccanismo di segnalazione.

Ogni filosofo esegue un loop infinito: pensa, prova a prendere le forchette, mangia, le rilascia.

take_forks(i)

1. down(mutex) - entra nella sezione critica.
2. state[i] = HUNGRY - il filosofo dichiara di avere fame.
3. test(i) - controlla se può mangiare.
4. up(mutex) - esce dalla sezione critica.
5. down(s[i]) - se test non ha messo lo stato a EATING, il filosofo si blocca qui (il semaforo è a 0). Se invece test ha fatto up(s[i]), il filosofo passa subito.

Nota: facciamo la down prima che il filosofo possa mangiare (prima di eat), perché s[i] indica solamente se il filosofo ha il permesso di mangiare. Se s[i] è 1

allora ha il permesso, con la down torna a 0 e prosegue. In caso contrario, si blocca. Si risveglierà solo più avanti, quando i suoi vicini avranno controllato per lui e saranno sicuri che il filosofo può mangiare, e quindi riprenderà l'esecuzione da quel punto.

test(i)

Controlla tre condizioni:

1. state[i] $==$ HUNGRY - il filosofo vuole mangiare.
2. state[LEFT] != EATING - il filosofo alla sinistra non sta mangiando.
3. state[RIGHT] != EATING - il filosofo alla destra non sta mangiando.

Se tutte e tre sono vere, significa che entrambe le forchette del filosofo i sono libere (perché non sono usate dai filosofi adiacenti). Allora:

- state[i] $=$ EATING
- up(s[i]) - risveglia il filosofo i (o impedisce che si blocchi sulla successiva down(s[i])).

put_forks(i)

1. down(mutex) - entra nella sezione critica.
2. state[i] = THINKING - il filosofo torna a pensare.
3. test(LEFT) - controlla se il filosofo alla sinistra ora può mangiare (dato che i ha appena liberato una forchetta). Se sì, lo risveglia con up(s[LEFT]).
4. test(RIGHT) - stessa cosa per il filosofo alla destra.
5. up(mutex) - esce dalla sezione critica.

Il punto fondamentale: quando un filosofo finisce di mangiare, non si limita a rilasciare le risorse, ma controlla attivamente se i suoi vicini (che potrebbero essere bloccati in down(s[i])) ora possono procedere, e in caso affermativo li risveglia.

Soluzione 6: basata sui monitor

La stessa logica della soluzione con semafori, ma espressa con il meccanismo dei monitor.

Differenze rispetto ai semafori

- Le procedure take_forks e put_forks vengono inserite all'interno del monitor, quindi la mutua esclusione è garantita automaticamente dal monitor stesso.

Non serve un semaforo mutex esplicito.

- Al posto dei semafori s[N], si usano N variabili condizione (una per filosofo). Le variabili condizione servono per permettere a un filosofo di auto-addormentarsi (wait) quando le condizioni non sono soddisfatte, e di essere risvegliato (signal) da un altro filosofo.

test(i) nel monitor

Stesso controllo: se il filosofo i è HUNGRY e nessuno dei due vicini sta mangiando, allora:

- state[i] = EATING
- signal(cv[i]) - risveglia il filosofo i sulla sua variabile condizione.

put_forks(i) nel monitor

Stessa logica della versione con semafori, ma senza down(mutex) / up(mutex) perché il monitor già garantisce la mutua esclusione.

Nota sulle variabili condizione

Le variabili condizione sono essenzialmente delle etichette (label) che identificano una condizione logica per cui un processo si sta bloccando. Non hanno un valore numerico come i semafori: rappresentano semplicemente una coda di attesa associata a una certa condizione. Un altro processo può invocare signal su quella variabile condizione per risvegliare il processo in attesa.

Problema dei lettori e scrittori

Da non confondere col problema dei produttori-consumatori.

Descrizione del problema

Problema classico che modella l'accesso concorrente a una struttura dati condivisa (tipicamente un database) da parte di due tipi di processi:

- Lettori: accedono alla struttura dati in sola lettura.
- Scrittori: modificano la struttura dati.

Vincoli di accesso

Se usassimo la mutua esclusione per qualsiasi tipo di accesso (sia lettura che scrittura), il problema sarebbe risolto banalmente. Però questo approccio è molto restrittivo: dal punto di vista delle prestazioni, avere un lock globale su

tutto il database (sia per lettori che per scrittori) diventa un serio collo di bottiglia, perché impedisce a più lettori di leggere contemporaneamente.

In realtà, più letture concorrenti non creano problemi: leggere un dato non lo modifica, quindi non c'è conflitto tra lettori. L'unico conflitto reale è tra lettura e scrittura (o tra scrittura e scrittura): se una lettura e una scrittura sullo stesso dato avvengono contemporaneamente, il lettore potrebbe leggere un'informazione inconsistente, a metà tra il dato vecchio e quello aggiornato.

Quindi le regole sono:

- Più lettori possono accedere contemporaneamente → nessun conflitto.
- Uno scrittore deve avere accesso esclusivo: nessun altro lettore o scrittore può accedere al database mentre uno scrittore sta scrivendo.

Soluzione basata sui semafori

Strutture dati

- mutex: semaforo binario che protegge la variabile rc. Serve a garantire che l'incremento/decremento di rc avvenga in modo atomico.
- db: semaforo binario che protegge l'intero database. Quando uno scrittore vuole scrivere, fa down(db) per ottenere l'accesso esclusivo.
- rc (reader counter): conta il numero di lettori attualmente presenti nel database.

Differenza tra mutex e db: entrambi sono semafori mutex, ma mentre nel caso di mutex il lock viene acquisito e rilasciato rapidamente, nel caso di db il lock viene acquisito dal primo lettore (o da uno scrittore) e rilasciato potenzialmente molto tempo dopo, dall'ultimo lettore (o dallo scrittore quando finisce).

writer()

Lo scrittore è semplice: fa down(db) per ottenere l'accesso esclusivo al database, scrive, e poi fa up(db) per rilasciarlo. Se il database è già occupato (da un altro scrittore o da lettori), lo scrittore si blocca sulla down(db).

reader()

Il lettore è più complesso. Analizziamolo passo passo. Fase di ingresso (prima sezione critica):

1. down(mutex) - protegge l'accesso a rc.
2. $\mathrm{rc}=\mathrm{rc}+1$ — un lettore in più.

3. if ($\mathrm{rc}==1$) down(db) - solo il primo lettore fa down(db), bloccando l'accesso agli scrittori. Se sono il secondo, terzo, ecc. lettore, il database è già "lockato" dai lettori, non serve un'altra down(db).
4. up(mutex) - rilascia la protezione su rc.

Lettura (fuori dalla sezione critica):

1. read_database() - la lettura avviene fuori dalla sezione critica protetta da mutex. Questo è fondamentale: se fosse dentro, solo un lettore alla volta potrebbe leggere, vanificando l'intero scopo della soluzione.

Fase di uscita (seconda sezione critica):

1. down(mutex) - protegge di nuovo l'accesso a rc.
2. $\mathrm{rc}=\mathrm{rc}-1$ — un lettore in meno.
3. if ($\mathrm{rc}==0$) up(db) - solo l'ultimo lettore fa up(db), sbloccando il database per eventuali scrittori in attesa.
4. up(mutex) - rilascia la protezione su rc.

Scenario: scrittore arriva prima dei lettori

1. Lo scrittore fa down $(\mathrm{db}) \rightarrow$ il database è lockato.
2. Arriva il primo lettore: entra nella sua sezione critica, incrementa rc a 1 , prova a fare down(db) → si blocca, perché db è già a 0 . Il primo lettore resta bloccato dentro la sezione critica (cioè prima di up(mutex)).
3. Arriva un secondo lettore: prova a fare down(mutex) → si blocca fuori dalla sezione critica, perché il primo lettore non ha ancora fatto up(mutex) (è bloccato sulla down(db)).

Questo "side effect" (il primo lettore che blocca anche tutti i successivi) è intenzionale e desiderato: impedisce che i lettori accedano al database mentre uno scrittore sta scrivendo.

Problema di starvation

Questa soluzione favorisce i lettori: finché continuano ad arrivare lettori, rc non scende mai a 0 , e up(db) non viene mai chiamata. Uno scrittore in attesa potrebbe restare bloccato indefinitamente → starvation dello scrittore.

Questo è un problema reale in sistemi dove i lettori sono molti più degli scrittori (es. un sito meteo: migliaia di utenti leggono le previsioni, ma gli aggiornamenti

sono pochi). In questi casi, la soluzione basata sui semafori potrebbe non essere la scelta migliore.

Soluzione n. 1 basata sui monitor

Strutture dati

- rc: reader counter (come nella soluzione a semafori).
- busy_on_write: indica se uno scrittore è attualmente attivo.
- read, write: variabili condizione. read è la coda su cui si bloccano i lettori, write è la coda su cui si bloccano gli scrittori.

Non c'è bisogno di un semaforo mutex: il monitor garantisce automaticamente la mutua esclusione sulle sue procedure.

start_read()

1. Se c'è uno scrittore attivo, il lettore si blocca sulla variabile condizione read.
2. Altrimenti, incrementa rc.
3. Fa signal(read): se ci sono altri lettori in coda (bloccati perché c'era uno scrittore), li risveglia a catena. Il primo lettore risvegliato eseguirà a sua volta signal(read), risvegliando il successivo, e così via. Questo garantisce che quando uno scrittore finisce, tutti i lettori in attesa vengano sbloccati, non solo uno. Tutti questi lettori si sono accumulati in wait(read) dato che non appena un lettore si addormenta, allora ne può entrare un altro nel monitor

Nota: come fanno a entrare dei lettori nel monitor se c'è uno scrittore? I lettori entrano e vanno in attesa perché il lettore è fuori dal monitor. Guardando la funzione writer, infatti, notiamo che write_database è fuori dal monitor. Lo scrittore entra nel monitor solo per potersi assicurare di essere l'unico ad avere accesso alla risorsa, e poi entra alla fine per rilasciarla.

end_read()

Decrementa il contatore dei lettori. Se è l'ultimo lettore, fa signal(write) per risvegliare un eventuale scrittore in attesa.

start_write()

Lo scrittore si blocca se ci sono lettori attivi ($\mathrm{rc}>0$) oppure se un altro scrittore sta già scrivendo (busy_on_write). Se passa, imposta busy_on_write = true.

end_write()

Lo scrittore ha finito. Imposta busy_on_write = false, poi decide chi risvegliare: se ci sono lettori in coda, ne risveglia uno (che a sua volta risveglierà gli altri tramite la catena di signal(read) in start_read). Se non ci sono lettori in coda, risveglia un eventuale scrittore in attesa.

Comportamento complessivo

Questa soluzione è praticamente equivalente alla soluzione basata sui semafori: favorisce i lettori. Finché arrivano lettori, gli scrittori aspettano. Lo scrittore viene servito solo quando non ci sono lettori in coda.

Soluzione n. 2 basata sui monitor (bilanciata)

L'unica differenza rispetto alla soluzione n. 1 è nella start_read:

start_read() - modificata

È stata aggiunta la condizione in_queue(write): il lettore si blocca non solo se c'è uno scrittore attivo, ma anche se c'è almeno uno scrittore in coda. Questo impedisce ai lettori di continuare a entrare quando uno scrittore sta aspettando.

end_write() - invariata

La end_write resta identica alla soluzione n.1: quando uno scrittore finisce, favorisce i lettori (fa signal(read) se ce ne sono in coda).

Comportamento complessivo: alternanza

Questa combinazione produce un meccanismo di alternanza bilanciata tra lettori e scrittori:

1. Un gruppo di lettori entra e legge.
2. Arriva uno scrittore → si mette in coda su write.
3. I nuovi lettori che arrivano vedono in_queue(write) $==$ true → si bloccano anche loro.
4. I lettori attivi finiscono → l'ultimo fa signal(write) → lo scrittore entra e scrive.
5. Lo scrittore finisce → end_write fa signal(read) (perché ci sono lettori in coda) → i lettori entrano.
6. Si ripete il ciclo.

Il pattern è: gruppo di lettori $\boldsymbol{\rightarrow}$ uno scrittore $\boldsymbol{\rightarrow}$ gruppo di lettori $\boldsymbol{\rightarrow}$ uno scrittore $\boldsymbol{\rightarrow}$...

Questa soluzione non favorisce al 100% gli scrittori: anche se nella start_read si bloccano i lettori quando c'è uno scrittore in coda, nella end_write lo scrittore è "gentile" perché dà la precedenza ai lettori. Il risultato è un tradeoff bilanciato: gli scrittori non vengono bloccati indefinitamente (come nella soluzione n.1), ma non hanno nemmeno priorità assoluta.

Importante: in questo modo, dopo che uno scrittore è andato in attesa (entrando in start_write) a causa dei lettori attivi, tutti i nuovi lettori resteranno nella wait e nessuno li potrà risvegliare, dato che nessuno si trova dentro start_read per fare signal alla fine (dato che prima era entrato lo scrittore nel monitor, e quindi ovviamente nella funzione start_read non c'era nessuno).

Nota su in_queue

in_queue(read) e in_queue(write) sono predicati (funzioni booleane) che restituiscono true se c'è almeno un processo in attesa sulla variabile condizione specificata.

Soluzione n. 3 basata sui monitor (favorisce gli scrittori)

L'unica differenza rispetto alla soluzione n. 2 è nella end_write:

end_write() - modificata

Qui la priorità nella end_write è invertita: quando uno scrittore finisce, controlla prima se ci sono altri scrittori in coda (e ne risveglia uno), e solo se non ce ne sono risveglia i lettori. start_read() - invariata rispetto alla soluzione n. 2

Comportamento complessivo

Questa soluzione favorisce gli scrittori: sia nella start_read (i lettori si bloccano se c'è uno scrittore in coda) sia nella end_write (gli scrittori danno la priorità ad altri scrittori). Il rischio simmetrico è la starvation dei lettori se gli scrittori sono molto frequenti.

Riepilogo delle tre soluzioni monitor

	start_read blocca se...	**end_write risveglia prima...**	**Favoris ce**
Sol. 1	busy_on_write	lettori	Lettori
Sol. 2	busy_on_write OR in_queue(write)	lettori	Bilanciat a
Sol. 3	busy_on_write OR in_queue(write)	scrittori	Scrittori

La soluzione n .2 è la più bilanciata delle tre: impedisce ai lettori di fare starvation sugli scrittori (grazie al check in_queue(write) in start_read), ma allo stesso tempo non dà priorità assoluta agli scrittori (perché end_write risveglia prima i lettori).

Lezione (14/04/26)

Scheduling della CPU

Concetti generali

Quando la CPU si libera, il sistema operativo deve scegliere a quale processo assegnarla. Questa decisione è presa da un algoritmo di scheduling.

CPU burst e I/O burst

L'esecuzione di un processo alterna fasi di CPU burst (il processo usa attivamente la CPU) e fasi di I/O burst (il processo è in attesa di un'operazione di I/O). La fase di CPU burst termina quando il processo esegue un'operazione bloccante (es. una richiesta di I/O). Ciò che segue è un periodo di attesa non legato alla CPU.

Processi CPU-bound e I/O-bound

I processi si distinguono in due categorie in base al loro comportamento prevalente:

- CPU-bound: processi che trascorrono la maggior parte del tempo usando la CPU, con CPU burst lunghi e I/O burst rari o brevi. Esempi: mining di bitcoin, thread che esegue il controllo ortografico.
- I/O-bound: processi che trascorrono la maggior parte del tempo in attesa di I/O, con CPU burst brevi e frequenti operazioni di I/O. Esempio: un processo con interfaccia grafica è per sua natura I/O-bound, perché resta in attesa di interazioni da parte dell'utente (ovviamente questo è limitato al thread che si occupa della GUI).

In un sistema reale, un processo non è necessariamente CPU-bound o I/Obound per tutta la sua vita: può cambiare nel tempo. Per esempio, un compilatore all'inizio carica dei file dal disco (I/O-bound), poi esegue la compilazione vera e propria (CPU-bound), e infine scrive i risultati su disco (I/Obound). Non è immediato capire a priori se un processo è CPU-bound o I/Obound: non ce l'hanno scritto in fronte. Bisognerebbe essere capaci di rilevare possibili cambiamenti dinamicamente.

Perché favorire i processi I/O-bound?

Se lo scheduler è in grado di distinguere le due tipologie, dovrebbe favorire i processi I/O-bound. Il motivo è pratico: questi processi non monopolizzano la CPU, la usano per poco tempo e poi la restituiscono. Schedulandoli per primi, "spariscono" velocemente dalla coda dei processi pronti (perché si bloccano subito in attesa di I/O), e nella coda rimangono solo i processi CPU-bound, che poi vengono serviti.

Quando viene attivato lo scheduler

L'algoritmo di scheduling viene invocato in momenti ben precisi:

- Alla terminazione (o creazione) di un processo.
- Quando un processo esegue una chiamata bloccante (es. richiesta di I/O).
- All'arrivo di un interrupt (es. completamento di un'operazione di I/O, che riporta un processo nello stato di pronto).
- Agli interrupt periodici del timer (nei sistemi preemptive).

Scheduler e dispatcher

Lo scheduler è il componente che decide quale processo mandare in esecuzione. Il dispatcher è il componente che effettivamente realizza la decisione: esegue il cambio di contesto, passa in modalità utente e salta all'istruzione del processo selezionato. È il "braccio" dello scheduler.

La latenza di dispatch è il tempo necessario al dispatcher per effettuare il cambio di contesto. Questa latenza dipende dal tipo di context switch: è minore se il cambio avviene tra thread dello stesso processo (condividono lo spazio di indirizzamento) e maggiore se avviene tra thread di processi diversi.

Sistemi preemptive e non-preemptive

- Non-preemptive (senza prelazione): una volta che un processo ottiene la CPU, la mantiene fino a quando non la rilascia volontariamente (terminazione, I/O, ecc.). Lo scheduler non può "togliergli" la CPU.
- Preemptive (con prelazione): lo scheduler può interrompere un processo in esecuzione e assegnare la CPU a un altro processo, tipicamente allo scadere di un quanto di tempo o quando arriva un candidato migliore.

Obiettivi degli algoritmi di scheduling

Gli algoritmi di scheduling operano in ambienti differenti (batch, interattivi, real-time), ognuno con obiettivi specifici.

Obiettivi comuni a tutti i sistemi

- Equità nell'assegnazione della CPU: tutti i processi, salvo casi giustificati da differenze di priorità, dovrebbero essere trattati allo stesso modo. Lo scheduler non deve riservare trattamenti diversi a processi che differiscono per caratteristiche non rilevanti (es. non deve preferire un processo con PID dispari rispetto a uno con PID pari).
- Bilanciamento nell'uso delle risorse: lo scheduler dovrebbe cercare di tenere occupate tutte le risorse del sistema (CPU, dischi, rete) in modo bilanciato.

Obiettivi dei sistemi batch

- Massimizzare il throughput (produttività): il numero di job completati per unità di tempo. Attenzione: questa metrica va interpretata con cautela. Alcuni algoritmi potrebbero "barare" prediligendo i job più brevi, gonfiando il throughput senza necessariamente essere migliori. Confrontare il throughput di due algoritmi ha senso solo a parità di input. Inoltre, sapere a priori quanto durerà un job è in generale impossibile: determinare il tempo di esecuzione di un programma arbitrario è un problema indecidibile, legato al problema della fermata (Halting Problem), quindi in generale conviene eseguire il job direttamente senza cercare di analizzarne il tempo a priori.
- Minimizzare il tempo di turnaround (tempo di completamento): il tempo totale trascorso dall'arrivo di un job alla sua terminazione. Include sia il tempo di attesa in coda sia il tempo di esecuzione effettiva.
- Minimizzare il tempo medio di attesa: il tempo che un processo trascorre nella coda dei processi pronti prima di ottenere la CPU. È una metrica più "pulita" del throughput, perché non risente della durata dei singoli job. Noi assumiamo che un processo batch, una volta schedulato, non farà I/O: è totalmente CPU-bound e si completerà in un lasso di tempo determinato. Il tempo di esecuzione dipende dallo specifico processo, non dallo scheduler: se il processo X ha bisogno di 3 secondi per completarsi, questo non riguarda lo scheduler in alcun modo. Quello su cui lo scheduler può agire è l'ordine in cui i processi vengono serviti, e quindi il tempo di attesa.

Relazione tra tempo di attesa e tempo di completamento

Il tempo di completamento di un processo è la somma del suo tempo di attesa e della sua durata di esecuzione. Dato che la durata è fissa e non dipende dallo scheduler, minimizzare il tempo medio di attesa equivale a minimizzare il tempo medio di completamento. Per questo nelle analisi si può lavorare su entrambe le metriche in modo intercambiabile.

Obiettivi dei sistemi interattivi

- Minimizzare il tempo di risposta: il tempo che intercorre tra il momento in cui l'utente compie un'azione (operazione di I/O) e il momento in cui il processo ottiene la CPU per gestirla.

Le metriche dei sistemi batch (throughput, turnaround) stridono con la natura dei sistemi interattivi: in un sistema interattivo non ci sono "job" che iniziano e finiscono, ma processi che interagiscono continuamente con l'utente. Un SO moderno è sempre tarato su sistemi interattivi, quindi adotta una struttura che favorisce tempi di risposta bassi. Sistemi batch e sistemi interattivi richiedono tecniche di scheduling totalmente diverse.

Obiettivi dei sistemi real-time

- Rispetto delle scadenze: ogni task ha una deadline entro cui deve completarsi.
- Prevedibilità: il sistema deve garantire comportamenti temporali deterministici.

Gli approcci usati per i sistemi interattivi non possono mai garantire il rispetto di una scadenza, e questo non sarebbe accettabile nei sistemi real-time.

Scheduling nei sistemi batch

First-Come First-Served (FCFS)

Algoritmo che sceglie i processi in base al loro ordine di arrivo, usando una semplice coda FIFO. Non fa nessun tipo di scelta e non ha preferenze: il primo arrivato è il primo servito.

È un algoritmo non-preemptive: un processo che ottiene la CPU la mantiene fino al completamento.

Diagramma di Gantt: diagramma temporale che mostra quale risorsa è assegnata a quale attore in ogni istante. Nel nostro caso gli attori sono i processi/thread.

Esempio 1 (tutti arrivano a tempo 0)

Process Arriv Durat

o	**o**	**a**
P1	0	24

Process Arriv Durat

o	o	a
P2	0	3
P3	0	3

Diagramma di Gantt (ordine FCFS):

P1 | P2 | P3 | | 0 | 24 | 27 | 30

- Tempi di attesa: $\mathrm{P} 1=0, \mathrm{P} 2=24, \mathrm{P} 3=27$.
- Tempo medio di attesa (t.m.a.): $(0+24+27) / 3=\mathbf{1 7}$
- Tempo medio di completamento (t.m.c.): $(24+27+30) / 3=\mathbf{2 7}$

Si può fare di meglio? Sì. Se scegliamo prima P2, poi P3, poi P1: | P2 | P3 | P1 | | 0 | 3 | 6 | 30

- Tempi di attesa: $\mathrm{P} 1=6, \mathrm{P} 2=0, \mathrm{P} 3=3$.
- t.m.a.: $(6+0+3) / 3=\mathbf{3}$

Da 17 a 3: un miglioramento enorme. Nota che il tempo finale (30) non cambia - il tempo totale per completare tutti i processi è lo stesso, ma l'ordine influisce sul tempo di attesa dei singoli processi.

Esempio 2 (tempi di arrivo diversi)

Process Arriv Durat

0	0	a
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Diagramma di Gantt (FCFS):

P1 | P2 | P3 | P4 | | 0 | 7 | 11 | 12 | 16

- Tempi di attesa: $\mathrm{P} 1=0, \mathrm{P} 2=7-2=5, \mathrm{P} 3=11-4=7, \mathrm{P} 4=12-5=$ 7. . t.m.a.: $(0+5+7+7) / 4=\mathbf{4 . 7 5}$
- t.m.c.: $(7+9+8+11) / 4=\mathbf{8 . 7 5}$ (dove: P1 completa a 7 , P2 a 11 , P3 a 12, P4 a 16; il tempo di completamento di ciascuno è calcolato come istante di completamento - istante di arrivo).

Shortest Job First (SJF)

Idea: a parità di disponibilità, scegliere sempre il processo più breve. In questo modo si posticipa un processo lungo e se ne anticipa uno breve, riducendo il tempo medio di attesa.

È un algoritmo non-preemptive e presuppone la conoscenza della durata di ogni job.

Perché funziona (intuizione della dimostrazione)

Se scambio l'ordine di due processi consecutivi X e Y dove X è più lungo di Y , sto barattando un decremento del tempo di attesa pari alla durata di X con un aumento pari alla durata di Y. Ma dato che $X>Y$, il decremento è maggiore dell'aumento: il tempo medio di attesa migliora. Questa è una scelta greedy.

Ottimalità

Lo SJF è ottimale (minimizza il t.m.a.) solo se tutti i processi sono disponibili nello stesso momento (stesso tempo di arrivo). Se i tempi di arrivo sono diversi - e questo accade sempre in scenari non ideali - lo SJF non garantisce la soluzione ottimale.

Esempio di SJF non ottimale

Process Arriv Durat

o	**o**	**a**
P1	0	2
P2	0	4
P3	3	1
P4	3	1
P5	3	1

Ordine SJF: a tempo 0 sono disponibili P1 (durata 2) e P2 (durata 4). SJF sceglie P1. P1 finisce a tempo 2. A tempo 2 c'è solo P2 (P3, P4, P5 arrivano a tempo 3), quindi SJF schedula P2. Essendo non-preemptive, anche quando P3, P4, P5 arrivano a tempo 3, la CPU resta a P2 fino a tempo 6. Poi SJF sceglie P3 (tempo 7), P4 (tempo 8), P5 (tempo 9).

- Tempi di attesa: $\mathrm{P} 1=0, \mathrm{P} 2=2, \mathrm{P} 3=6-3=3, \mathrm{P} 4=7-3=4, \mathrm{P} 5=8 -3=5$. . t.m.a.: $(0+2+3+4+5) / 5=\mathbf{2 . 8}$ Ordine alternativo: scheduliamo prima P 2 , poi $\mathrm{P} 3, \mathrm{P} 4, \mathrm{P} 5$, poi P 1 .
- P2 da 0 a 4, P3 da 4 a 5, P4 da 5 a 6, P5 da 6 a 7, P1 da 7 a 9.

- Tempi di attesa: $\mathrm{P} 1=7, \mathrm{P} 2=0, \mathrm{P} 3=1$ (perché P 3 arriva a tempo 3 , non a tempo 0), $\mathrm{P} 4=2, \mathrm{P} 5=3$.

Piccola nota: qui nell'esempio delle slide P3, P4, P5 arrivano a tempo 3, quindi il loro tempo di attesa va calcolato dall'istante 3 , non dall'istante 0 . . t.m.a.: $(7+0+1+2+3) / 5=\mathbf{2 . 6}$$2.6<2.8$: l'ordine alternativo è migliore dello SJF. Questo dimostra che lo SJF non è ottimale quando i tempi di arrivo differiscono.

Shortest Remaining Time Next (SRTN)

È la versione preemptive dello SJF. La differenza fondamentale: non si confronta la durata totale del processo, ma il tempo residuo di esecuzione. Se arriva un nuovo processo il cui tempo di esecuzione è minore del tempo residuo del processo attualmente in CPU, si effettua un context switch (prelazione).

Conviene il context switch?

Nei sistemi batch i context switch sono rari (avvengono solo quando arriva un processo più breve), quindi l'overhead è trascurabile rispetto al vantaggio ottenuto dall'algoritmo.

Esempio con 5 processi

Usando gli stessi processi dell'esempio SJF non ottimale:

Process Arriv Durat | o | o | a | | :— | :— | :— | | P1 | 0 | 2 | | P2 | 0 | 4 | | P3 | 3 | 1 | | P4 | 3 | 1 | | P5 | 3 | 1 |

- $\mathrm{t}=0$: SRTN sceglie P1 (durata $2<4$). P1 termina a $\mathrm{t}=2$.
- $\mathrm{t}=2$: solo P 2 disponibile, viene schedulato. Tempo residuo di $\mathrm{P} 2=4$.
- $\mathrm{t}=3$: arrivano $\mathrm{P} 3, \mathrm{P} 4, \mathrm{P} 5$ (durata 1). Tempo residuo di $\mathrm{P} 2=3.1<3 \rightarrow$ prelazione. P3 viene schedulato. Termina a $\mathrm{t}=4$.
- $\mathrm{t}=4$: P4 (durata 1) vs P2 (residuo 3). $1<3 \rightarrow \mathrm{P} 4$. Termina a $\mathrm{t}=5$.
- $\mathrm{t}=5$: P5 (durata 1) vs P2 (residuo 3). $1<3 \rightarrow \mathrm{P5}$. Termina a $\mathrm{t}=6$.

- $\mathrm{t}=6$: rimane solo P2 (residuo 3). Termina a $\mathrm{t}=9$.
- Tempi di attesa: $\mathrm{P} 1=0, \mathrm{P} 2=2+3=5$ (ha aspettato da 0 a 2 per P 1 , poi da 3 a 6 per $\mathrm{P} 3, \mathrm{P} 4, \mathrm{P} 5$), $\mathrm{P} 3=0, \mathrm{P} 4=1, \mathrm{P} 5=2$.
- t.m.a.: $(0+5+0+1+2) / 5=\mathbf{1 . 6}$

Migliore sia dello SJF (2.8) che dell'ordine alternativo (2.6). Nota: i tempi di attesa si calcolano come: tempo di completamento - tempo di arrivo - durata.

Nota: il tempo totale non dipende dall'algoritmo

Indipendentemente dall'algoritmo usato, il tempo necessario per completare tutti i processi è sempre lo stesso. Lo scheduler può solo modificare l'ordine di esecuzione, influenzando i tempi di attesa e completamento dei singoli processi.

Stima della durata dei processi

Prevedere a priori quanto dura un processo non si può fare in modo esatto. Tuttavia, è possibile fare una stima statistica basata sullo storico dei processi (quanto tempo hanno impiegato in esecuzioni precedenti). Un tempo, nei vecchi sistemi batch, era il programmatore stesso a dover fornire un upperbound per la durata del processo, ma non era una stima particolarmente precisa.

Rischio di starvation

Un processo molto lungo potrebbe essere continuamente scavalcato da processi più brevi, senza mai ottenere la CPU → starvation. Per evitare questa situazione si possono adottare tecniche come l'aging (invecchiamento della priorità), che vedremo nel contesto degli algoritmi con priorità. Tuttavia, nel nostro modello ideale si ipotizza un set di processi chiuso (non arrivano continuamente nuovi processi brevi), quindi il problema non si pone.

Scheduling nei sistemi interattivi

Tutti i ragionamenti fatti finora per i sistemi batch non si possono adottare nei sistemi interattivi, dove la metrica fondamentale è il tempo di risposta, non il throughput o il turnaround. Servono approcci diversi.

Scheduling Round-Robin (RR)

Può essere visto come un'evoluzione del FCFS con l'aggiunta della prelazione. Lo scheduler assegna a ogni processo un quanto di tempo (time slice o

quantum) q, tipicamente tra 20 e 50 ms , che rappresenta il tempo massimo di utilizzo continuativo della CPU.

Funzionamento

I processi sono organizzati in una coda circolare. Lo scheduler prende il processo in testa alla coda e gli assegna la CPU. A questo punto ci sono due scenari possibili:

1. Il processo non termina entro il time slice: scade il quanto di tempo, scatta un interrupt del timer, e il SO effettua la prelazione. Il processo viene spostato in fondo alla coda dei processi pronti (con un nuovo quanto di tempo a disposizione), e la CPU viene assegnata al processo successivo in coda.
2. Il processo cede la CPU prima dello scadere del time slice: questo accade quando il processo esegue un'operazione di I/O (chiamata bloccante). Il processo passa allo stato di bloccato (non nella coda dei processi pronti, ma nella coda dei processi bloccati). Quando l'I/O si completa, il processo torna nella coda dei processi pronti. In alcuni sistemi, per favorire i processi I/O-bound, il processo che ritorna dall'I/O viene inserito in testa alla coda (davanti agli altri) anziché in fondo. Questa è un'euristica utile nei sistemi interattivi.

Proprietà temporali

Con n processi e un quanto di tempo q ms:

- Ogni processo ottiene circa $\mathbf{1} / \mathbf{n}$ della CPU.
- Nel caso peggiore, un processo dovrà attendere al massimo ($\mathbf{n - 1}$) $\times \mathbf{q}$ ms prima di ricevere nuovamente la CPU.

Questo upperbound influisce direttamente sulla reattività del sistema: più è piccolo, più il sistema è reattivo.

Il tradeoff del time slice

Ad ogni cambio di processo si paga il costo di un context switch, quantificabile in circa 2 ms . Quindi il tempo "utile" per ogni quanto non è q, ma q ms di lavoro seguiti da circa 2 ms di overhead.

Time slice piccolo (es. 20 ms): l'overhead percentuale è 2/22 $\approx 9 \%$. I context switch sono frequenti, quindi si spreca più CPU in overhead. Però il tempo di attesa nel caso peggiore è basso → alta reattività.

Time slice grande (es. 200 ms): l'overhead percentuale è $2 / 202 \approx 1 \%$. I context switch sono rari, l'overhead è minimo. Ma il tempo di attesa massimo cresce → peggiore reattività.

Questo è un tradeoff senza un valore ottimo universale: diminuendo q si favorisce la reattività ma si spreca più CPU in context switch; aumentando q si riduce l'overhead ma si peggiora la reattività.

Comportamento con processi CPU-bound e I/O-bound

La vita di un processo interattivo è scandita da CPU burst: anche se il processo ha la CPU tutta per sé, a un certo punto si interromperà per operazioni di I/O. Un processo I/O-bound tenderà a usare solo una parte del quanto di tempo, perché si bloccherà spesso in attesa di I/O, cedendo volontariamente la CPU. Un processo CPU-bound, invece, tenderà a usare l'intero quanto di tempo, fino a quando non viene interrotto dal timer.

Caso limite: cosa succede quando q tende all'infinito?

Se q tende all'infinito, il time slice diventa talmente grande che nessun processo verrà mai interrotto dal timer: ogni processo userà la CPU fino al completamento (o fino a un'operazione di I/O). In questo caso il Round-Robin degenera nel FCFS.

Lezione (16/04/26)

Scheduling a priorità

Idea di base

Non tutti i processi sono uguali e non devono essere trattati allo stesso modo. A ogni processo viene associata un'etichetta numerica (es. da 0 a 10) che ne indica la priorità. Bisogna stabilire una convenzione: in generale, un numero più grande indica una priorità più alta (servizio migliore).

La regola di base è semplice: si assegna la CPU al processo con priorità più alta. I processi a bassa priorità vengono schedulati solo quando non ci sono processi a priorità alta nella coda dei processi pronti.

Prelazione e priorità

Questo tipo di meccanismo prevede solitamente la prelazione: se un processo a bassa priorità è in esecuzione e nel frattempo entra nella coda dei processi pronti un processo a priorità più alta, la CPU viene tolta al processo con priorità bassa e assegnata a quello con priorità alta. Senza la prelazione, questo non

sarebbe possibile: il processo con bassa priorità terrebbe la CPU fino al completamento, vanificando il sistema di priorità.

Perché esistono processi con priorità diverse?

Un motivo tipico è la distinzione tra processi I/O-bound e CPU-bound. Come visto, conviene favorire i processi I/O-bound perché usano la CPU per poco tempo e la restituiscono rapidamente. Assegnando loro una priorità più alta, si ottiene un sistema più reattivo.

Assegnazione statica vs dinamica

Le priorità possono essere assegnate in modo statico (fissate alla creazione del processo e mai cambiate) o dinamico (ricalcolate durante l'esecuzione in base al comportamento del processo).

Un metodo di assegnazione dinamica consiste nel fare una statistica sull'uso dell'ultimo quanto di tempo. La priorità viene calcolata come l'inverso della percentuale di utilizzo del quanto di tempo:

- Un processo che ha usato il 100% del quanto → priorità $=1 / 1=1$ (bassa).
- Un processo che ha usato il 50% del quanto → priorità $=1 / 0.5=2$ (più alta).
- Un processo che ha usato il 10% del quanto → priorità $=1 / 0.1=10$ (molto alta).

In questo modo, meno un processo usa la CPU, più alta sarà la sua priorità. Questo favorisce naturalmente i processi I/O-bound. Il limite di questo approccio è che si basa su un'analisi puntuale (solo l'ultimo quanto di tempo): il fatto che un processo non abbia usato tutta la CPU potrebbe essere una coincidenza. È un sistema dinamico che "cambia idea" spesso, forse troppo spesso.

SJF come sistema a priorità

In realtà abbiamo già visto un sistema basato su priorità: lo SJF assegna implicitamente priorità più alta ai job più brevi. Il concetto di scheduling a priorità è una generalizzazione di quell'idea.

Problema: starvation

I processi a bassa priorità potrebbero non essere mai schedulati se i processi ad alta priorità si avvicendano continuamente nella coda dei processi pronti → starvation.

Soluzione: aging (invecchiamento)

L’aging è un’euristica per mitigare la starvation: la priorità di un processo che sta trascorrendo troppo tempo nella coda dei processi pronti viene aumentata temporaneamente. In questo modo, prima o poi, anche i processi a bassa priorità raggiungeranno una priorità sufficiente per essere schedulati. L’aumento è temporaneo: una volta che il processo ottiene la CPU e viene successivamente prelazionato, riparte dalla sua priorità originale nella coda dei processi pronti. Non è una soluzione “pulita” (è una “pezza”), ma funziona in pratica.

Scheduling a code multiple (classi di priorità)

Struttura

Variante dello scheduling a priorità. Invece di avere una singola coda dei processi pronti, si mantiene una coda separata per ogni livello di priorità. Ogni coda contiene i PCB dei processi appartenenti a quel livello. Le code sono gestite dinamicamente, tipicamente come liste doppiamente linkate.

Questo sistema è molto vicino a come funziona Windows.

Scheduler verticale e orizzontale

Il meccanismo di scelta si articola in due livelli: Scheduler verticale: decide da quale coda prelevare il prossimo processo. La regola è semplice: sceglie la coda non vuota con priorità più alta. Se la coda 4 (la più alta) è vuota, guarda la coda 3 , e così via.

Scheduler orizzontale: una volta scelta la coda, decide quale processo all’interno di quella coda schedulare. Dato che tutti i processi nella stessa coda hanno la stessa priorità, non ci sono “figli e figliastri”: sono tutti alla pari. Si usa tipicamente il Round-Robin con coda FIFO. L’algoritmo usato per ogni coda non è necessariamente lo stesso: il sistema permette di personalizzare l’algoritmo per ogni livello di priorità.

Dimensionamento dei quanti di tempo

La scelta del Round-Robin non è casuale: attraverso il quanto di tempo si può fare un tuning del comportamento (ovvero si può regolare il comportamento) per ogni coda.

Code ad alta priorità (processi I/O-bound): quanti di tempo piccoli. Questi processi hanno CPU burst brevi e non infastidiscono il sistema. Se il quanto di tempo è commisurato bene ai loro CPU burst, la prelazione verrà usata

raramente (perché il processo cederà la CPU spontaneamente per I/O prima che scada il quanto). Il quanto di tempo piccolo aumenta l'interattività.

Code a bassa priorità (processi CPU-bound): quanti di tempo grandi. Questi processi vorrebbero usare la CPU a lungo senza fare I/O. In un sistema interattivo sono una classe "strana" - per esempio un processo di mining o una computazione lunga di un progetto personale. Conviene metterli nella coda più bassa, dove useranno la CPU lasciata libera da tutti gli altri. Per ottimizzare l'uso delle risorse, i processi in queste code assomigliano molto ai processi dei sistemi batch. Nella coda a priorità più bassa si potrebbe anche usare un FCFS. Idealmente, asintoticamente, il quanto di tempo nella coda più bassa potrebbe essere infinito.

Se nel frattempo compaiono processi nelle code più alte, lo scheduler verticale darà sempre la priorità a quei processi.

Code con feedback (retroazione)

Il sistema a code multiple può prevedere un meccanismo di downgrade e upgrade delle priorità: la priorità di un processo può cambiare nel tempo. In questo caso si parla di code multiple con feedback (o retroazione).

Quando un processo viene rimesso nella coda dei processi pronti (dopo una prelazione o dopo il completamento di un'operazione di I/O), si osserva come ha usato il suo ultimo quanto di tempo (o meglio, il trend degli ultimi quanti) per decidere in quale coda reinserirlo:

- Se il trend mostra che il processo usa tutto il suo quanto di tempo, probabilmente quel quanto gli sta stretto: il processo è più CPU-bound di quanto si pensava. Viene spostato nella coda immediatamente inferiore (priorità -1), dove avrà un quanto di tempo più grande.
- Se il trend mostra che il processo usa poco del suo quanto di tempo, probabilmente è più I/O-bound di quanto si pensava. Viene spostato nella coda immediatamente superiore (priorità +1), dove avrà un quanto di tempo più piccolo e maggiore reattività.

In questo modello non ci sono priorità di "base" fissate per sempre. Si sceglie una priorità iniziale (es. 3) che viene assegnata a tutti i nuovi processi appena creati, e poi il sistema la adatta dinamicamente. I sistemi reali fanno esattamente così.

Altro meccanismo anti-starvation: percentuali di utilizzo

Un’alternativa all’aging per evitare la starvation è assegnare a ogni coda una percentuale di utilizzo garantita della CPU.

Il sistema suddivide un periodo di tempo fisso (es. 4 secondi) tra le code secondo percentuali prestabilite. Per esempio: 50% alla coda 4 (priorità più alta), 30% alla coda $3,15 \%$ alla coda $2,5 \%$ alla coda 1 (priorità più bassa).

Lo scheduler opera così: per il primo 50% del periodo dà la CPU ai processi della coda 4. Esaurita quella percentuale, passa alla coda 3 per il 30%, poi alla coda 2 per il 15%, e infine alla coda 1 per il 5%. Se una coda è vuota, la sua percentuale viene saltata e il periodo complessivo dura meno. Il tempo non utilizzato non si accumula.

Questo evita la starvation perché anche in una situazione caotica (tutte le code piene) i processi a bassa priorità sono garantiti di usare la CPU per almeno la loro percentuale. Se non ci sono processi nelle altre code, la percentuale della coda a bassa priorità verrà ripetuta più volte.

Problema (intenzionale): con la percentuale di utilizzo garantita, si annulla il meccanismo della prelazione se la percentuale di utilizzo si è esaurita.

Altro problema: se la percentuale di utilizzo della coda a priorità più alta non è stata usata tutta, e siamo nella coda a priorità più bassa, e nel frattempo arriva un processo con priorità max, la prelazione scatta o no? Dipende dal sistema. Tipicamente sì, perché la reattività è più importante della garanzia anti-starvation (che in questo caso viene indebolita) in un sistema interattivo.

Shortest Process Next (SPN)

Idea

Applicare la logica dello SJF ai sistemi interattivi. Se nella coda dei processi pronti avessimo un’etichetta che ci dice “questo processo, appena lo scheduli, farà un CPU burst di $10 \mathrm{~ms}^{\prime \prime}$, potremmo schedulare prima i processi con il CPU burst più breve. Se un processo promette di usare la CPU per pochissimo tempo, gli si dà la precedenza.

Problema: identificare la durata del prossimo CPU burst

Nei sistemi interattivi non si sa a priori quanto durerà il prossimo CPU burst. Guardare solo l’ultimo CPU burst può essere fuorviante: magari c’è stato uno spike anomalo. Conviene fare una media pesata degli ultimi CPU burst.

Formula della media esponenziale

$\$$ S_ $n+1\}=$ S_n $(1-a)+T _n \cdot a \$$ Dove:

- $\$$ S_{n+1}$: è la stima per il prossimo CPU burst (ciò che vogliamo calcolare).
- $\$ \mathrm{~S} _\mathrm{n} \$$ è la vecchia stima (basata sullo storico precedente).
- T_nè l'ultima misurazione reale del CPU burst.
- a è un parametro tra 0 e 1 (tipicamente non tocca gli estremi), utile per stabilire il peso delle due parti della formula.

La stima è una combinazione tra l'ultimo valore reale misurato e la stima precedente. La vecchia stima serve a tenere d'occhio come si è comportato il processo in un passato più remoto, smorzando eventuali picchi temporanei.

Effetto del parametro a

- a piccolo (vicino a 0): più peso alla vecchia stima S_n. La stima è "conservativa" e reagisce lentamente ai cambiamenti. Picchi isolati vengono quasi ignorati.
- a grande (vicino a 1): più peso all'ultima misurazione reale T_n. La stima è "reattiva" e si adatta rapidamente, ma è più sensibile a spike anomali.
- $\mathbf{a}=\mathbf{0}$: S n+1=S n$. La stima non cambia mai, l'ultimo burst reale viene ignorato completamente.
- $\mathbf{a}=\mathbf{1}$: Sn $+1=$ Tn. La stima è esattamente l'ultimo burst, senza alcuna memoria del passato.

La stima (linea rossa nel grafico della slide) "insegue" i valori reali (linea blu), ma con un ritardo: si adatta, ma lentamente. È una stima a un passo avanti: deve indovinare in che posizione andrà la linea dei valori reali.

Utilizzo pratico

Non esistono sistemi reali che utilizzino esattamente solo questo metodo in questa forma. Tuttavia, l'idea della media esponenziale è alla base di molte euristiche usate negli scheduler reali. Il parametro a è tipicamente fisso.

Scheduling garantito

Idea

Il sistema fa delle promesse ai processi in termini di percentuale di utilizzo della CPU, e poi cerca di mantenerle. Con n processi, la promessa base è che

ognuno riceverà 1/n della CPU.

L’algoritmo assegna la CPU al processo che è più indietro rispetto alle promesse che gli erano state fatte. Questo favorisce naturalmente i processi I/O-bound: dato che questi per loro natura cedono spesso la CPU (a causa di operazioni di I/O), saranno sempre “indietro” rispetto alla loro quota promessa rispetto ai processi CPU-bound. Il mancato rispetto delle promesse non dipende dal SO ma dal comportamento dei processi stessi.

Equità reale vs apparente

Nei processi I/O-bound, con un quanto di tempo uguale per tutti, la percentuale di utilizzo effettiva della CPU è molto bassa (perché si bloccano spesso). Lo scheduling garantito tiene conto di questo e cerca di compensare.

CFS di Linux

Lo scheduler CFS (Completely Fair Scheduler, scheduler completamente equo) di Linux è molto vicino a quest’idea. La sua caratteristica principale è che traccia per quanto tempo ogni processo ha effettivamente usato la CPU attraverso il virtual runtime (vruntime): un contatore che misura il tempo di CPU consumato da ciascun processo. Lo scheduler assegna la CPU al processo con il vruntime più basso, cioè quello che è più indietro rispetto alla sua quota.

Su Linux non esiste il concetto di quanto di tempo come in Windows e macOS: la decisione di scheduling si basa sul virtual runtime. Questa è una differenza architetturale significativa.

Nota: in CFS ogni processo ha un valore di nice (da-20 a +19), che determina il suo “peso”. Un processo con nice basso ha peso alto, e viceversa. Il vruntime non cresce alla stessa velocità per tutti. Un processo con peso alto lo accumula più lentamente di un processo con peso basso. Lo scheduler sceglie sempre il processo col vruntime più basso. Il risultato è che i processi con nice basso ottengono più CPU. Questo perché il contatore avanza più lentamente e quindi restano in vantaggio più a lungo.

Scheduling a lotteria

Meccanismo

A ogni processo vengono assegnati dei biglietti (ticket). A ogni decisione di scheduling, il sistema estrae un biglietto a caso: il processo che possiede quel biglietto ottiene la CPU. Un processo con più biglietti ha una probabilità proporzionalmente maggiore di essere scelto.

Flessibilità

Questo meccanismo è molto flessibile nella strategia di assegnazione dei biglietti:

- Equità perfetta: se ci sono n processi, si assegnano a ciascuno lo stesso numero di biglietti → ognuno ha probabilità 1/n di ottenere la CPU.
- Priorità: si assegnano più biglietti ai processi che si vogliono favorire (anche dinamicamente).
- Processi cooperanti: un processo padre può cedere i propri biglietti a un processo figlio (es. se il figlio deve completare un'operazione importante). Quando il figlio termina, i biglietti tornano al padre.

Quando tutti i biglietti di un ciclo sono stati estratti, si riparte da capo con una nuova distribuzione.

Il criterio è semplice, chiaro e statisticamente garantisce le percentuali di utilizzo promesse sul lungo periodo. I dettagli implementativi (come funziona la prelazione in questo contesto, ecc.) sono aspetti collaterali.

Scheduling fair-share

Il problema dell'equità tra utenti

Tutti gli scheduler visti finora garantiscono equità tra processi, ma non tra utenti. In un sistema multi-utente, ogni utente può avere un numero diverso di processi. Se un utente ha 9 processi e un altro ne ha 1 , uno scheduler equo tra processi assegnerà il 90% della CPU al primo utente e il 10% al secondo. I processi sono trattati equamente, ma gli utenti no.

Soluzione: gruppi di processi

Lo scheduling fair-share (quota equa) introduce il concetto di equità tra utenti (o gruppi di utenti). I processi vengono raggruppati in base alla loro appartenenza (tipicamente all'utente proprietario), e lo scheduler distribuisce la CPU equamente tra i gruppi, non tra i singoli processi.

Su Linux questo meccanismo è implementato attraverso i cgroup (control groups): strutture del kernel che permettono di raggruppare processi e assegnare a ciascun gruppo una quota di risorse (CPU, memoria, I/O, ecc.). I cgroup sono presenti nel kernel ma sono disattivati di default - vanno configurati esplicitamente dall'amministratore di sistema.

Tutti i sistemi operativi moderni implementano almeno in parte quest'idea di equità tra utenti/gruppi.